蓝牙打印机实战

手把手敲代码

——会 STM32 的人，15 级台阶写出 ESP32 打印机——

适用硬件：ESP32-WROOM-32E + JX-2R-01/JX-700-48 兼容打印头

软件框架：Arduino-ESP32 3.3.12（PlatformIO espressif32@8.0.0）

通信方式：经典蓝牙 SPP，兼容喵喵机公版 App

文档版本：v2.0

编写日期：2026-11

15 个分级工程均在本机实测编译通过，

书中的 RAM / Flash 占用与耗时都是真实构建结果，可直接对照。

目　　录

前言 4
为什么这样编排 4
怎么用这本书 4
你需要的硬件 4
第 1 章 从 STM32 到 ESP32：12 个必须改掉的直觉 6
1.1 没有“外设初始化结构体” 6
1.2 时钟不用你配，但你可以改 6
1.3 定时器不是 TIM2/TIM3，是“你要哪种定时器” 7
1.4 中断：能做的事更少 7
1.5 内存分三块，malloc 可能失败 7
1.6 没有 HAL，只有“Arduino + IDF 两层” 8
1.7 烧录方式完全不同 8
1.8 双核：你有一个“第二 CPU”没用 8
1.9 RTOS 是默认存在的，不是可选项 9
1.10 看门狗有两层，默认可能已经开着 9
1.11 低功耗不是 WFI，是 deep sleep 9
1.12 蓝牙是内置的，但“经典”和“低功耗”是两套东西 10
第 2 章 环境：一次配好，后面不再折腾 11
2.1 只需要装两样东西 11
2.2 platformio.ini 逐行解释 11
2.3 三个命令走天下 12
2.4 常见编译/烧录错误 12
2.5 目录结构约定 12
第 3 章 S00 · 点灯与串口：确认你真的连上了 13
3.1 本节目标 13
3.2 先动手 13
3.3 逐行拆解 14
3.4 烧录后该看到什么 14
3.5 验收清单 15
3.6 卡住了看这里 15
3.7 与上一级的 diff 15
第 4 章 S01 · 按键：把抖动和长按一次解决 16
4.1 本节目标 16
4.2 先动手 16
4.3 逐行拆解 18
4.4 烧录后该看到什么 18
4.5 验收清单 18
4.6 卡住了看这里 19
4.7 与上一级的 diff 19
第 5 章 S02 · 自建 shell：给自己造一个“串口控制台” 20
5.1 本节目标 20
5.2 先动手 20
5.3 逐行拆解 23
5.4 烧录后该看到什么 24
5.5 验收清单 24
5.6 卡住了看这里 25
5.7 与上一级的 diff 25
第 6 章 S03 · 打印头移位：48 字节怎么走进去 26
6.1 本节目标 26
6.2 先动手 26
6.3 逐行拆解 28

6.4 烧录后该看到什么 31
6.5 验收清单 31
6.6 卡住了看这里 32
6.7 与上一级的 diff 32
第 7 章 S04 · 第一次加热：一个点、一条线、一次危险 33
7.1 本节目标 33
7.2 先动手 33
7.3 逐行拆解 35
7.4 烧录后该看到什么 36
7.5 验收清单 36
7.6 卡住了看这里 36
7.7 与上一级的 diff 37
第 8 章 S05 · 步进电机：让纸精确地走 0.125mm 38
8.1 本节目标 38
8.2 先动手 38
8.3 逐行拆解 40
8.4 烧录后该看到什么 40
8.5 验收清单 41
8.6 卡住了看这里 41
8.7 与上一级的 diff 41
第 9 章 S06 · 打出第一条完整的行 42
9.1 本节目标 42
9.2 先动手 42
9.3 逐行拆解 45
9.4 烧录后该看到什么 46
9.5 验收清单 46
9.6 卡住了看这里 46
9.7 与上一级的 diff 47
第 10 章 S07 · 画布：在内存里画好，再打出去 48
10.1 本节目标 48
10.2 先动手 48
10.3 逐行拆解 52
10.4 烧录后该看到什么 53
10.5 验收清单 53
10.6 卡住了看这里 53
10.7 与上一级的 diff 54
第 11 章 S08 · 传感器：缺纸、电池、温度 55
11.1 本节目标 55
11.2 先动手 55
11.3 缺纸检测：一条电平，五个坑 59
11.4 电池电压采样：一个分压，三个公式 60
11.5 打印头 NTC：B 值方程的来龙去脉 64
11.6 ESP32 的 ADC 到底什么水平 67
11.7 逐行拆解 68
11.8 烧录后该看到什么 68
11.9 验收清单 69
11.10 卡住了看这里 69
11.11 与上一级的 diff 70
第 12 章 S09 · 经典蓝牙：让手机看见你 71
12.1 本节目标 71
12.2 先动手 71
12.3 逐行拆解 73
12.4 烧录后该看到什么 74
12.5 验收清单 74
12.6 卡住了看这里 74
12.7 与上一级的 diff 74

第 13 章 S10 · 协议帧：把字节流切成命令 75
13.1 本节目标 75
13.2 先动手 75
13.3 逐行拆解 78
13.4 烧录后该看到什么 80
13.5 验收清单 80
13.6 卡住了看这里 81
13.7 与上一级的 diff 81
第 14 章 S11 · 握手：让公版 App 认你 82
14.1 本节目标 82
14.2 先动手 82
14.3 逐行拆解 86
14.4 烧录后该看到什么 87
14.5 验收清单 88
14.6 卡住了看这里 88
14.7 与上一级的 diff 88
第 15 章 S12 · 打印数据通路：App 发图，机器出纸 89
15.1 本节目标 89
15.2 先动手 89
15.3 逐行拆解 90
15.4 烧录后该看到什么 91
15.5 验收清单 91
15.6 卡住了看这里 92
15.7 与上一级的 diff 92
第 16 章 S13 · 可靠性：扔进包里一小时不出事 93
16.1 本节目标 93
16.2 先动手 93
16.3 逐行拆解 94
16.4 烧录后该看到什么 95
16.5 验收清单 96
16.6 卡住了看这里 96
16.7 与上一级的 diff 96
第 17 章 S14 · 整机自检：一页纸看出好坏 97
17.1 本节目标 97
17.2 先动手 97
17.3 逐行拆解 99
17.4 烧录后该看到什么 100
17.5 验收清单 100
17.6 卡住了看这里 101
17.7 与上一级的 diff 101
第 18 章 从样机到产品 102
18.1 你现在有什么 102
18.2 四条“产品化”必修 102
18.3 五个可继续做的方向 102
18.4 这本书的三条主线 103
18.5 写在最后 103
附录 A · 引脚总表 104
附录 B · 指令速查卡 105
附录 C · 排错总表 106
附录 D · 实测数据 107
D.1 十五级资源占用（espressif32@8.0.0 + huge_app.csv） 107
D.2 打印性能 107
D.3 关键实测帧 107
D.4 尺寸与重量参考 108

前言

这本书是给“会 STM32、要第一次碰 ESP32”的人写的。

它不解释热敏打印机为什么能发热（那在《蓝牙热敏打印机开发教程》里），也不讲协议设计的来龙去脉。它只回答一个问题：**面对一块空白的 ESP32，从零开始，代码一行一行怎么敲，每一步怎么知道自己敲对了。**

为什么这样编排

硬件项目最常见的失败不是“不懂原理”，而是“不知道自己在第几步”。所以这本书把整机拆成 15 级台阶，每一级都满足三个条件：

- 有一个**可以立刻看到的物理现象**（灯亮、纸动、手机连上、打出黑条）；
- 只引入**一个新概念**，其余都是上一级已经验证过的；
- 编译通过的数字是真实的（RAM/Flash 占用、耗时），你可以拿自己的结果对照。

每一级都是**独立的 PlatformIO 工程**，放在 `stages/s00` … `stages/s14`。想跳着看可以直接打开对应目录编译烧录，不需要先补前面几级——除了 S07 用到 S06 的打印函数、S08 之后用到前面的模块，工程里都已经带好了。

怎么用这本书

1. 先看第 1 章的“12 个不同”。它把你在 STM32 上形成的直觉逐条改写，省掉大量“为什么这样不行”的试错。
2. 按顺序做 S00 → S14。每级先照着“先动手”把代码敲进去（建议真的手敲，不要复制），再看“逐行拆解”。
3. 每级末尾的“验收清单”逐条打勾。有一条不通过就别往下走——后面所有现象都会变成噪声。
4. 卡住时看“卡住了看这里”，那里列的都是这一步真实会遇到的坑。

你需要的硬件

物品	说明	参考价
ESP32 开发板	ESP32-WROOM-32E 模组，38/39 脚，带 USB 口	20 元
热敏打印头	JX-2R-01 / JX-700-48 兼容，384 点，带 FPC 排线	25 元
步进电机	两相四线，步距角 9°，带 0.0625mm 减速齿轮	15 元
打印头升压模块	或整机板（本项目原理图那版）	30 元

物品	说明	参考价
热敏纸	58mm 宽（打印宽度 48mm）	5 元
USB 线	要能传数据，不是纯充电线	5 元

强烈建议先用整机板（5，原理图 那版）。它的电源、升压、电机驱动、缺纸检测都已经接好，你只需要写代码；自己从零搭电路，光是 8.4V 升压和打印头排线就能耗掉一周。

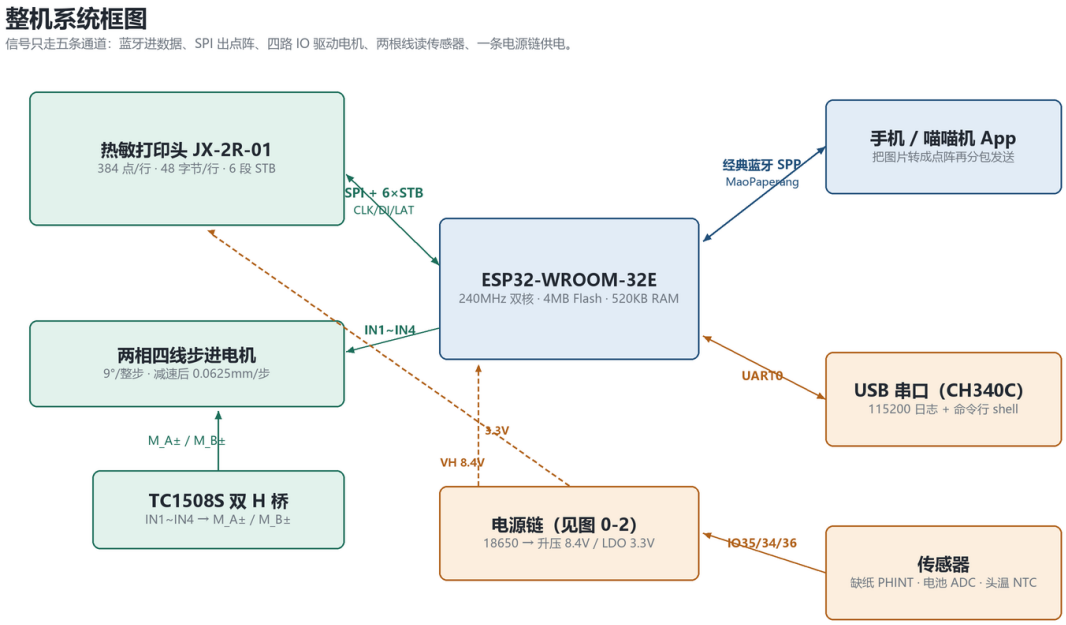


图 0-1 整机系统框图：ESP32 居中，左边打印头与电机，右边手机 App

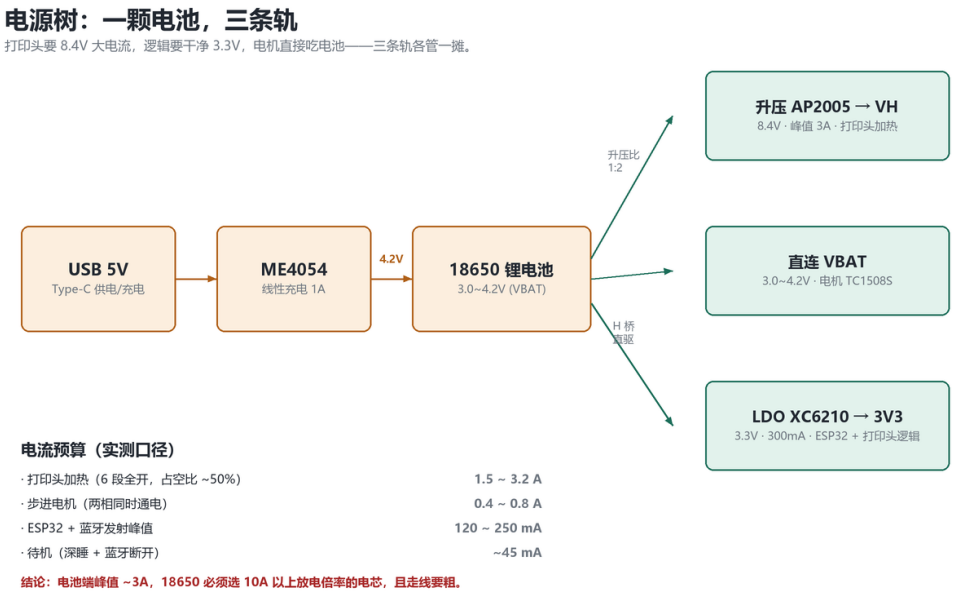


图 0-2 电源树：电池 → 升压 8.4V 供打印头，LDO 3.3V 供逻辑

> 一句话原则：**第一周只验证软件，硬件用现成的板子。** 等 S12 能出纸了，再回头研究电路。

第 1 章 从 STM32 到 ESP32：12 个必须改掉的直觉

你在 STM32 上的经验 80% 都能用：中断、定时器、SPI、状态机、DMA 的思想完全一样。但剩下 20% 会让你怀疑人生，因为它们**没有报错，只是不工作**。

下面 12 条，按“最容易踩”的顺序排列。

1.1 没有“外设初始化结构体”

STM32 上你写 `HAL_GPIO_Init(GPIOA, &GPIO_InitStruct)`，一堆 `Mode/Pull/Speed/Alternate`。ESP32 的 Arduino 框架里，这些被压成一行：

```
pinMode(18, OUTPUT);            // 等价于 GPIO_InitStruct 的一整套
digitalWrite(18, LOW);
```

代价是：**你失去了对“复用功能编号”的显式控制**。ESP32 没有 `AF5 = SPI1` 这种编号，它有一张 162 输入的交换矩阵（GPIO Matrix），任何外设信号可以路由到几乎所有引脚。所以：

- 好处：SPI 的 CLK 想用 IO15 就用 IO15，不用查手册哪几个脚能复用；
- 坑：某些引脚**出厂就被占用**（IO6~IO11 接 Flash），或者**启动时决定芯片模式**（strapping 脚），你复用它们会导致“程序烧不进去”或“上电就进下载模式”。

引脚	出厂用途	能不能随便用
IO6~IO11	内部 Flash	**绝对不能**
IO34~IO39	只输入，无内部上下拉	可做 ADC/按键，不能输出
IO0	strapping：低电平进下载模式	可做输出，但上电时别拉低
IO2	strapping：影响启动日志	可做输出，别接强下拉
IO12	strapping：影响 Flash 电压	可做输出，别接强上拉
IO15	strapping：影响启动日志	同上
IO16/IO17	默认接 PSRAM（有的板子）	可用，但先确认板子
IO1/IO3	UART0 下载/日志	调试时别占用

> 记忆法：**0、2、12、15 是“上电姿势脚”**，34~39 是“只能看的脚”，6~11 是“别碰的脚”。

1.2 时钟不用你配，但你可以改

STM32 上你要开 RCC、配 PLL、设分频。ESP32 的 `setup()` 跑起来时，CPU 已经是 240MHz，APB 已经是 80MHz，外设时钟门控也开好了。

你要做的只有两件事：

```
setCpuFrequencyMhz(80);      // 降频省电：240 -> 80，电流约降 40%
```

```
uint32_t f = getCpuFrequencyMhz();
```

以及**知道每个外设挂在哪个时钟域**：SPI 挂在 APB（80MHz），所以 `SPISettings(1000000, ...)` 的 1MHz 是从 80MHz 分频来的，实际误差可以忽略；但如果你写 80MHz，就会被静默限制。

1.3 定时器不是 TIM2/TIM3，是“你要哪种定时器”

STM32 上定时器就是 TIMx。ESP32 上有四种，用途完全不同：

你要做的事	用什么	说明
微秒级精确延时	`delayMicroseconds()` / `esp_timer`	忙等，别在中断里用长的
毫秒级周期任务	`millis()` 轮询 或 `Ticker`	本书全部用 millis 轮询，最简单也最稳
硬件 PWM	`ledcSetup/ledcAttachPin`（旧）/ `ledcAttach`（新）	16 路，可调频率和分辨率
精确定时中断	`esp_timer_create` + `esp_timer_start_periodic`	微秒级，跑在独立任务里

打印机的加热时间是 800μs 级别，用 `delayMicroseconds()` 就够——因为这段时间你本来也不能干别的（打印头正在烧）。别为了“优雅”上定时器中断，只会引入抖动。

1.4 中断：能做的事更少

STM32 的中断服务函数里可以干不少活。ESP32 的 ISR 有严格限制：

- 必须加 `IRAM_ATTR`，否则中断触发时如果 Flash 正在写，会崩；
- 不能调用 `Serial.print`、`malloc`、`delay`；
- 只能置标志、给队列发消息、操作寄存器。

```
volatile bool g_btn_flag = false;
void IRAM_ATTR on_btn() { g_btn_flag = true; }        // 只做这一件事
```

然后在 `loop()` 里处理。**本书的按键不用中断**，用 5ms 轮询消抖——对这个应用足够，而且好调试。

1.5 内存分三块，malloc 可能失败

STM32 上你基本不关心堆。ESP32 上：

区域	大小（本板）	用途
DRAM	~320KB	全局变量、堆、栈
IRAM	~128KB	中断代码、高频代码

区域	大小（本板）	用途
Flash	4MB	代码 + 文件系统 + OTA

蓝牙协议栈启动会吃掉约 60KB DRAM。所以：

- 大缓冲区（一页位图 17KB）**一定要判 `malloc` 返回值**；
- 栈默认 8KB，递归和大局部数组会栈溢出（现象是重启，日志 `Stack canary watchpoint triggered`）；
- `ESP.getFreeHeap()` 是你的好朋友，怀疑内存问题时先打它。

1.6 没有 HAL，只有“Arduino + IDF 两层”

你写的是 Arduino 风格（`digitalWrite`、`SPI`、`Serial`），底层其实是 ESP-IDF。两层可以混用：

```
#include <driver/ledc.h>          // 直接用 IDF 的 LEDC
#include <esp_task_wdt.h>         // 直接用 IDF 的看门狗
```

Arduino-ESP32 3.x 已经把这层关系整理得比较干净（IDF 5.x 底座）。**遇到 Arduino API 不够用时，直接调 IDF**，不要自己写寄存器。

1.7 烧录方式完全不同

	STM32	ESP32
调试器	ST-Link / J-Link	一般不用，串口即可
下载口	SWD（SWCLK/SWDIO）	UART0（TX/RX）+ 自动复位电路
进入下载	调试器自动	DTR/RTS 拉 EN/IO0（板上有电路）
工具	Keil/IAR/CubeIDE	`pio run -t upload` / `esptool.py`
看日志	SWO/printf 重定向	同一个串口，115200

所以 ESP32 的“调试”就是**串口打印**。这也是本书大量使用 shell 命令的原因——你只能用 printf 观测。

> 烧录失败三件套：① 换一根能传数据的 USB 线；② 按住 BOOT 再点下载；③ 把 `upload_speed` 从 921600 降到 115200。

1.8 双核：你有一个“第二 CPU”没用

ESP32 有两个核。Arduino 的 `loop()` 默认跑在 core 1，core 0 跑 WiFi/蓝牙协议栈。

```
xTaskCreatePinnedToCore(print_task, "print", 8192, NULL, 5, NULL, 1);
```

这一行就把打印任务钉在 core 1 上，蓝牙中断在 core 0，互不打扰。**打印一页要 8 秒，绝不能放在 `loop()` 里阻塞**——否则蓝牙断连、看门狗复位。

1.9 RTOS 是默认存在的，不是可选项

STM32 上你可以裸机。ESP32 上你已经在 FreeRTOS 里了：`loop()` 本身就是一个任务的函数体，`delay()` 会让出 CPU。

概念	STM32 裸机	ESP32
主循环	`while(1)`	`loop()`（被 RTOS 调用）
延时	`HAL_Delay` 死等	`vTaskDelay` 让出
任务间通信	全局变量 + 关中断	队列/信号量/事件组
栈	启动文件里定	每任务独立，可指定

本书只用两样：`xQueueSend/xQueueReceive`（蓝牙收到数据 → 打印任务）和 `xTaskCreatePinnedToCore`。

1.10 看门狗有两层，默认可能已经开着

看门狗	说明
RTC WDT	上电自检用，启动后关
Task WDT (TWDT)	监视任务，超时不喂就复位
Interrupt WDT	监视中断，默认 300ms

`loop()` 里如果写了 `while (!SerialBT.available()) ;`，8 秒后芯片复位，日志是 `rst:0x8 (TG1WDT_SYS_RST)`。这不是 bug，是保护。**正确做法是状态机 + 非阻塞**，本书所有代码都遵守。

1.11 低功耗不是 WFI，是 deep sleep

STM32 的 STOP 模式还能保留 RAM 和状态。ESP32 的 `esp_deep_sleep_start()` 是**几乎全关**：RAM 丢失、程序从头跑、唤醒相当于复位。

```
esp_sleep_enable_ext0_wakeup(GPIO_NUM_0, 0);   // IO0 低电平唤醒
esp_deep_sleep_start();
```

所以“自动关机”不能靠 deep sleep 保存状态——醒来就是重新 `setup()`。本书 S13 的做法：关机前不需要保存任何东西，因为打印机没有需要保留的会话。

1.12 蓝牙是内置的，但“经典”和“低功耗”是两套东西

	经典蓝牙 (BT Classic)	BLE
用途	串口透传 SPP	低功耗传感器
库	`BluetoothSerial`	`BLEDevice/BLEServer`
手机兼容	安卓原生支持 SPP	需要 App
内存	约 60KB	约 30KB
本项目	**用这个**	不用

喵喵机 App 用的是经典蓝牙 SPP（UUID `00001101-...`），所以必须用 `BluetoothSerial`。它在新版 Arduino-ESP32 里被标记为 `deprecated`（3.x 仍可用，4.0 默认关闭），编译时会看到警告——**这是正常的**，S09 会专门讲怎么处理。

> 一个 ESP32 同时只能选一种蓝牙角色。想同时支持 BLE 配网 + 经典打印，得用双模库或换芯片（ESP32-S3 不支持经典蓝牙）。

第 2 章 环境：一次配好，后面不再折腾

2.1 只需要装两样东西

1. **VS Code**（不是 VS，别下错）；
2. **PlatformIO IDE 插件**（在 VS Code 扩展里搜 `PlatformIO IDE`）。

   不需要装 Arduino IDE，不需要装 ESP-IDF，不需要 Python（PlatformIO 自带一套）。

   装完后 VS Code 左侧会出现一个蚂蚁图标，点开就是 PlatformIO 主页。

2.2 platformio.ini 逐行解释

本书每个 stage 的 `platformio.ini` 都是同一份：

```
[env:esp32dev]
platform = espressif32@8.0.0
board = esp32dev
framework = arduino

monitor_speed = 115200
upload_speed  = 460800

board_build.flash_mode = dio
board_build.partitions = huge_app.csv

build_flags = -DCORE_DEBUG_LEVEL=0
```

行	含义	为什么这么写
`platform = espressif32@8.0.0`	锁定平台版本	不锁的话某天自动升级，`BluetoothSerial` 行为可能变
`board = esp32dev`	通用 ESP32 开发板定义	4MB Flash、无 PSRAM 的板子都用它
`framework = arduino`	用 Arduino-ESP32 3.3.12	有 `BluetoothSerial`、`SPI`、`ledc`
`monitor_speed = 115200`	串口监视器波特率	和代码里 `Serial.begin(115200)` 必须一致
`upload_speed = 460800`	烧录波特率	921600 有些线材会失败，460800 稳
`flash_mode = dio`	Flash 访问模式	兼容性最好；qio 更快但个别模组不稳
`partitions = huge_app.csv`	3MB 应用分区	默认 1.2MB 装不下蓝牙程序（实测 84%）
`CORE_DEBUG_LEVEL=0`	关掉 Arduino 内核日志	否则串口被 `[D][...]` 刷屏

> 关于分区：默认 `default.csv` 给应用 1310720 字节，S09 之后 Flash 占用 1111692，只剩 15%，随便加点东西就爆。`huge_app.csv` 给 3145728 字节，从此不用再管。

2.3 三个命令走天下

在 PlatformIO 的终端（或系统终端，先激活环境）里：

```
pio run                 # 编译
pio run -t upload       # 编译 + 烧录
pio device monitor      # 串口监视（Ctrl+] 退出）
```

本书所有编译结果都是这样得到的，例如 S00：

```
RAM:   [=         ]   6.8% (used 22172 bytes from 327680 bytes)
Flash: [          ]   9.4% (used 294312 bytes from 3145728 bytes)
======================== [SUCCESS] Took 27.31 seconds ========================
```

看到 `SUCCESS` 和这两个百分比，才算这一步真的过了。 数字对不上说明你改过代码或配置。

2.4 常见编译/烧录错误

现象	原因	解决
`Failed to connect to ESP32: Timed out waiting for packet header`	没进下载模式	按住 BOOT 再点上传，或降 upload_speed
`A fatal error occurred: Could not open port COMx`	串口被占用	关掉串口监视器/其它串口工具
`error: 'BluetoothSerial' was not declared`	平台版本太新（4.x）或 board 不支持	锁 `espressif32@8.0.0`
`region 'iram0_0_seg' overflowed`	代码太多/加了 IRAM_ATTR	去掉多余的 IRAM_ATTR
编译很久（>3 分钟）	首次下载工具链	正常，第二次 20~30 秒
上传成功但没反应	板子没复位/波特率不对	按一下 EN，或检查 `monitor_speed`

2.5 目录结构约定

```
s10/
├── platformio.ini
├── include/          # 头文件（.h）
│   ├── config.h
│   ├── frame.h
│   └── crc32.h
└── src/              # 源文件（.cpp）
    ├── main.cpp
    ├── frame.cpp
    └── crc32.cpp
```

`include/` 和 `src/` 都会自动加入编译，不用改任何配置。**不要**把 .cpp 放进 `include/`，PlatformIO 不会编译它，你会得到一堆“未定义引用”。

第 3 章 S00 · 点灯与串口：确认你真的连上了

> 台阶 S00 · 目标产物：`stages/s00` · 实测 RAM 22172 (6.8%) / Flash 294312 (9.4%)

3.1 本节目标

让板子上的 LED 以 500ms 周期闪烁，并在串口打印出芯片信息。听起来像“Hello World”，但它同时验证了四件事：**供电正常、下载成功、代码在跑、串口能通**。

这四件事任何一件不成立，后面 14 级全是白费。

3.2 先动手

1. 用 USB 线把板子连到电脑。
2. 新建工程（PlatformIO → New Project → Board: `ESP32 Dev Module`，Framework: `Arduino`），或者直接打开 `stages/s00`。
3. 把下面两份文件敲进去。
4. `pio run -t upload`，然后 `pio device monitor`。

`include/config.h`：

```
#pragma once
// 全工程共享的引脚定义，改板子只改这里
#include <Arduino.h>

#define PIN_LED   18       // 板载 LED，低电平点亮
#define PIN_BTN    5       // 按键，按下为低
```

`src/main.cpp`：

```
#include <Arduino.h>
#include "config.h"

static uint32_t s_last = 0;
static bool     s_on   = false;

void setup() {
  Serial.begin(115200);
  delay(200);                       // 等 USB 串口枚举完成

  Serial.println();
  Serial.println("=== STAGE 00 · blink + serial ===");
  Serial.printf("chip   : %s rev%d, %d cores @ %u MHz\n",
                ESP.getChipModel(), ESP.getChipRevision(),
                ESP.getChipCores(), ESP.getCpuFreqMHz());
  Serial.printf("flash  : %u KB\n", ESP.getFlashChipSize() / 1024);
  Serial.printf("heap   : %u bytes free\n", ESP.getFreeHeap());

  uint64_t mac = ESP.getEfuseMac();     // 6 字节 MAC，低 48 位有效
  Serial.printf("mac    : %04X%08X\n", (uint16_t)(mac >> 32), (uint32_t)mac);
```

```
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, HIGH);          // 高电平 = 灭
}

void loop() {
  uint32_t now = millis();
  if (now - s_last >= 500) {            // 非阻塞定时
    s_last = now;
    s_on = !s_on;
    digitalWrite(PIN_LED, s_on ? LOW : HIGH);
    Serial.printf("[%6lu ms] led %s\n", (unsigned long)now, s_on ? "ON" : "off");
  }
}
```

3.3 逐行拆解

`Serial.begin(115200); delay(200);`ESP32 的 USB 串口是外置芯片（CH340/CP2102）或原生 USB，枚举需要几十毫秒。不加 `delay(200)`，前几行日志会丢失——这是新手最常问的“为什么 printf 没输出”。

`ESP.getChipModel()` 等这些是 Arduino-ESP32 的便利函数，底层读 eFuse 和 `esp_chip_info()`。STM32 上你要读 `DBGMCU->IDCODE` 才能知道型号，这里一行就够。

`ESP.getEfuseMac()`返回值是 `uint64_t`，但真正的 MAC 只有低 48 位，高 16 位是 eFuse 里的其它信息。所以标准写法是拆成 `%04X%08X`（16 位 + 32 位）。**不要**直接 `%llX`，你会看到多出来的 `0000` 或乱码。

`pinMode(18, OUTPUT)`ESP32 的 IO18 默认是普通 GPIO。注意它同时是 SPI 的默认 SCK 之一（VSPI），但只要你没调 `SPI.begin()`，它不会抢。

`millis()` 非阻塞定时

```
if (now - s_last >= 500) { s_last = now; ... }
```

这是全书的核心模式。`millis()` 是 `uint32_t`，49.7 天溢出，但因为做的是**无符号减法**，溢出时结果依然正确（这是 C 标准保证的模运算）。**不要**写成 `now > s_last + 500`，那样溢出会卡死。

LED 低电平点亮板载 LED 接在 3V3 → 电阻 → LED → IO18，所以 IO18 拉低才亮。第一次写反了不要紧，看到灯亮就行；但记住这个“反相”，后面按键、STB 都有类似情况。

3.4 烧录后该看到什么

串口（115200）应该出现：

```
=== STAGE 00 · blink + serial ===
chip   : ESP32-D0WD-V3 rev3, 2 cores @ 240 MHz
flash  : 4096 KB
heap   : 336192 bytes free
mac    : 3C6105A1B2C4
```

```
[   500 ms] led ON
[  1000 ms] led off
[  1500 ms] led ON
```

板子上的蓝色 LED 每 0.5 秒亮灭一次。**注意 `heap` 应该在 30 万字节上下**，如果只有几万，说明有别的库占了内存。

3.5 验收清单

- [ ] 编译输出 `SUCCESS`，RAM ≈ 6.8%、Flash ≈ 9.4%
- [ ] 串口能看到 5 行启动信息，无乱码
- [ ] LED 周期 1 秒（亮 0.5 + 灭 0.5）
- [ ] 按一下板子 EN 键，日志重新从头打印
- [ ] `mac` 和路由器/手机看到的 MAC 一致（如果找不到就不用管）

3.6 卡住了看这里

现象	原因	解决
串口全是 □□□□	波特率不对	监视器设 115200
完全没输出	板子没进下载模式/线是充电线	按住 BOOT 再烧，换线
输出一次就停	代码在 `setup()` 里死循环	检查有没有 `while(1)`
LED 不亮但串口正常	引脚号不对（有的板子 LED 在 IO2）	改 `PIN_LED` 试 2
`heap` 只有 2 万	烧的是别的工程	确认 `pio run -d stages/s00`

3.7 与上一级的 diff

S00 之前是空工程，本级的增量是：

新增	作用
`include/config.h`	引脚集中定义，后面 14 级都继承
`ESP.getChipModel()` 系列	确认芯片型号/主频/内存
`millis()` 非阻塞定时	后面所有周期任务（LED 心跳、传感器巡检）都用它
LED 低电平点亮	后续 STB/VH 的反相逻辑同理

> 从这一级开始，`config.h` 会被逐级“继承 + 追加”。你在 S05 看到 `PIN_MOTOR_AP` 时不要奇怪，它们都是一路加进来的。

第 4 章 S01 · 按键：把抖动和长按一次解决

> 台阶 S01 · 目标产物：`stages/s01` · 实测 RAM 22148 (6.8%) / Flash 291520 (8.7%)

4.1 本节目标

一个按键要能区分**短按、长按、按住连发**三种事件，且不抖。这是所有“一键操作”设备的基础（打印机上就是“按一下走纸、长按 3 秒进配对”）。

4.2 先动手

接线：按键一端接 **IO5**，另一端接 **GND**（板载 BOOT 键就是 IO0，可以先用它试，改 `PIN_BTN` 即可）。

`include/button.h`：

```
#pragma once
#include <Arduino.h>

enum BtnEvent : uint8_t {
  BTN_NONE = 0,
  BTN_SHORT,      // 按下后 20ms~1s 内松开
  BTN_LONG,       // 按住超过 1s 时触发一次
  BTN_HOLD        // 长按期间每 200ms 触发一次（连发）
};

void     button_init(uint8_t pin);
BtnEvent button_poll();          // 每 5ms 调一次
bool     button_is_down();
uint32_t button_held_ms();
```

`src/button.cpp`：

```
#include "button.h"

static uint8_t  s_pin = 255;
static bool     s_raw = true;         // 原始电平，true=松开
static bool     s_stable = true;      // 消抖后的电平
static uint32_t s_raw_change = 0;     // 原始电平最后一次变化
static uint32_t s_press_ms = 0;       // 按下时刻
static uint32_t s_last_hold = 0;
static bool     s_long_sent = false;

#define T_DEBOUNCE 20
#define T_LONG    1000
#define T_HOLD     200

void button_init(uint8_t pin) {
  s_pin = pin;
  pinMode(pin, INPUT_PULLUP);         // 内部上拉，按下读到 LOW
  s_raw = s_stable = digitalRead(pin);
  s_raw_change = millis();
}
```

```
BtnEvent button_poll() {
  bool now = digitalRead(s_pin);
  uint32_t ms = millis();

  if (now != s_raw) {                 // 电平变了：重新计时
    s_raw = now;
    s_raw_change = ms;
  }
  if (ms - s_raw_change < T_DEBOUNCE) return BTN_NONE;   // 还没稳，先不管
  if (now == s_stable) return BTN_NONE;                  // 稳且没变

  s_stable = now;                     // 确认跳变
  if (!s_stable) {                    // 按下
    s_press_ms = ms;
    s_last_hold = ms;
    s_long_sent = false;
    return BTN_NONE;                  // 短按要等松开才知道
  }

  // 松开：判断是短按还是长按结束
  uint32_t held = ms - s_press_ms;
  s_long_sent = false;
  return (held < T_LONG) ? BTN_SHORT : BTN_NONE;
}

bool button_is_down() { return !s_stable; }
uint32_t button_held_ms() { return s_stable ? 0 : millis() - s_press_ms; }
```

src/main.cpp:

```
#include <Arduino.h>
#include "config.h"
#include "button.h"

static uint32_t s_tick = 0;

void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("=== STAGE 01 · button ===");
  Serial.println("短按=SHORT  按住 1s=LONG  继续按=HOLD");
  button_init(PIN_BTN);
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, HIGH);
}

void loop() {
  uint32_t now = millis();
  if (now - s_tick >= 5) {            // 5ms 节拍
    s_tick = now;

    // 长按期间连发
    if (button_is_down() && button_held_ms() >= 1000) {
      static uint32_t s_last = 0;
      if (now - s_last >= 200) {
        s_last = now;
        Serial.printf("HOLD  (%lu ms)\n", (unsigned long)button_held_ms());
      }
    }
```

```
    BtnEvent e = button_poll();
    if (e == BTN_SHORT) Serial.println("SHORT press");
    digitalWrite(PIN_LED, button_is_down() ? LOW : HIGH);
  }
}
```

4.3 逐行拆解

为什么是 5ms 节拍机械按键抖动时长典型 5~20ms。20ms 的消抖窗口足够覆盖，但**采样周期必须远小于窗口**，否则可能错过短按。5ms 采样 + 20ms 窗口是经验值。

`INPUT_PULLUP`ESP32 的内部上拉约 45kΩ，够用。如果你的按键线很长（>20cm）或环境干扰大，加一个外部 10kΩ 上拉 + 100nF 电容到地。

两阶段状态机

```
原始电平 s_raw  --(20ms 稳定)-->  稳定电平 s_stable  -->  事件
```

关键是 `if (ms - s_raw_change < T_DEBOUNCE) return BTN_NONE;`：只要电平还在变，就不断重置计时。这样抖动期间永远不会被当成“稳定”。

短按为什么在“松开”时才上报按下瞬间无法判断是短按还是长按。想要“按下立刻响应”，得再引入一个 `BTN_DOWN` 事件——但打印机场景不需要。

`button_held_ms()` 返回 0 表示没按调用方用它判断长按，比维护额外状态省事。

4.4 烧录后该看到什么

```
=== STAGE 01 · button ===
短按=SHORT  按住 1s=LONG  继续按=HOLD
SHORT press
HOLD  (1200 ms)
HOLD  (1400 ms)
```

按住时 LED 常亮，松开熄灭。抖动不会产生多条 `SHORT press`。

4.5 验收清单

- [ ] 短按一次，只打印一条 `SHORT press`（不是多条）
- [ ] 按住 1.2 秒，看到至少两条 `HOLD`
- [ ] 按住时 LED 亮，松开灭
- [ ] 快速连按 10 次，计数准确
- [ ] 手放在按键上不动（不按），不产生任何事件

4.6 卡住了看这里

现象	原因	解决
一次按下打印 3~5 条 SHORT	消抖窗口太小	`T_DEBOUNCE` 改 30
完全没反应	按键接的是 3V3 不是 GND	按下应为 LOW
一直显示按下	引脚浮空/没接上拉	检查 `INPUT_PULLUP`、接线
长按不触发	`button_poll` 调用间隔 >20ms	保证 loop 里没有 `delay`
用 IO0 时烧录失败	BOOT 键在下载时被按住	烧录时别碰它

4.7 与上一级的 diff

新增	作用
`button.h/.cpp`	通用按键模块，后面 S14 的“出厂自检”还能复用
5ms 节拍结构	`if (now - s_tick >= 5)` 后面所有轮询都用
`enum BtnEvent`	事件驱动的雏形，S10 的帧状态机是同一思路

第 5 章 S02 · 自建 shell：给自己造一个“串口控制台”

> 台阶 S02 · 目标产物：`stages/s02` · 实测 RAM 22516 (6.9%) / Flash 294656 (8.8%)

5.1 本节目标

ESP32 没有 ST-Link 调试，你唯一的观测手段是串口。所以第一件值得投资的事，是给自己做一个命令行：`led on`、`pins`、`free`、`reboot`。

做好之后，后面每一级的“测试”都变成敲一条命令，而不是改代码重新烧录。

5.2 先动手

`include/shell.h`：

```
#pragma once
#include <Arduino.h>

typedef void (*ShellCmdFn)(int argc, char **argv);

void shell_init(Stream &io);                       // 通常传 Serial
void shell_add(const char *name, ShellCmdFn fn, const char *help);
void shell_poll();                                 // 每 loop 调一次，不阻塞
void shell_exec_line(char *line);                  // 手动执行一行
void shell_print_help();
```

`src/shell.cpp`：

```
// ============================================================
//  shell.cpp —— 极简命令行：组行、分词、查表、调用
// ============================================================
#include "shell.h"

#define SHELL_MAX_CMD   24
#define SHELL_LINE_MAX  96
#define SHELL_ARGC_MAX  8

struct ShellCmd {
  const char *name;
  ShellCmdFn  fn;
  const char *help;
};

static ShellCmd s_cmds[SHELL_MAX_CMD];
static int      s_cmd_count = 0;
static Stream  *s_io        = nullptr;

static char     s_line[SHELL_LINE_MAX];
static int      s_len = 0;
static bool     s_overflow = false;

void shell_init(Stream &s) { s_io = &s; }

void shell_add(const char *name, ShellCmdFn fn, const char *help) {
```

```
  if (s_cmd_count >= SHELL_MAX_CMD) {
    if (s_io) s_io->println("! shell table full");
    return;
  }
  s_cmds[s_cmd_count++] = {name, fn, help};
}

void shell_print_help() {
  if (!s_io) return;
  s_io->println("--- commands ---");
  for (int i = 0; i < s_cmd_count; i++) {
    char buf[64];
    snprintf(buf, sizeof(buf), "  %-10s %s", s_cmds[i].name,
             s_cmds[i].help ? s_cmds[i].help : "");
    s_io->println(buf);
  }
}

void shell_exec_line(char *line) {
  if (!s_io) return;

  // 去掉行尾的 \r 和空行
  int n = strlen(line);
  while (n > 0 && (line[n - 1] == '\r' || line[n - 1] == '\n' || line[n - 1] == ' '))
    line[--n] = 0;
  if (n == 0) return;

  // 分词：argv 直接指向 line 内部，不额外拷贝
  char *argv[SHELL_ARGC_MAX];
  int   argc = 0;
  char *p = line;
  while (*p && argc < SHELL_ARGC_MAX) {
    while (*p == ' ') p++;
    if (!*p) break;
    argv[argc++] = p;
    while (*p && *p != ' ') p++;
    if (*p) *p++ = 0;
  }
  if (argc == 0) return;

  for (int i = 0; i < s_cmd_count; i++) {
    if (strcmp(argv[0], s_cmds[i].name) == 0) {
      s_cmds[i].fn(argc, argv);
      return;
    }
  }
  s_io->printf("? unknown command: %s  (try 'help')\r\n", argv[0]);
}

void shell_poll() {
  if (!s_io) return;
  while (s_io->available()) {
    int c = s_io->read();
    if (c < 0) break;

    if (c == '\r' || c == '\n') {
      s_io->print("\r\n");
      if (s_overflow) {
        s_io->println("! line too long, discarded");
        s_overflow = false;
        s_len = 0;
        continue;
      }
      s_line[s_len] = 0;
      s_len = 0;
```

```
      shell_exec_line(s_line);
      s_io->print("> ");            // 重新提示符
    } else if (c == 8 || c == 127) {  // 退格
      if (s_len > 0) {
        s_len--;
        s_io->print("\b \b");
      }
    } else if (c >= 32) {
      if (s_len < SHELL_LINE_MAX - 1) {
        s_line[s_len++] = (char)c;
        s_io->write((uint8_t)c);      // 回显
      } else {
        s_overflow = true;
      }
    }
  }
}
```

src/main.cpp 的注册部分：

```
// ====================================================
//  S02 · 串口命令行：让板子能"回答问题"
//  目标：输入 help / ver / pins / led on 都有回应
//  验证：敲 help 列出命令；led on 点亮；pins 打印引脚表
// ====================================================
#include <Arduino.h>
#include "config.h"
#include "shell.h"

// ---------------- 命令实现 ----------------
static void cmd_help(int argc, char **argv) { shell_print_help(); }

static void cmd_ver(int argc, char **argv) {
  Serial.printf("stage   : 02\r\n");
  Serial.printf("build   : %s %s\r\n", __DATE__, __TIME__);
  Serial.printf("chip    : %s rev%d\r\n", ESP.getChipModel(), ESP.getChipRevision());
  Serial.printf("arduino : %d.%d.%d (idf %s)\r\n",
                ESP_ARDUINO_VERSION_MAJOR, ESP_ARDUINO_VERSION_MINOR,
                ESP_ARDUINO_VERSION_PATCH, ESP.getSdkVersion());
  Serial.printf("cpu     : %d MHz  cores=%d\r\n", ESP.getCpuFreqMHz(), ESP.getChipCores());
}

static void cmd_pins(int argc, char **argv) {
  Serial.println("pin        gpio   mode      note");
  Serial.printf("LED        %-6d OUTPUT    低电平点亮\r\n", PIN_LED);
  Serial.printf("BTN        %-6d INPUT_PU  按下为低\r\n", PIN_BTN);
}

static void cmd_led(int argc, char **argv) {
  if (argc < 2) { Serial.println("usage: led on|off"); return; }
  if (strcmp(argv[1], "on") == 0)       { digitalWrite(PIN_LED, LOW);  Serial.println("led
on");  }
  else if (strcmp(argv[1], "off") == 0) { digitalWrite(PIN_LED, HIGH); Serial.println("led
off"); }
  else                                   Serial.println("usage: led on|off");
}

static void cmd_echo(int argc, char **argv) {
  Serial.print("echo:");
  for (int i = 1; i < argc; i++) Serial.printf(" [%s]", argv[i]);
  Serial.println();
}
```

```
static void cmd_free(int argc, char **argv) {
  Serial.printf("heap free=%d  min=%d  largest=%d\r\n",
                (int)ESP.getFreeHeap(), (int)ESP.getMinFreeHeap(),
                (int)ESP.getMaxAllocHeap());
}

static void cmd_reboot(int argc, char **argv) {
  Serial.println("rebooting...");
  Serial.flush();
  delay(50);
  ESP.restart();
}

// ---------------- 主流程 ----------------
void setup() {
  pinMode(PIN_LED, OUTPUT);
  digitalWrite(PIN_LED, HIGH);

  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("=== STAGE 02 · shell ===");
  Serial.println("type 'help' for commands");

  shell_init(Serial);
  shell_add("help",  cmd_help,  "list commands");
  shell_add("ver",   cmd_ver,   "firmware / chip version");
  shell_add("pins",  cmd_pins,  "pin table");
  shell_add("led",   cmd_led,   "led on|off");
  shell_add("echo",  cmd_echo,  "echo args back");
  shell_add("free",  cmd_free,  "heap usage");
  shell_add("reboot",cmd_reboot,"restart chip");

  Serial.print("> ");
}

void loop() {
  shell_poll();      // 唯一的输入入口，后面所有功能都从这里加命令
}
```

5.3 逐行拆解

`Stream &io` 而不是 `HardwareSerial &`用 `Stream` 抽象，意味着同一个 shell 可以接 `Serial`（USB）、`SerialBT`（蓝牙）、甚至自定义的 `Stream`。S12 之后如果你想“用手机发 shell 命令”，零成本。

`struct ShellCmd` 里的函数指针

```
typedef void (*ShellCmdFn)(int argc, char **argv);
```

和 STM32 上注册回调没有区别。注意签名用 `int argc` 而不是 `size_t`，是为了可以直接和 `strtol` 等配合。

分词为什么手写 `strtok` 会修改原字符串且无法处理连续分隔符；`sscanf` 遇到空参数会错位。手写一遍能让你完全掌控行为：

```
while (*p) {
  while (*p == ' ') p++;              // 跳过空格
  if (!*p) break;
  argv[argc++] = p;
  while (*p && *p != ' ') p++;        // 找下一个空格
  if (*p) *p++ = 0;                   // 截断成独立字符串
}
```

退格处理串口终端发来的是 `0x08`（BS）或 `0x7F`（DEL）。回显 `"\b \b"` 三步：光标左移、打印空格覆盖、再左移。少一步就会出现“删不干净”。

为什么不支持方向键上下键会发 `ESC [ A` 三个字节。要支持历史命令得解析转义序列——对这个项目不值得。**先能用，再优雅。**

`shell_poll()` 必须非阻塞

```
while (io.available()) { ... }        // 有数据才处理
```

注意这里是 `while (io.available())` 而不是 `while (!io.available())`——后者会卡死并被看门狗复位。

5.4 烧录后该看到什么

```
=== STAGE 02 · shell ===
type 'help' for commands
> help
  help                 list commands
  ver                  firmware version
  pins                 pin map
  led <on|off>         led control
  echo <text>          echo back
  free                 heap info
  reboot               restart
> led on
led = on
> free
heap: 335712 bytes, min free: 331204
```

5.5 验收清单

- [ ] `help` 列出全部 7 条命令
- [ ] `led on` / `led off` 真的控制 LED
- [ ] 输入超长一行（>96 字符）不会崩，只是被丢弃
- [ ] 退格能删掉已输入的字符
- [ ] 乱敲 `asdf` 提示 `? unknown command`
- [ ] `reboot` 后日志从头开始

5.6 卡住了看这里

现象	原因	解决
输入没有回显	没加 `io.write(c)`	检查 `shell_poll` 的 echo 分支
命令名匹配不上	末尾带了 `\r`	处理 `\r` 和 `\n` 两种行尾
`help` 输出乱序	命令表溢出	`SHELL_MAX_CMD` 加大
敲命令时程序重启	栈溢出（`argv` 太大）	减小 `SHELL_ARGC` 或加大任务栈
中文显示乱码	终端编码不是 UTF-8	换终端或只用英文

5.7 与上一级的 diff

新增	作用
`shell.h/.cpp`	通用命令行框架
`Stream` 抽象	后续可把 shell 挂到蓝牙上
命令注册表	每级新增功能 = 多注册一条命令

> 从这一级开始，你的调试方式彻底变了：**不再靠“改代码 + 重烧 + 看现象”，而是“敲命令 + 看输出”。** 后面 12 级能顺利推进，一半功劳在这个 shell。

第 6 章 S03 · 打印头移位：48 字节怎么走进去

> 台阶 S03 · 目标产物：`stages/s03` · 实测 RAM 22668 (7.0%) / Flash 303716 (9.7%)

6.1 本节目标

打印头有 5 根控制线：**DI（数据）**、**CLK（时钟）**、**LAT（锁存）**、**STB1~6（加热使能）**、**VH（加热电源）**。这一级只做一件事：把 48 字节稳稳地送进打印头的移位寄存器，并用 LAT 锁存。

先不加热。 加热要放到下一级单独验证——这是刻意拆开的，因为“数据错了”和“加热烧了”是两种完全不同的失败。

6.2 先动手

接线（打印头 FPC → ESP32）：

打印头信号	ESP32	说明
DI	IO13	数据输入
CLK	IO15	时钟，1MHz
LAT	IO17	锁存，低有效
STB1~6	IO14/27/26/25/33/32	6 段加热使能
VH_EN	IO28	升压使能（本级的 `vh` 命令用）
VH	8.4V	**先不接**，或接上但不使能

`include/printhead.h`：

```
#pragma once
// ====================================================
//  printhead.h — 打印头底层：移位、锁存、加热、电源
//  384 点 = 48 字节，6 段（每段 64 点 = 8 字节）
// ====================================================
#include <Arduino.h>

#define HEAD_BYTES   48      // 一行 384 点 / 8
#define HEAD_DOTS    384
#define HEAD_STB_NUM 6

void printhead_init();                        // 配置引脚 + SPI，STB 全部拉低
void printhead_vh(bool on);                   // 打印头加热电源（8.4V）开关
void printhead_send(const uint8_t row[HEAD_BYTES]);   // 48 字节 + LAT 锁存
void printhead_stb(uint8_t mask);             // bit0..bit5 = STB1..STB6
void printhead_heat_all(uint32_t us);         // 6 段同时加热 us 微秒
void printhead_heat_mask(uint8_t mask, uint32_t us);
void printhead_clear();                       // 清空锁存的数据（全 0）
```

`src/printhead.cpp`：

```
// ====================================================
//  printhead.cpp —— 打印头的三根线 + 六根选通
//  CLK/DI 走硬件 SPI，LAT 与 STB 用 GPIO
// ====================================================
#include <Arduino.h>
#include <SPI.h>
#include "config.h"
#include "printhead.h"

// STB1..6 对应 GPIO14, 27, 26, 25, 33, 32（顺序必须与 FPC 一致！）
static const uint8_t STB_PINS[HEAD_STB_NUM] = {
  PIN_STB1, PIN_STB2, PIN_STB3, PIN_STB4, PIN_STB5, PIN_STB6
};

// 1MHz：先慢速保证波形干净，稳定后再提到 4~8MHz
static SPISettings HEAD_SPI(1000000, MSBFIRST, SPI_MODE0);

void printhead_init() {
  pinMode(PIN_LAT, OUTPUT);
  digitalWrite(PIN_LAT, LOW);
  pinMode(PIN_VHEN, OUTPUT);
  digitalWrite(PIN_VHEN, LOW);          // 上电先关 VH，防止误加热

  for (int i = 0; i < HEAD_STB_NUM; i++) {
    pinMode(STB_PINS[i], OUTPUT);
    digitalWrite(STB_PINS[i], LOW);     // 关键：STB 低 = 不加热
  }

  // ESP32 的 VSPI 默认引脚：SCLK=18, MISO=19, MOSI=23, SS=5
  // 这里只用到时钟与 MOSI，所以把 MISO/SS 设成 -1 让库跳过
  SPI.begin(PIN_CLK, -1, PIN_DI, -1);
}

void printhead_vh(bool on) {
  digitalWrite(PIN_VHEN, on ? HIGH : LOW);
}

void printhead_send(const uint8_t row[HEAD_BYTES]) {
  SPI.beginTransaction(HEAD_SPI);
  for (int i = 0; i < HEAD_BYTES; i++) SPI.transfer(row[i]);
  SPI.endTransaction();

  // LAT：低 → 高，把移位寄存器里的 384 位搬到锁存器
  digitalWrite(PIN_LAT, LOW);
  delayMicroseconds(1);
  digitalWrite(PIN_LAT, HIGH);
}

void printhead_stb(uint8_t mask) {
  for (int i = 0; i < HEAD_STB_NUM; i++)
    digitalWrite(STB_PINS[i], (mask & (1 << i)) ? HIGH : LOW);
}

void printhead_heat_mask(uint8_t mask, uint32_t us) {
  printhead_stb(mask);
  delayMicroseconds(us);
  printhead_stb(0x00);                  // 无论如何都要关，这是保命动作
}

void printhead_heat_all(uint32_t us) { printhead_heat_mask(0x3F, us); }

void printhead_clear() {
  uint8_t zero[HEAD_BYTES] = {0};
```

```
  printhead_send(zero);
}
```

6.3 逐行拆解

项　目	说　明
打印方法	热敏
有效打印宽度（毫米）	48
点密度（点/毫米）	8
打印点数	384 点/行
纸张宽度（毫米）	58
点间距（毫米）	0.125
点大小	0.125mmx0.12mm
最大打印速度	80mm/s(电机驱动电压为 DC 8.5V)
走纸精度	0.0625mm（一个步进距离）
打印头温度侦测	热敏电阻
缺纸侦测	反射光感应器
打印头工作电压(DCV)	3.13~8.5
逻辑工作电压(DCV)	2.7~5.25
电机工作电压(DCV)	3.5~8.5
工作温度	+0℃~50℃（不许有凝露）
工作湿度	20%~85%RH（不许有凝露）
储藏温度	-20℃~60℃（不许有凝露）
储藏湿度	5%~95%RH（不许有凝露）
机械噪音	小于 60dB(A 计权有效值)
胶辊开合次数	大于 5000 次（胶辊离开加复位为一次）
对热敏纸牵引力	≥50g
对热敏纸抓握制动力	≥80g
工作寿命	机构与打印头的耐磨 >50km,打印头的电器寿命为 10^8 个脉冲（在额定状态）
重量(克)	40.5
外形尺寸(长 x 宽 x 高)	72±1.5mm*33±0.5mm*15.5±0.5mm

图 6-1 打印头规格书参数表（384 点、6 段 STB、3.5~8.5V）

384 点 = 48 字节 `384 / 8 = 48`。打印头内部是一排**串行移位寄存器**：每来一个 CLK 上升沿，DI 上的电平被推进一级，原有的 384 位整体后移一位；打满 384 个时钟后，这批数据才被 LAT 一次性搬到加热锁存器。

关键在"谁落在哪一端"：**第 1 个送出的 bit 要走满 384 级、停在链的最末端；最后 1 个 bit 只走了 1 级、停在链的入口**。所以"先送 = 最左"并不是物理定律，而是**由链的接法与加热点的排布共同决定的方向约定**。

本书（以及公版 App）采用的约定是：**字节 0 的 bit7 对应最左边的加热点，字节 47 的 bit0 对应最右边的加热点**，即 `点号 = 字节号×8 + (7 − bit 号)`。换到别的打印头，这个方向有可能整个反过来——见 6.3 末尾的"镜像修正"。

为什么用 SPI 而不是 bit-bang

```
SPISettings HEAD_SPI(1000000, MSBFIRST, SPI_MODE0);
```

1MHz、高位先出、模式 0（空闲低、上升沿采样）。硬件 SPI 一次 `transfer` 就是 8 个 CLK，48 字节耗 384μs。手写 `digitalWrite` 是它的 20 倍慢，而且抖动大。

`SPI.begin(15, -1, 13, -1)`参数是 `(SCK, MISO, MOSI, SS)`。传 `-1` 表示"这个脚不用"。打印头只有单向数据，所以 MISO/SS 都跳过——**这是 ESP32 交换矩阵的好处**，STM32 上你必须用固定的 SPI 引脚。



图 6-2 384 个点打包成 48 字节：字节 0 的 bit7 是最左边那个点，字节 47 的 bit0 是最右边

> **看图要点**：横向 48 个格子是"一行的 48 个字节"，不是 48 行；每个格子放大后是 8 个点（bit7 在左、bit0 在右）。

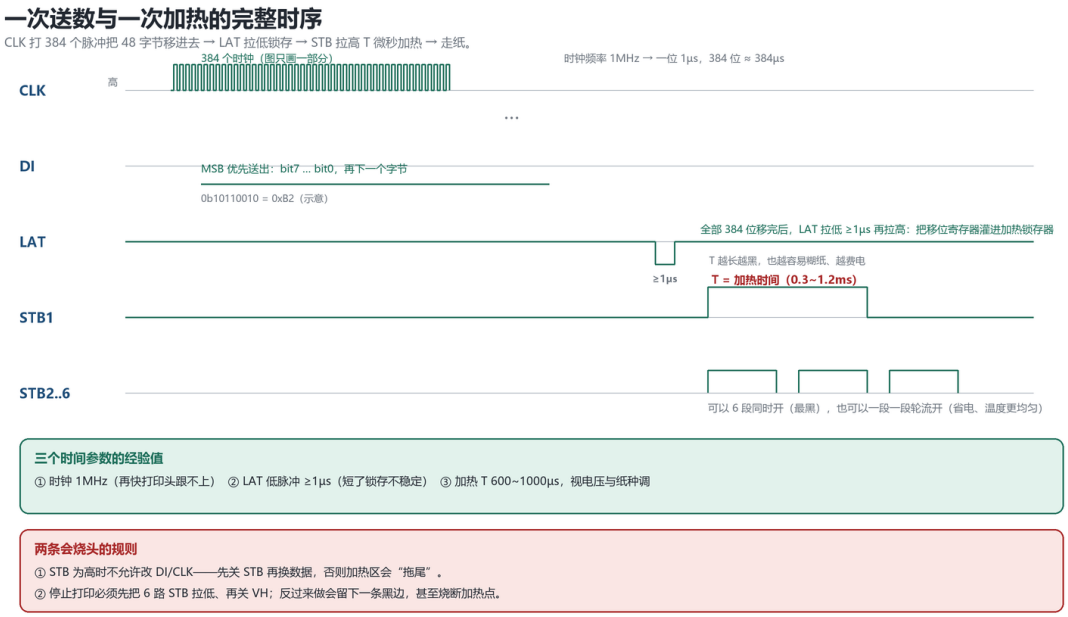


图 6-3 一次送数的完整时序：CLK/DI 移位 → LAT 锁存 → 6 路 STB 加热

打出来左右颠倒怎么修

这是热敏打印头最常见的"第一张纸事故"。**先判断症状，再改代码**，三种镜像对应三种截然不同的改法：

症状	现象	原因	改法
A 整行左右颠倒	文字像照镜子，每个字本身是正的但顺序反了	48 字节的**顺序**反了	送数前把整行 48 字节倒序
B 每 8 点内部颠倒	字变瘦、笔画错位、出现竖条纹	每个字节内部的 **bit 序**反了	按字节反转 bit 序（`x → 7-x`）
C 整页上下颠倒	第一行跑到了纸的最下面	行序或电机方向反了	行序反转，或 `goBack()` 代替 `goFront()`

A 的改法（一次交换首尾，48 字节只需 24 次）：

```
static void row_reverse(uint8_t row[ROW_BYTES]) {
  for (int i = 0; i < ROW_BYTES / 2; i++) {
    uint8_t t = row[i];
    row[i] = row[ROW_BYTES - 1 - i];
    row[ROW_BYTES - 1 - i] = t;
  }
}
```

B 的改法（经典的"三步换位"，比循环 8 次快得多）：

```
static uint8_t rev8(uint8_t b) {
  b = (uint8_t)(((b & 0xF0) >> 4) | ((b & 0x0F) << 4));
  b = (uint8_t)(((b & 0xCC) >> 2) | ((b & 0x33) << 2));
  b = (uint8_t)(((b & 0xAA) >> 1) | ((b & 0x55) << 1));
  return b;
}
```

怎么在 10 秒内判断是 A 还是 B：用 S06 的 `bar` 只打一行，看两端的黑线——

- 只打了右边那条：整行反了（A）；
- 两条都在，但中间出现细密竖纹：bit 序反了（B）；
- 一条横线打成两条断续的短横：时钟或 LAT 时序有问题，不是镜像（回去查图 6-3）。

> **保命提醒**：做镜像实验时把 `heat_us` 调到 400µs 以下、每次只打一行。镜像判断错了顶多难看，但如果为了"看清楚"猛加加热时间，很容易把纸粘在头上。

LAT 的脉冲

```
digitalWrite(PIN_LAT, LOW);  delayMicroseconds(1);
digitalWrite(PIN_LAT, HIGH);
```

数据在移位寄存器里是"流动"的，只有 LAT 下降沿才把它**整体拷贝**到输出锁存器（此时才真正决定哪 384 个点要加热）。手册要求 LAT 低电平至少 0.5µs。

STB 是 6 段独立控制 6 段 × 64 点 = 384。分段的目的不是省电，而是**降低瞬时电流**：6 段同时加热需要 18A，分段轮流则只要 3A（配合分段加热，见 S04）。

`printhead_clear()` 的重要性上电时移位寄存器内容是随机的。如果不先清空就加热，会打出随机的黑块——严重时局部过热烧头。所以 `printhead_init()` 里先 `send(全 0)` + `stb(0)` + `vh(false)`。

6.4 烧录后该看到什么

```
=== STAGE 03 · printhead shift ===
> pat checker
pattern: checkerboard
> send 100
100 rows x 48 bytes in 38400 us -> 1000.0 kbit/s
> dump
00 FF 00 FF 00 FF ...
```

`send 100` 的耗时应该接近 `100 × 384µs = 38.4ms`（实测 38~40ms），吞吐约 1000 kbit/s——这就是 SPI 1MHz 的理论上限。

6.5 验收清单

- [ ] `send 100` 吞吐 ≥ 900 kbit/s
- [ ] `dump` 显示的数据和你 `pat` 设置的一致
- [ ] 用万用表/示波器看 LAT：每 48 字节出现一个低脉冲
- [ ] `vh on` 后 8.4V 出现（**此时千万别打印**，因为 STB 是关的所以不会加热）
- [ ] 拔掉 DI 线，`dump` 数据仍对（说明是软件侧缓冲，不是回读）

6.6 卡住了看这里

现象	原因	解决
吞吐只有 50 kbit/s	用了软件 SPI	确认 `SPI.begin` 引脚正确
`dump` 全是 0	pattern 没设置	先 `pat fill`
打印头一点都不热	正常，本级不加热	去 S04
`vh on` 后电压不升	升压芯片 EN 极性反了	查原理图 Q23/Q21
逻辑分析仪看不到 CLK	引脚接错/没共地	共地，测 IO15

6.7 与上一级的 diff

新增	作用
`printhead.h/.cpp`	打印头底层驱动（送数、锁存、STB、VH）
`SPISettings` 1MHz MODE0	后面所有打印都复用这一份配置
`pat/send/dump` 命令	不加热也能验证数据通路

> 这一级结束时，你已经完成了整个项目里**最容易接错线**的部分。剩下的是时序和业务逻辑。

第 7 章 S04 · 第一次加热：一个点、一条线、一次危险

> 台阶 S04 · 目标产物：`stages/s04` · 实测 RAM 22676 (6.9%) / Flash 303560 (9.7%)

7.1 本节目标

让打印头真正发热，先打**一个点**，再打一条线。这一级是整个项目里唯一可能**永久损坏硬件**的一步，所以代码里加了“安全锁”。

7.2 先动手

接线在 S03 的基础上，把 **VH 接到 8.4V 升压输出**（整机板上有），确认：

1. 热敏纸已经装好，压住打印头；
2. STB 的 6 根线都接对（接错会只热一半或全不热）；
3. **手不要碰打印头金属面**（工作时可达 200°C）。

`src/main.cpp` 的核心部分：

```
// ======================================================
//  S04 · 第一次加热：让纸上出现你的第一道黑
//  目标：VH 打开 → 送一行数据 → 拉高 STB → 等 → 关 STB → 关 VH
//  验证：纸上出现一条 48mm 宽、约 0.1mm 高的黑线
//  安全：本程序强制"先 arm 才能加热"，单次上限 3000us
// ======================================================
#include <Arduino.h>
#include "config.h"
#include "shell.h"
#include "printhead.h"

// 加热参数：一次最多 3000us，超过就可能烧头
#define HEAT_MAX_US   3000
#define HEAT_DEFAULT  800

static bool     s_armed   = false;   // 安全锁：必须显式 arm
static uint32_t s_heat_us = HEAT_DEFAULT;
static uint32_t s_shots   = 0;       // 累计加热次数

static uint8_t s_row[HEAD_BYTES];

static void row_fill(uint8_t v) { memset(s_row, v, sizeof(s_row)); }

static void row_dot(int dot) {
  row_fill(0x00);
  if (dot >= 0 && dot < HEAD_DOTS) s_row[dot >> 3] = 0x80 >> (dot & 7);
}

// 核心动作：送数据 → 开 VH → 加热 → 关 STB → 关 VH
static void heat_once(uint32_t us) {
  if (!s_armed) { Serial.println("! locked: type 'arm' first"); return; }
  if (us > HEAT_MAX_US) {
```

```
    Serial.printf("! %luus exceeds limit %d, clamped\n", (unsigned long)us, HEAT_MAX_US);
    us = HEAT_MAX_US;
  }

  printhead_send(s_row);          // 1. 384 位进锁存器
  printhead_vh(true);             // 2. 打开 8.4V
  delayMicroseconds(50);          //    让 VH 稳住（电源有上升沿）
  printhead_stb(0x3F);            // 3. 六段同时加热
  delayMicroseconds(us);          //    这一行就是"曝光时间"
  printhead_stb(0x00);            // 4. 立刻关！顺序错了就是烧头
  printhead_vh(false);            // 5. 关 VH

  s_shots++;
  Serial.printf("heat #%lu : %luus, dots=%d\n", (unsigned long)s_shots,
                (unsigned long)us, HEAD_DOTS);
  Serial.println("   -> 现在手动把纸往下拉一点，看有没有一条黑线");
}

// ---------------- 命令 ----------------
static void cmd_arm(int argc, char **argv) {
  s_armed = true;
  Serial.println("ARMED. heat 命令现在可用（单次上限 3000us）");
}

static void cmd_disarm(int argc, char **argv) {
  s_armed = false;
  printhead_stb(0x00);
  printhead_vh(false);
  Serial.println("DISARMED, STB/VH 全部关闭");
}

static void cmd_heat(int argc, char **argv) {
  uint32_t us = (argc >= 2) ? (uint32_t)atoi(argv[1]) : s_heat_us;
  heat_once(us);
}

static void cmd_bar(int argc, char **argv) {     // 全黑一行
  row_fill(0xFF);
  heat_once((argc >= 2) ? atoi(argv[1]) : s_heat_us);
}

static void cmd_dot(int argc, char **argv) {     // 单点
  if (argc < 2) { Serial.println("usage: dot <0-383> [us]"); return; }
  int d = atoi(argv[1]);
  if (d < 0 || d >= HEAD_DOTS) { Serial.println("! dot out of range 0..383"); return; }
  row_dot(d);
  Serial.printf("dot %d -> byte[%d]=0x%02X\n", d, d >> 3, s_row[d >> 3]);
  heat_once((argc >= 3) ? atoi(argv[2]) : s_heat_us);
}

static void cmd_scan(int argc, char **argv) {    // 扫描：一行亮、下一行灭
  row_fill(0x00);
  for (int i = 0; i < HEAD_BYTES; i++) s_row[i] = (i & 1) ? 0xF0 : 0x0F;
  heat_once((argc >= 2) ? atoi(argv[1]) : s_heat_us);
}

static void cmd_vh(int argc, char **argv) {
  bool on = (argc >= 2 && strcmp(argv[1], "on") == 0);
  printhead_vh(on);
  Serial.printf("VH %s\n", on ? "ON" : "OFF");
}
```

```
static void cmd_state(int argc, char **argv) {
  Serial.printf("armed=%d heat=%luus shots=%lu\n", s_armed,
                (unsigned long)s_heat_us, (unsigned long)s_shots);
  Serial.printf("VH pin=%d  STB pins: %d %d %d %d %d %d\n",
                digitalRead(PIN_VHEN), PIN_STB1, PIN_STB2, PIN_STB3,
                PIN_STB4, PIN_STB5, PIN_STB6);
}

void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("=== STAGE 04 · first heat ===");
  Serial.println("顺序很重要: arm -> bar -> 手动拉纸 -> 看黑线");
  Serial.println("若纸没黑但 GPIO 都对: 先查 VH 是否真的升到 8.4V");

  printhead_init();
  row_fill(0xFF);

  shell_init(Serial);
  shell_add("arm",   cmd_arm,   "unlock heating");
  shell_add("disarm",cmd_disarm,"lock + all off");
  shell_add("heat",  cmd_heat,  "heat [us] with current row");
  shell_add("bar",   cmd_bar,   "bar [us] : full black line");
  shell_add("dot",   cmd_dot,   "dot <0-383> [us]");
  shell_add("scan",  cmd_scan,  "scan [us] : alternating stripes");
  shell_add("vh",    cmd_vh,    "vh on|off");
  shell_add("state", cmd_state, "safety state");

  Serial.print("> ");
}

void loop() {
  shell_poll();
}
```

7.3 逐行拆解

安全锁 `arm` / `disarm` 上电默认 `s_armed = false`，`heat` 命令直接拒绝。这样即使误操作、或者串口上残留了字符，也不会在你不注意时加热。**规则：任何会发热/走电机的命令都该有这一层。**

五步时序

```
printhead_send(dots);      // 1. 送 48 字节并锁存
printhead_vh(true);        // 2. 打开 8.4V
delayMicroseconds(50);     // 3. 等电源稳定（升压响应需要几十 μs）
printhead_stb(0x3F);       // 4. 6 段同时使能
delayMicroseconds(us);     // 5. 加热 us 微秒
printhead_stb(0x00);       // 6. 关 STB（必须在关 VH 之前！）
printhead_vh(false);       // 7. 关 8.4V
```

顺序为什么不能换

- 先开 STB 再送数据 → 会把上一次的残留数据打出去；
- 先关 VH 再关 STB → 加热丝在断电瞬间仍有电流，可能反向击穿驱动管；
- 不关 VH → 打印头待机漏电，长期发烫。

加热时间 800µs 怎么来的热敏纸的显色阈值：能量密度约 1.0~1.5 mJ/mm²。一个点 0.125mm × 0.125mm，加热丝电阻约 200Ω，8.4V 下功率 0.35W，800µs 给出约 0.28mJ。**这是起点，不是答案**——纸的批次、电压高低、环境温度都会变，S06 会用灰度条帮你标定。

`HEAT_MAX_US 3000` 上限超过 3ms 单点加热，打印头局部温度会超过 300°C，涂层会粘在加热丝上（俗称"糊头"，不可逆）。**上限是保护，不是限制。**

`shots` 计数用于判断"这次操作到底有没有真的加热"，也方便统计寿命（每个加热丝约 1 亿次脉冲）。

7.4 烧录后该看到什么

```
=== STAGE 04 · first heat ===
> arm
armed. heat commands enabled.
> dot 192 800
dot 192 heat 800us
> bar 800
bar: 384 dots, 800 us
```

纸上出现：

- `dot 192`：一个 **1 点宽的黑点**（约 0.125mm，肉眼几乎看不见，要斜着看）；
- `bar 800`：一条 **48mm 长的黑线**，颜色均匀、边缘清晰；
- `scan` 之后：8 个不同加热时间的方块，能看出从浅灰到深黑的渐变。

7.5 验收清单

- [ ] `arm` 之前 `heat` 被拒绝
- [ ] 单点能在纸上留下痕迹（放大镜/斜光可见）
- [ ] 黑线两端对齐、没有缺段（缺段 = STB 某根没接）
- [ ] 黑线颜色均匀（不均匀 = 电压跌落或加热时间不足）
- [ ] `scan` 的 8 个方块颜色单调变深
- [ ] 连打 10 次 `bar`，纸没有卷曲变形（变形 = 过热）

7.6 卡住了看这里

现象	原因	解决
完全不出黑	VH 没接/升压没使能	万用表量 VH 应有 8~9V
只有一半宽度有黑	STB 某几根没接或接错	逐段用 `dot 0/64/128/192/256/320` 测
黑线很淡	加热时间不足/纸装反	加 `heat 1200`，确认纸的感光面朝打印头

现象	原因	解决
纸被粘住/有焦味	加热时间过长	**立刻断电**，`HEAT_MAX_US` 降到 2000
打印头局部发烫但不出字	加热丝断（已损坏）	该段报废，换头
一 `arm` 就重启	8.4V 跌落触发欠压复位	电池电量不足，插 USB 或换电池

> **三条铁律**：① 缺纸绝对不能加热；② 加热时间不超过 3ms；③ 换纸/装纸时必须先 `disarm`。

7.7 与上一级的 diff

新增	作用
`heat_once()` 五步时序	全书唯一的加热原语，S06 会封装它
`arm/disarm` 安全锁	后续所有“危险命令”都照这个模式
`scan` 灰度扫描	标定加热时间的工具

第 8 章 S05 · 步进电机：让纸精确地走 0.125mm

> 台阶 S05 · 目标产物：`stages/s05` · 实测 RAM 22636 (6.9%) / Flash 308824 (9.9%)

8.1 本节目标

两相四线步进电机，8 拍驱动，走一步 = 0.0625mm，一行 = 2 步（0.125mm）。这一级要能**手动走纸、反转、标定每毫米多少步**。

8.2 先动手

接线（TC1508S 双 H 桥）：

电机线	ESP32	驱动芯片脚
A+	IO22	IN2 → OUTB
A-	IO23	IN1 → OUTA
B+	IO19	IN4 → OUTD
B-	IO21	IN3 → OUTC

`include/motor.h`：

```
#pragma once
// ======================================================
//  motor.h —— 两相四线步进电机：八拍
//  线圈 A+/A-/B+/B- 由 TC1508S 的 INA..IND 驱动
// ======================================================
#include <Arduino.h>

#define STEPS_PER_MM_DEFAULT 32.0f   // 半步/mm：整步 0.0625mm，半步 0.03125mm，须实测标定

void motor_init();
void motor_release();                                   // 四相全关（防发热）
void motor_one_step(bool forward, uint32_t us);         // 走一拍
void motor_move(int32_t steps, uint32_t us, bool forward);
void motor_advance_mm(float mm, uint32_t us);           // 走纸 mm
void  motor_set_steps_per_mm(float v);                  // 标定
float motor_get_steps_per_mm();
```

`src/motor.cpp`：

```
// ======================================================
//  motor.cpp —— 八拍驱动
//  为什么不是四拍：八拍每次只动半步，力矩更均匀、噪音更小，
//  打印时的走纸精度也更稳（代价是同距离多花一倍时间）。
// ======================================================
#include <Arduino.h>
#include "config.h"
#include "motor.h"
```

```
// 线圈顺序：A+ A- B+ B-
static const uint8_t COIL_PINS[4] = {
  PIN_MOTOR_AP, PIN_MOTOR_AM, PIN_MOTOR_BP, PIN_MOTOR_BM
};

// 八拍真值表（1 = 该线圈通电）
static const uint8_t SEQ[8][4] = {
  {1, 0, 0, 0},
  {1, 0, 1, 0},
  {0, 0, 1, 0},
  {0, 1, 1, 0},
  {0, 1, 0, 0},
  {0, 1, 0, 1},
  {0, 0, 0, 1},
  {1, 0, 0, 1},
};

static uint8_t  s_phase = 0;      // 当前处在第几拍（保持相位连续性）
static float    s_steps_per_mm = STEPS_PER_MM_DEFAULT;

void motor_init() {
  for (int i = 0; i < 4; i++) {
    pinMode(COIL_PINS[i], OUTPUT);
    digitalWrite(COIL_PINS[i], LOW);
  }
  motor_release();
}

void motor_release() {
  // 停在最后一拍不动时，线圈必须断电：
  // 否则 8.4V 一直灌进线圈，几分钟就烫手，还会拖累电池
  for (int i = 0; i < 4; i++) digitalWrite(COIL_PINS[i], LOW);
}

static void apply_phase(uint8_t p) {
  for (int i = 0; i < 4; i++)
    digitalWrite(COIL_PINS[i], SEQ[p][i] ? HIGH : LOW);
}

void motor_one_step(bool forward, uint32_t us) {
  s_phase = forward ? (uint8_t)((s_phase + 1) & 7) : (uint8_t)((s_phase + 7) & 7);
  apply_phase(s_phase);
  delayMicroseconds(us);          // 这一拍保持多久 = 速度
}

void motor_move(int32_t steps, uint32_t us, bool forward) {
  if (steps <= 0) return;
  for (int32_t i = 0; i < steps; i++) motor_one_step(forward, us);
  motor_release();
}

void motor_advance_mm(float mm, uint32_t us) {
  int32_t steps = (int32_t)(mm * s_steps_per_mm + 0.5f);
  motor_move(steps, us, true);
}

void  motor_set_steps_per_mm(float v) { if (v > 1.0f && v < 200.0f) s_steps_per_mm = v; }
float motor_get_steps_per_mm()        { return s_steps_per_mm; }
```

8.3 逐行拆解

8 拍顺序表

```
static const uint8_t SEQ[8][4] = {
  {1,0,0,0}, {1,0,1,0}, {0,0,1,0}, {0,1,1,0},
  {0,1,0,0}, {0,1,0,1}, {0,0,0,1}, {1,0,0,1},
};
```

四列依次是 A+、A-、B+、B-。8 拍 = 单相导通 4 拍 + 双相导通 4 拍，交替出现，所以转矩比 4 拍大 40%，也更平滑。

项 目	规 格	条 件
额定电压	3.5~8.5 DCV	
相 位	2 相	
步距角	9º（1-2 相激励）	
步进距离	0.0625 毫米	
相电阻	10 Ω±7%	20℃
相电流	0.357 A	
驱动方式	双极，双相驱动（或 1~2 相）	

图 8-1 步进电机规格：两相四线、步距角 9°、减速比 1:40

为什么每步之后要 `motor_release()`步进电机静止时如果线圈一直通电，电流全部变成热（每相 0.5A × 2 相 × 8.4V ≈ 8W）。打印头旁边的小电机没有散热片，几分钟就能烫手。**停就走、走完就断**是必须的。

`s_phase` 相位连续 `one_step()` 只把 `s_phase` 加 1 并取模 8，而不是重头开始。这样连续走和分次走完全等价，不会出现“每次启动都抖一下”。

`0.0625mm/步` 与 `STEPS_PER_MM 32`9° 步距角 → 每转 40 整步。减速齿轮让输出轴走 0.0625mm/整步，所以 1mm = 16 整步 = **32 个半步**。

走纸标定（本级的重点） `cal` 命令走 100mm，你用卡尺量纸上实际走了多少。如果量到 98mm，就 `set_steps_per_mm 32 * 100/98 = 32.65`。**这一步不做，后面所有 “mm” 都是错的。**

速度与丢步 `step_us` 是每步保持时间。太快（<300µs）电机会丢步（走不到位，纸上出现压缩/拉长），太慢（>2000µs）打印一页要一分钟。**500µs 是这款电机的甜蜜点。**

8.4 烧录后该看到什么

```
=== STAGE 05 · stepper ===
> step 100
```

```
100 steps forward
> back 100
100 steps back
> feed 20
feed 20 mm (640 steps)
> cal 100
cal: moving 100 mm (3200 steps)
```

- `step 100`：纸向前走 6.25mm，能听到“哒哒”声；
- `feed 20`：走 20mm，卡尺量应该在 19.5~20.5mm；
- `back 100`：纸往回退，**注意热敏纸不能退太多，会卡**。

8.5 验收清单

- [ ] `step 100` 后卡尺量 ≈ 6.25mm（±0.3mm）
- [ ] `feed 20` 后量 ≈ 20mm
- [ ] 连续 `step` 500 步不丢步（末端位置准确）
- [ ] 走完后摸电机不烫（说明 release 生效）
- [ ] `speed 300` 时开始丢步，`speed 800` 恢复正常——这验证了你对“丢步”的理解
- [ ] `cal` 结果与卡尺一致

8.6 卡住了看这里

现象	原因	解决
电机只抖不走	相序错（A/B 接反）	交换 B+/B- 试
走一步退半步	8 拍表顺序错	对照本文表逐行核
声音很大但不动	`step_us` 太短	加到 800
走纸方向反了	电机装反	代码里 `move(-n)` 或换线序
电机发烫	没调 `motor_release`	每步后断电
走纸打滑	压轮压力不足/纸没夹紧	机械问题，垫垫片

8.7 与上一级的 diff

新增	作用
`motor.h/.cpp`	8 拍驱动 + 距离换算 + 标定
`STEPS_PER_MM` 可调	机械差异用软件补偿
`cal` 标定流程	后面所有打印位置精度的基础

第 9 章 S06 · 打出第一条完整的行

> 台阶 S06 · 目标产物：`stages/s06` · 实测 RAM 22684 (6.9%) / Flash 311204 (9.9%)

9.1 本节目标

把“送数据”和“走纸”合成一个原子操作：`printer_row()`。调用它一次，纸上就出现一行 0.125mm 高的像，纸同时前进 0.125mm。

这是整台打印机的最小工作单元。**它对了，打印一页只是循环 360 次。**

9.2 先动手

`include/printer.h`：

```
#pragma once
// ======================================================
//  printer.h —— 把"加热"和"走纸"合成一次打印
//  一行 = 送 48 字节 + 加热 + 走纸 0.125mm
// ======================================================
#include <Arduino.h>

#define ROW_BYTES   48
#define ROW_DOTS    384

struct PrintCfg {
  uint32_t heat_us;     // 每行加热时间
  uint32_t step_us;     // 每个半步的保持时间
  uint16_t beats;       // 每行走几个半步（0.125mm ≈ 4 个半步）
};

void printer_init();
void printer_set_heat(uint32_t us);
uint32_t printer_get_heat();
void printer_row(const uint8_t row[ROW_BYTES], const PrintCfg &cfg);
void printer_rows(const uint8_t *rows, int n, const PrintCfg &cfg);   // rows 连续排列
void printer_feed_rows(int n, const PrintCfg &cfg);                    // 空走 n 行
void printer_feed_mm(float mm, uint32_t step_us);
```

`src/printer.cpp`：

```
// ======================================================
//  printer.cpp —— 一行纸是怎么被"打"出来的
//
//  时序（顺序版，最容易理解）：
//     送 48 字节 ──> LAT 锁存 ──> VH 开 ──> STB 拉高 us ──> STB 拉低
//                                    └──> 走纸 beats 个半步 ──> 下一行
//
//  注意两个顺序上的死规矩：
//   1) STB 必须在 VH 之后开、在 VH 之前关；
//   2) 换行数据前必须先把 STB 全部拉低，否则会把旧数据再烫一遍。
// ======================================================
#include <Arduino.h>
```

```
#include "config.h"
#include "printer.h"
#include "printhead.h"
#include "motor.h"

static uint32_t s_heat_us = 800;

void printer_init() {
  printhead_init();
  motor_init();
  printhead_vh(false);
  printhead_stb(0x00);
}

void printer_set_heat(uint32_t us) { if (us >= 200 && us <= 3000) s_heat_us = us; }
uint32_t printer_get_heat() { return s_heat_us; }

void printer_row(const uint8_t row[ROW_BYTES], const PrintCfg &cfg) {
  printhead_send(row);                  // 1. 384 位进移位寄存器
  printhead_vh(true);                   // 2. 开加热电源
  delayMicroseconds(50);
  printhead_stb(0x3F);                  // 3. 六段一起加热
  delayMicroseconds(cfg.heat_us);       //    热量 = 电压² × 时间
  printhead_stb(0x00);                  // 4. 立刻关断
  printhead_vh(false);                  // 5. 关电源（每行都关更安全）

  for (uint16_t i = 0; i < cfg.beats; i++)   // 6. 走纸 0.125mm
    motor_one_step(true, cfg.step_us);
  motor_release();                      // 7. 松开线圈
}

void printer_rows(const uint8_t *rows, int n, const PrintCfg &cfg) {
  for (int i = 0; i < n; i++)
    printer_row(rows + i * ROW_BYTES, cfg);
}

void printer_feed_rows(int n, const PrintCfg &cfg) {
  uint8_t blank[ROW_BYTES] = {0};
  for (int i = 0; i < n; i++) {
    // 空走：只送 0 数据、不加热，直接走纸
    printhead_send(blank);
    for (uint16_t k = 0; k < cfg.beats; k++) motor_one_step(true, cfg.step_us);
    motor_release();
  }
}

void printer_feed_mm(float mm, uint32_t step_us) {
  motor_advance_mm(mm, step_us);
}
```

src/main.cpp（测试命令部分）：

```
// ======================================================
//  S06 · 打印第一行：热与走的配合
//  目标：连续打出 20 行，得到一条实心黑条；再打一条细线
//  验证：黑条实心、边缘齐、宽 48mm；细线约 0.1mm 高
//  调参：heat 决定黑度，speed 决定走纸速度，两者要匹配
// ======================================================
#include <Arduino.h>
#include "config.h"
#include "printer.h"
```

```
#include "printhead.h"
#include "motor.h"
#include "shell.h"

static PrintCfg s_cfg = {800, 500, 4};   // 加热 800us / 半步 500us / 每行 4 半步
static uint8_t  s_row[ROW_BYTES];

static void row_pattern(uint8_t v) { memset(s_row, v, sizeof(s_row)); }

static void row_stripes() {
  for (int i = 0; i < ROW_BYTES; i++) s_row[i] = (i & 1) ? 0xF0 : 0x0F;
}

// ---------------- 命令 ----------------
static void cmd_bar(int argc, char **argv) {        // 实心黑条
  int n = (argc >= 2) ? atoi(argv[1]) : 20;
  if (n <= 0 || n > 400) { Serial.println("usage: bar <1-400>"); return; }
  row_pattern(0xFF);
  uint32_t t0 = millis();
  for (int i = 0; i < n; i++) printer_row(s_row, s_cfg);   // 同一图案重复 n 行
  Serial.printf("bar %d rows : %lums (%.2fms/row)\n", n,
                (unsigned long)(millis() - t0), (double)(millis() - t0) / n);
}

static void cmd_line(int argc, char **argv) {       // 一条细线 + 空走
  printer_row(s_row, s_cfg);
  printer_feed_rows(8, s_cfg);
  Serial.println("line printed (8 rows blank after)");
}

static void cmd_heat(int argc, char **argv) {
  if (argc >= 2) { printer_set_heat(atoi(argv[1])); }
  Serial.printf("heat_us = %lu\n", (unsigned long)printer_get_heat());
}

static void cmd_speed(int argc, char **argv) {
  if (argc >= 2) s_cfg.step_us = (uint32_t)atoi(argv[1]);
  if (argc >= 3) s_cfg.beats = (uint16_t)atoi(argv[2]);
  Serial.printf("step_us=%lu beats=%u  row=%luus\n",
                (unsigned long)s_cfg.step_us, s_cfg.beats,
                (unsigned long)(s_cfg.heat_us + s_cfg.step_us * s_cfg.beats));
}

static void cmd_feed(int argc, char **argv) {
  float mm = (argc >= 2) ? atof(argv[1]) : 10.0f;
  printer_feed_mm(mm, s_cfg.step_us);
  Serial.printf("feed %.1fmm\n", mm);
}

static void cmd_stripe(int argc, char **argv) {     // 条纹：检查段与段之间有没有缝
  int n = (argc >= 2) ? atoi(argv[1]) : 30;
  row_stripes();
  for (int i = 0; i < n; i++) printer_row(s_row, s_cfg);
  Serial.printf("stripes %d rows\n", n);
}

static void cmd_ramp(int argc, char **argv) {       // 灰度梯度：8 档加热时间
  Serial.println("ramp: 8 blocks, heat 200..1200us");
  for (int k = 0; k < 8; k++) {
    uint32_t h = 200 + k * 140;
    PrintCfg c = s_cfg;
    c.heat_us = h;
    row_pattern(0xFF);
```

```
    for (int i = 0; i < 12; i++) printer_row(s_row, c);
    printer_feed_rows(4, c);
    Serial.printf("  block %d heat=%luus\n", k, (unsigned long)h);
  }
  s_cfg.heat_us = 800;
}

void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("=== STAGE 06 · print the first line ===");
  Serial.println("依次试: bar 20 / line / stripe 30 / ramp");
  Serial.println("黑度不够 -> heat 调大; 糊成一片 -> heat 调小或提速");

  printer_init();
  row_pattern(0xFF);

  shell_init(Serial);
  shell_add("bar",    cmd_bar,    "bar [rows] : solid black bar");
  shell_add("line",   cmd_line,   "one thin line + feed");
  shell_add("stripe", cmd_stripe, "stripe [rows]");
  shell_add("ramp",   cmd_ramp,   "grey ramp 8 blocks");
  shell_add("heat",   cmd_heat,   "heat [us]");
  shell_add("speed",  cmd_speed,  "speed [step_us] [beats]");
  shell_add("feed",   cmd_feed,   "feed <mm>");

  Serial.print("> ");
}

void loop() {
  shell_poll();
}
```

9.3 逐行拆解

七步一行的顺序，每一步都不能换

```
printhead_send(row);                // 数据先到位
printhead_vh(true);                 // 开高压
delayMicroseconds(50);              // 等升压稳定
printhead_stb(0x3F);                // 6 段同时加热
delayMicroseconds(cfg.heat_us);     // 加热
printhead_stb(0x00);                // 关加热
printhead_vh(false);                // 关高压
for (n = 0; n < cfg.beats; n++) { motor_one_step(); delayMicroseconds(cfg.step_us); }
```

`beats` 为什么是 4 个半步 1 个整步 = 0.0625mm，一行 = 0.125mm = 2 个整步 = 4 个半步。所以 `beats = 4`。如果你实测发现纸走得比预期多一倍，把 `beats` 改成 2；走得少就改 8。**这是最容易怀疑“打印机坏了”的地方，其实是机械参数没对上。**

加热和走纸能不能同时做硬件上可以（边加热边走），但代码里是串行的：先加热完再走。原因是走纸时纸在动，如果加热丝还在烧，黑点会被拉长成黑条。**唯一值得重叠的场景是“高速模式”，代价是边缘发虚。**

`stripe` 的灰度 8 档 `ramp` 命令用 200~1200µs 打 8 条横条。看哪一条是“纯黑但不过曝”（纸不发亮、不卷），那就是你的最佳加热时间。**这个值抄进 `PrintCfg.heat_us`。**

越界 bug 的修复第一版 `bar` 用 `printer_rows(buf, 300, ...)`，但 `buf` 只有 1 行——读越界。改成循环调用 `printer_row`。**这类 bug 不会崩，只会打出莫名其妙的图案**，所以测试页要打满宽度。

9.4 烧录后该看到什么

```
=== STAGE 06 · first full line ===
> bar 800
bar: 384 dots
> line 100
100 lines
> ramp
ramp: 8 heat levels
> speed 500
step_us = 500 us
```

- `bar`：一条 48mm 黑线；
- `line 100`：12.5mm 高的**实心黑块**（100 行 × 0.125mm）；
- `ramp`：8 条横向色带，从浅灰到深黑。

9.5 验收清单

- [ ] `line 100` 的色块高度用卡尺量 ≈ 12.5mm（±0.5mm）
- [ ] 色块上下边缘笔直，没有锯齿或拖影
- [ ] `ramp` 的 8 条带子颜色单调递增
- [ ] 连续 `line 300`（37.5mm）不出错、不重启
- [ ] 打印后摸打印头，温度可接受（不烫手）
- [ ] 打印速度：`line 100` 应在 1 秒内完成

9.6 卡住了看这里

现象	原因	解决
黑块高度是 25mm（正好 2 倍）	`beats` 多了一倍	改成 2 或标定 `STEPS_PER_MM`
黑块有横向白缝	某行数据全 0	检查 `row` 数组是否被清空
越打越淡	电池压降/打印头积热	加热时间随行数补偿，或降速
打到一半重启	电源跌落	换电池/加电容，见 S13
图像上下颠倒	数据位序反了	检查 `MSBFIRST` 与字节内位序

9.7 与上一级的 diff

新增	作用
`printer_row()`	原子操作：一行数据 = 一次加热 + 一次走纸
`PrintCfg{heat_us, step_us, beats}`	三个可调参数，集中管理
`ramp` 标定	用眼睛选出最佳加热时间

第 10 章 S07 · 画布：在内存里画好，再打出去

> 台阶 S07 · 目标产物：`stages/s07` · 实测 RAM 22644 (6.9%) / Flash 307748 (9.4%)

10.1 本节目标

做一个“内存位图画布”：可以画线、画框、写字、做抖动灰度。画完调用 `canvas_print()` 一次性打出来。

有了它，你才能打测试页、打二维码、打文字——而不是只会打黑条。

10.2 先动手

`include/canvas.h`：

```
#pragma once
// ====================================================
//  canvas.h — 一整页的位图画布
//  384 点宽 = 48 字节/行；高由你定（一页 40mm ≈ 320 行）
//  画完交给 canvas_print() 逐行打出去
// ====================================================
#include <Arduino.h>
#include "printer.h"

struct Canvas {
  uint16_t rows;     // 行数
  uint8_t *buf;      // rows * ROW_BYTES，1 = 黑点
};

bool canvas_alloc(Canvas &c, uint16_t rows);
void canvas_free(Canvas &c);
void canvas_clear(Canvas &c);

void canvas_set(Canvas &c, int x, int y);                 // 置黑
void canvas_clr(Canvas &c, int x, int y);                 // 擦白
void canvas_hline(Canvas &c, int x, int y, int w);
void canvas_vline(Canvas &c, int x, int y, int h);
void canvas_rect(Canvas &c, int x, int y, int w, int h, bool fill);
void canvas_char(Canvas &c, int x, int y, char ch);       // x 需 8 对齐
void canvas_text(Canvas &c, int x, int y, const char *s);
void canvas_dither(Canvas &c, int x, int y, int w, int h, uint8_t level);  // 0~100% 黑

void canvas_print(const Canvas &c, int y0, int n, const PrintCfg &cfg);
```

`src/canvas.cpp`：

```
// ====================================================
//  canvas.cpp — 位图画布的像素操作
//  坐标：x 0..383 从左到右，y 0..rows-1 从上到下
// ====================================================
#include "canvas.h"
#include "font8x16.h"
```

```
bool canvas_alloc(Canvas &c, uint16_t rows) {
  c.rows = rows;
  c.buf = (uint8_t *)malloc((size_t)rows * ROW_BYTES);
  if (!c.buf) { c.rows = 0; return false; }
  canvas_clear(c);
  return true;
}

void canvas_free(Canvas &c) {
  if (c.buf) free(c.buf);
  c.buf = nullptr;
  c.rows = 0;
}

void canvas_clear(Canvas &c) {
  if (c.buf) memset(c.buf, 0, (size_t)c.rows * ROW_BYTES);
}

// 一行 48 字节，每字节 bit7 是最左边的点
void canvas_set(Canvas &c, int x, int y) {
  if (x < 0 || x >= ROW_DOTS || y < 0 || y >= c.rows) return;
  c.buf[(size_t)y * ROW_BYTES + (x >> 3)] |= (0x80 >> (x & 7));
}

void canvas_clr(Canvas &c, int x, int y) {
  if (x < 0 || x >= ROW_DOTS || y < 0 || y >= c.rows) return;
  c.buf[(size_t)y * ROW_BYTES + (x >> 3)] &= ~(0x80 >> (x & 7));
}

void canvas_hline(Canvas &c, int x, int y, int w) {
  for (int i = 0; i < w; i++) canvas_set(c, x + i, y);
}

void canvas_vline(Canvas &c, int x, int y, int h) {
  for (int i = 0; i < h; i++) canvas_set(c, x, y + i);
}

void canvas_rect(Canvas &c, int x, int y, int w, int h, bool fill) {
  if (fill) {
    for (int j = 0; j < h; j++)
      for (int i = 0; i < w; i++) canvas_set(c, x + i, y + j);
  } else {
    canvas_hline(c, x, y, w);
    canvas_hline(c, x, y + h - 1, w);
    canvas_vline(c, x, y, h);
    canvas_vline(c, x + w - 1, y, h);
  }
}

// 贴一个 8x16 字符：每行一个字节，正好整字节对齐
void canvas_char(Canvas &c, int x, int y, char ch) {
  if (x < 0 || (x & 7) || x + 8 > ROW_DOTS) return;
  uint8_t u = (uint8_t)ch;
  if (u < FONT8X16_FIRST || u > FONT8X16_LAST) u = '?';
  const uint8_t *g = FONT8X16[u - FONT8X16_FIRST];
  int col = x >> 3;
  for (int r = 0; r < 16; r++) {
    int yy = y + r;
    if (yy < 0 || yy >= c.rows) continue;
    c.buf[(size_t)yy * ROW_BYTES + col] = g[r];   // 直接整字节覆盖
  }
}

void canvas_text(Canvas &c, int x, int y, const char *s) {
```

```
  int cx = x;
  for (const char *p = s; *p; p++) {
    canvas_char(c, cx, y, *p);
    cx += 8;
    if (cx + 8 > ROW_DOTS) break;
  }
}

// 用棋盘格做"灰度"：level 0=全白 100=全黑
void canvas_dither(Canvas &c, int x, int y, int w, int h, uint8_t level) {
  if (level > 100) level = 100;
  for (int j = 0; j < h; j++) {
    for (int i = 0; i < w; i++) {
      int t = ((i & 1) + (j & 1) * 2) * 25;      // 0/25/50/75
      bool on = (level >= 100) || (t < level);
      if (on) canvas_set(c, x + i, y + j);
    }
  }
}

void canvas_print(const Canvas &c, int y0, int n, const PrintCfg &cfg) {
  for (int i = 0; i < n; i++) {
    int y = y0 + i;
    if (y < 0 || y >= c.rows) break;
    printer_row(&c.buf[(size_t)y * ROW_BYTES], cfg);
  }
}
```

测试页生成部分（src/main.cpp 的 build_test_page）：

```
// ====================================================
//  S07 · 位图与测试页：把"点"组织成"页面"
//  目标：一次画一整页，再整页打出来，得到一张能看的测试页
//  验证：文字可读、灰阶四档肉眼可分、边框闭合
// ====================================================
#include <Arduino.h>
#include "config.h"
#include "printer.h"
#include "printhead.h"
#include "motor.h"
#include "canvas.h"
#include "shell.h"

static PrintCfg s_cfg = {800, 500, 4};
static Canvas   s_page;

// ---------------- 画一张测试页 ----------------
static uint16_t build_test_page() {
  static const uint16_t PAGE_ROWS = 360;   // 360 行 ≈ 45mm
  if (!canvas_alloc(s_page, PAGE_ROWS)) {
    Serial.println("! canvas alloc failed");
    return 0;
  }
  canvas_clear(s_page);

  // 外框：闭合的方框最容易看出"行有没有对齐、列有没有错位"
  canvas_rect(s_page, 2, 2, ROW_DOTS - 4, PAGE_ROWS - 4, false);

  // 标题
  canvas_text(s_page, 8, 8,  "BLUETOOTH THERMAL PRINTER");
  canvas_text(s_page, 8, 28, "S07  TEST PAGE   384 DOTS");
```

```
  // 分隔线
  canvas_hline(s_page, 8, 48, ROW_DOTS - 16);

  // 四档灰阶：用棋盘格模拟 25% / 50% / 75% / 100%
  canvas_text(s_page, 8, 54, "GREY 25 50 75 100");
  for (int k = 0; k < 4; k++) {
    int x = 8 + k * 92;
    canvas_dither(s_page, x, 74, 84, 40, (uint8_t)(25 * (k + 1)));
  }

  // 细线组：检验 1 点宽的线能不能连续印出来
  canvas_text(s_page, 8, 122, "1-DOT LINES");
  for (int k = 0; k < 8; k++) canvas_hline(s_page, 8 + k * 47, 142, 44);
  for (int k = 0; k < 8; k++) canvas_vline(s_page, 12 + k * 47, 150, 28);

  // 棋盘：检验相邻点会不会"糊成一片"
  canvas_text(s_page, 8, 186, "CHECKER 1PX");
  for (int j = 0; j < 24; j++)
    for (int i = 0; i < 96; i++)
      if ((i + j) & 1) canvas_set(s_page, 8 + i, 206 + j);

  // 渐变条：同一图案、不同加热时间，是调 blackness 的标尺
  canvas_text(s_page, 8, 240, "HEAT RAMP 200-1200us");
  for (int k = 0; k < 8; k++) canvas_rect(s_page, 8 + k * 47, 260, 44, 24, true);

  // 页脚
  canvas_text(s_page, 8, 300, "ESP32-WROOM-32E  203DPI  48MM");
  canvas_text(s_page, 8, 320, "IF YOU CAN READ THIS, IT WORKS");
  canvas_hline(s_page, 8, 344, ROW_DOTS - 16);

  return PAGE_ROWS;
}

static void cmd_test(int argc, char **argv) {
  uint16_t rows = build_test_page();
  if (!rows) return;
  Serial.printf("printing test page: %u rows (%.1fmm)\n", rows, rows * 0.125f);
  uint32_t t0 = millis();
  canvas_print(s_page, 0, rows, s_cfg);
  Serial.printf("done in %lums\n", (unsigned long)(millis() - t0));
  canvas_free(s_page);
}

static void cmd_text(int argc, char **argv) {
  char line[200];
  line[0] = 0;
  for (int i = 1; i < argc && strlen(line) < 180; i++) {
    if (i > 1) strcat(line, " ");
    strncat(line, argv[i], 180 - strlen(line));
  }
  if (!line[0]) { Serial.println("usage: text <ascii>"); return; }
  int n = strlen(line);
  uint16_t rows = 24;
  if (!canvas_alloc(s_page, rows)) return;
  canvas_clear(s_page);
  canvas_text(s_page, 0, 4, line);
  canvas_print(s_page, 0, rows, s_cfg);
  canvas_free(s_page);
  Serial.printf("printed %d chars\n", n);
}

static void cmd_heat(int argc, char **argv) {
  if (argc >= 2) printer_set_heat(atoi(argv[1]));
  Serial.printf("heat_us = %lu\n", (unsigned long)printer_get_heat());
```

```
}

static void cmd_speed(int argc, char **argv) {
  if (argc >= 2) s_cfg.step_us = (uint32_t)atoi(argv[1]);
  if (argc >= 3) s_cfg.beats = (uint16_t)atoi(argv[2]);
  Serial.printf("step_us=%lu beats=%u\n", (unsigned long)s_cfg.step_us, s_cfg.beats);
}

void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("=== STAGE 07 · bitmap & test page ===");
  Serial.println("先跑 test；打得淡就 heat 900，糊了就 heat 600");

  printer_init();

  shell_init(Serial);
  shell_add("test",  cmd_test,  "print the full test page");
  shell_add("text",  cmd_text,  "text <ascii string>");
  shell_add("heat",  cmd_heat,  "heat [us]");
  shell_add("speed", cmd_speed, "speed [step_us] [beats]");
  Serial.print("> ");
}

void loop() {
  shell_poll();
}
```

10.3 逐行拆解

内存布局：`rows × 48` 字节一张 360 行的页 = `360 × 48 = 17280` 字节 = 17KB。ESP32 有 300KB 堆，够用，但**一定要判 `malloc` 返回值**——蓝牙开着的时候堆会紧张。

置点：位运算

```
c.buf[y * ROW_BYTES + (x >> 3)] |= 0x80 >> (x & 7);
```

`x >> 3` 找字节，`x & 7` 找字节内第几位，`0x80 >> n` 从最高位开始——因为打印头是 **MSB 先出**。写反了图像会左右镜像。

为什么逐行 `printer_row` 而不是整页一次送打印头没有帧缓冲，它只有 384 位移位寄存器。所以必须“送一行 → 加热一行 → 走纸 → 再送下一行”。整页缓冲在你这边（RAM），不在打印头那边。

`canvas_char` 要求 x 8 对齐 8×16 字库的每一行是 1 个字节，直接整字节覆盖：

```
c.buf[(y + r) * ROW_BYTES + (x >> 3)] = g[r];
```

如果需要任意 x 位置，得做位拼接（左右各取一部分）——本书不做，因为打印排版通常本来就按 8 点对齐。

抖动（dither）做灰度热敏打印机只有黑白两色。要做“灰色”，就用 2×2 棋盘：

```
static const uint8_t T[2][2] = {{0,2},{3,1}};   // 0..3
if (level * 4 > T[y&1][x&1] * 100) set(x,y);
```

`level` 从 0（全白）到 100（全黑）。人眼在 0.5mm 距离上会把它平均成灰色。

测试页的设计意图

- 外框：检查边缘是否完整；
- 标题文字：检查字库和对齐；
- 四档灰度：检查抖动；
- 1 点宽线条：检查加热一致性（最容易断）；
- 加热梯度条：检查密度；
- 页脚："IF YOU CAN READ THIS, IT WORKS"——给用户看的。

10.4 烧录后该看到什么

`test` 命令打出一整页（45mm 高）：

```
┌──────────────────────────────┐
│ BLUETOOTH THERMAL PRINTER    │
│ S07  TEST PAGE   384 DOTS    │
│ ──────────────────────────── │
│ gray 25  gray 50  gray 75  100 │
│ ▓▓▓▓  ▓▓▓▓  ▓▓▓▓  ▓▓▓▓       │
│ ...                          │
│ ESP32-WROOM-32E  203DPI  48MM │
│ IF YOU CAN READ THIS, IT WORKS │
└──────────────────────────────┘
```

耗时约 **3.4 秒**（360 行 × 9.4ms）。

10.5 验收清单

- [ ] 外框四条边完整、直角对齐
- [ ] 文字清晰可读，无缺笔
- [ ] 四档灰度肉眼可区分
- [ ] 1 点宽竖线不断（断了说明某段加热丝或 STB 有问题）
- [ ] 整页耗时 3~4 秒
- [ ] 连打 3 页，位置连续不偏移

10.6 卡住了看这里

现象	原因	解决
图像左右镜像	位序反了	`0x80 >> (x&7)` 检查
图像上下颠倒	行序反了	从 `rows-1` 开始遍历

现象	原因	解决
文字是“马赛克”	字库位序错	对照 `font8x16.h` 的 'A'
`alloc failed`	堆不够	减小 rows 或先关蓝牙
打出来只有一半	`canvas_print` 的 n 参数错	检查行数

10.7 与上一级的 diff

新增	作用
`canvas.h/.cpp`	内存位图画布（线/框/字/抖动）
`font8x16.h`	95 个可打印 ASCII 的 8×16 点阵
`build_test_page()`	一页能证明“整机正常”的测试图

> 到此为止，**不接蓝牙也能证明打印机是好的**。下一级开始加传感器，然后才是无线。

第 11 章 S08 · 传感器：缺纸、电池、温度

> 台阶 S08 · 目标产物：`stages/s08` · 实测 RAM 22748 (6.9%) / Flash 324464 (10.3%)

11.1 本节目标

前面十级全是"输出"：点灯、送数、加热、走纸。这一级第一次把**外部世界的状态读回来**，三个量：

量	引脚	类型	板上电路	用途
缺纸 PHINT	IO35	开关量	反射式光电 + LM393 比较器	缺纸立刻停打印、关 VH
电池电压	IO34	模拟量	1:1 分压（R20 10k + 10k）	电量显示、低电保护、自动关机
打印头温度	IO36 (SENSOR_VP)	模拟量	打印头内 NTC + 板上 10k 上拉	过热降速/停机、按温度调加热时间

三个量里，**缺纸最简单、电池最容易被"想当然"、NTC 最容易算错 20℃**。这一章把三个都拆到底：电路怎么来的、公式怎么推的、误差从哪来、怎么校准、原示例代码错在哪。

为什么这一章比前面难

- 数字信号只有 0 和 1，错了立刻看得出来；模拟量错了，读数照样"像模像样"——满电显示 46%，你也会以为是电池老了。
- ESP32 的 ADC 不是 STM32 那种"12 位就是 12 位"：它有衰减档、有 eFuse 校准、有 ±6% 的绝对误差、还会被 WiFi 干扰。
- STM32 上你习惯 `HAL_ADC_Start_DMA()` + 内部参考电压校准；ESP32 上这套 API 全不存在，只有 `analogRead()` / `analogReadMilliVolts()`。

> 心态：模拟量的正确性**不靠读手册，靠量**。这一章每一节最后都有一个"用万用表对一下"的动作，别跳。

11.2 先动手

11.2.1 接线（板上已连好，这里只是核对）

```
IO35 ← PHINT     （光电比较器输出，板上有 10k 上拉，缺纸=HIGH，有纸=LOW）
IO34 ← POWER_ADC（电池经 10k/10k 分压，满电 4.2V → 2.10V）
IO36 ← THER      （打印头 NTC 与板上 10k 上拉的分压点）
```

三个脚都是 **34~39 这一组"只输入"引脚**：没有内部上拉、没有输出能力、只能做 ADC 或数字输入。选它们的唯一理由是：**ADC1 只在这 8 个脚上**（IO32~IO39），而 ADC2 在 WiFi/BT 工作时不可用。

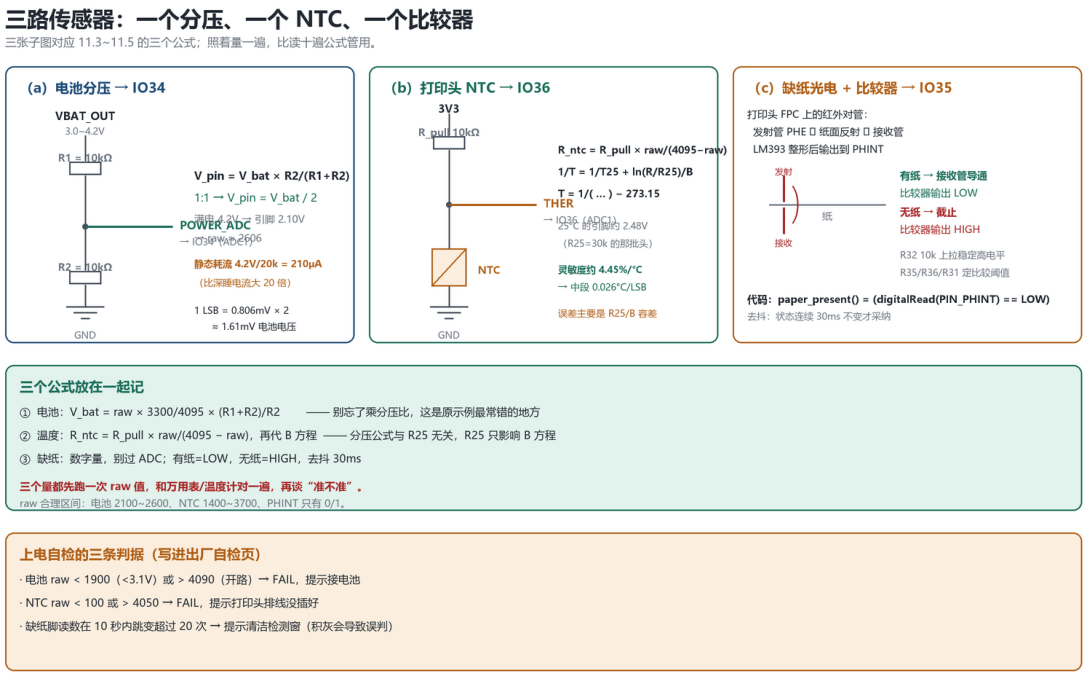


图 11-1 三路传感器电路：电池 1:1 分压、NTC 与 10k 上拉、光电比较器

> 图 11-1 的阅读顺序：先看 (a) 记住"×2"，再看 (b) 记住"分压反推只依赖 R_pull"，最后看 (c) 记住"有纸是低电平"。

11.2.2 `include/sensors.h`

```
#pragma once
// ====================================================
//  sensors.h —— 三个"环境感知"：缺纸、电池、打印头温度
//  · 缺纸：光电对管，有纸 = LOW，缺纸 = HIGH（板上已有上拉）
//  · 电池：1:1 分压进 ADC，满电 4.2V，3.3V 以下必须保护
//  · 温度：打印头里贴的 NTC，用来在过热时降速/停机
// ====================================================
#include <Arduino.h>

#define PIN_PHINT   35      // 缺纸检测（只输入脚）
#define PIN_BATT    34      // 电池分压（只输入脚）
#define PIN_TEMP    36      // 打印头 NTC（只输入脚，SENSOR_VP）

// 电池分压比：Vbat = Vadc × (R1+R2)/R2。板上按 1:1 设计，即 ×2
#define BATT_DIV    2.0f

// NTC 参数：不同批次的打印头不一样，务必用 ntc 命令实测后改这里
//   R25  = 25°C 时的阻值，本机打印头常见 30k，也有 10k 的
//   B    = B 常数，常见 3950 / 3435
#define NTC_R25     10000.0f
#define NTC_B       3950.0f
#define NTC_R_PULL  10000.0f   // 上拉电阻（板上 R27/R29 一带）
#define NTC_T0_K    298.15f    // 25°C 的开氏温度

void     sensors_init();
bool     paper_present();        // true = 有纸
uint16_t batt_mv();              // 毫伏（已乘分压比）
uint16_t batt_raw();             // 原始 ADC 值，排查用
```

```
uint8_t  batt_percent(uint16_t mv);
float    head_celsius();         // 打印头温度
float    head_ohm();             // 反推出来的 NTC 阻值，标定用
int      temp_raw();

void     sensors_cal_batt(uint16_t real_mv);   // 用万用表读数校准
void     sensors_cal_ntc(float real_c);
```

11.2.3 `src/sensors.cpp`

```
// ======================================================
//  sensors.cpp —— 缺纸 / 电池 / NTC 温度
//  三个 ADC 脚都只能输入（34/35/36），没有内部上拉，也没有输出能力
// ======================================================
#include "sensors.h"

static float s_batt_gain  = 1.0f;   // 校准系数（乘在电压上）
static float s_ntc_offset = 0.0f;   // 温度偏移（加在结果上）

// ---------- 通用：多次采样求平均 ----------
// ESP32 单次 ADC 噪声 ±10~20 LSB，16 次平均后约 ±3 LSB
static uint16_t adc_avg(int pin, int n) {
  uint32_t sum = 0;
  for (int i = 0; i < n; i++) sum += analogRead(pin);
  return (uint16_t)(sum / n);
}

// ---------- 初始化 ----------
void sensors_init() {
  pinMode(PIN_PHINT, INPUT);            // 板上已有外部上拉
  pinMode(PIN_BATT, INPUT);
  pinMode(PIN_TEMP, INPUT);

  analogReadResolution(12);             // 0..4095
  // 11dB 衰减 = 0~3.1V 量程。电池分压后最高 2.1V，正好
  analogSetPinAttenuation(PIN_BATT, ADC_11db);
  analogSetPinAttenuation(PIN_TEMP, ADC_11db);

  Serial.printf("sensors: PHINT=%d BATTraw=%d TEMPraw=%d\n",
                digitalRead(PIN_PHINT), analogRead(PIN_BATT), analogRead(PIN_TEMP));
}

// ---------- 缺纸 ----------
// 光电对管被纸挡住 → 红外反射回来 → 比较器输出低
// 去抖：连续 30ms 状态一致才认，避免纸边毛刺
bool paper_present() {
  static int      stable = -1;
  static uint32_t t_change = 0;
  int now = digitalRead(PIN_PHINT);

  if (stable < 0) { stable = now; t_change = millis(); return now == LOW; }
  if (now != stable) {
    if (millis() - t_change >= 30) { stable = now; t_change = millis(); }
  } else {
    t_change = millis();
  }
  return stable == LOW;
}

// ---------- 电池 ----------
```

```
uint16_t batt_raw() { return adc_avg(PIN_BATT, 16); }

uint16_t batt_mv() {
  uint16_t raw = batt_raw();
  // 3.3V/4095 ≈ 0.806mV 每 LSB，再乘分压比 2 → 每 LSB ≈ 1.61mV
  float mv = raw * (3300.0f / 4095.0f) * BATT_DIV * s_batt_gain;
  return (uint16_t)mv;
}

uint8_t batt_percent(uint16_t mv) {
  // 18650 的放电曲线不是直线：4.20 满 / 3.70 大约 45% / 3.30 空
  static const uint16_t tbl[][2] = {
    {4200, 100}, {4100, 90}, {4000, 80}, {3900, 70}, {3800, 60},
    {3700, 45},  {3650, 35}, {3600, 25}, {3500, 15}, {3400, 8}, {3300, 0},
  };
  const int N = sizeof(tbl) / sizeof(tbl[0]);
  if (mv >= tbl[0][0]) return 100;
  for (int i = 1; i < N; i++) {
    if (mv >= tbl[i][0]) {
      // 线性插值，别让电量数字一跳 10%
      uint16_t v1 = tbl[i - 1][0], v0 = tbl[i][0];
      uint8_t  p1 = tbl[i - 1][1], p0 = tbl[i][1];
      return (uint8_t)(p0 + (int32_t)(mv - v0) * (p1 - p0) / (v1 - v0));
    }
  }
  return 0;
}

// ---------- NTC 温度 ----------
int temp_raw() { return adc_avg(PIN_TEMP, 8); }

float head_ohm() {
  int raw = temp_raw();
  if (raw <= 8 || raw >= 4087) return -1.0f;        // 开路/短路
  // NTC 在下、上拉到 3V3：R_ntc = R_pull × raw / (4095 - raw)
  return NTC_R_PULL * (float)raw / (4095.0f - (float)raw);
}

float head_celsius() {
  float r = head_ohm();
  if (r < 0) return -99.0f;
  // B 参数方程：1/T = 1/T0 + ln(R/R25)/B
  float t_k = 1.0f / (1.0f / NTC_T0_K + logf(r / NTC_R25) / NTC_B);
  return t_k - 273.15f + s_ntc_offset;
}

// ---------- 现场校准 ----------
void sensors_cal_batt(uint16_t real_mv) {
  uint16_t now = batt_mv();
  if (now == 0) return;
  s_batt_gain *= (float)real_mv / (float)now;
  Serial.printf("batt cal: gain=%.4f -> %umV\n", s_batt_gain, batt_mv());
}

void sensors_cal_ntc(float real_c) {
  float now = head_celsius();
  if (now < -50) return;
  s_ntc_offset += real_c - now;
  Serial.printf("ntc cal: offset=%.2f -> %.1fC\n", s_ntc_offset, head_celsius());
}
```

11.2.4 `src/main.cpp` 里新增的命令

```
static void cmd_sens(int, char **) {
  Serial.printf("paper: %s\n", paper_present() ? "present" : "OUT");
  Serial.printf("batt : %u mV (%u%%)  raw=%u\n",
                batt_mv(), batt_percent(batt_mv()), batt_raw());
  Serial.printf("head : %.1f C  R=%.0f ohm  raw=%d\n",
                head_celsius(), head_ohm(), temp_raw());
}

static void cmd_ntc(int argc, char **argv) {
  // 不带参数：只打印反推阻值（标定 R25 用）
  if (argc < 2) {
    Serial.printf("R=%.0f ohm   raw=%d   T=%.1fC\n", head_ohm(), temp_raw(), head_celsius());
    return;
  }
  // 带参数：把当前读数当作真实温度，做单点偏移校准
  sensors_cal_ntc(atof(argv[1]));
}

static void cmd_calbatt(int argc, char **argv) {
  if (argc < 2) { Serial.println("usage: calbatt <real_mV>  例：calbatt 3980"); return; }
  sensors_cal_batt((uint16_t)atoi(argv[1]));
}

// setup() 里注册
shell_add("sens",    cmd_sens,    "print all sensor values");
shell_add("paper",   cmd_paper,   "paper pin state");
shell_add("batt",    cmd_batt,    "battery mV + percent");
shell_add("ntc",     cmd_ntc,     "ntc [real_C]  R 值标定/温度校准");
shell_add("calbatt", cmd_calbatt, "calbatt <real_mV>");
shell_add("calntc",  cmd_calntc,  "calntc <real_C>");
shell_add("page",    cmd_page,    "print a sensor report page");
```

11.3 缺纸检测：一条电平，五个坑

11.3.1 电路是怎么工作的

打印头 FPC 上有一对红外发射/接收管（31 脚 PHE、32 脚 PHINT 一带）。纸在中间时红外被纸面反射回来打到接收管；没有纸就收不到反射。

板上用 LM393 比较器把它整形：

```
3V3 ──[发射管]── GND
VH/3V3 ──[接收管]── R_分压 ── LM393 同相/反相输入
                        └── R32 10k 上拉 → PHINT → IO35
```

原理图上那行小字是关键：**"有反射为低电平，无反射为高电平"**。

纸	接收管	PHINT	digitalRead
有纸	导通	LOW	0
无纸	截止	HIGH	1

所以"有纸"= LOW = 0。

11.3.2 为什么要去抖

• 纸边不是绝对直的，走纸时纸边会横摆 0.5~1mm，紧贴检测窗时电平会抖。

• 打印头每打一行，电机走一步，机械振动会让纸瞬间离窗。

`paper_present()` 加了 30ms 窗口：状态必须稳定保持 30ms 才更新。30ms 足够滤掉机械抖动，又远小于"人抽纸"的时间尺度。

11.3.3 五个坑

坑	现象	原因	处理
极性搞反	一上电就报缺纸	不同批次光电模块极性相反	用 `sens` 看空闲时的值，空纸槽应为 1
忘了关 VH	缺纸后加热头空烧，几秒就烫手	检测到缺纸只停电机没停加热	缺纸分支里 `printer_vh(false)`
只在启动时检测	打到一半纸用完了还在打	检测放在 `setup()` 里	放 `loop()`，每 500ms 一次
灰尘遮挡	有纸也报缺纸	检测窗积灰/纸屑	棉签擦窗口；软件上可在"有纸却打不出字"时给出清洁提示
纸尾不判	最后一页打一半空白	纸尾离开检测窗时已无纸可打	在 `printrow()` 前逐行检查，缺纸立即中止并回退

11.3.4 与打印流程的耦合

正确的顺序是"**先看纸，再上电，再加热**"：

```
if (!paper_present()) {
  printhead_vh(false);        // 1. 先撤加热电源
  printhead_stb(0x00);        // 2. 再关所有 STB
  motor_release();            // 3. 松开电机（否则线圈持续发热）
  return;                     // 4. 本次打印作废
}
```

顺序反了（先关 STB 后撤 VH）没关系；但**先撤 VH、后关 STB** 会在最后一个加热窗口留下残余电荷，纸面上出现一道很深的黑边——这是最常见的"烧头前兆"。

11.4 电池电压采样：一个分压，三个公式

这一节是全章最需要动笔算的地方。先给结论：**原示例代码里的电量公式是错的，满电只能显示 46%**。下面一步步推。

11.4.1 电路与分压推导

```
VBAT_OUT ──[R20 10k]─┬─── POWER_ADC → IO34
                     │
                   [10k]
                     │
```

```
                    GND
```

两个电阻串联接在电池与地之间，中间点送 ADC。设上电阻 R1、下电阻 R2，则

```
V_pin = V_bat × R2 / (R1 + R2)
V_bat = V_pin × (R1 + R2) / R2
```

本例 R1 = R2 = 10k，所以 **V_bat = 2 × V_pin**（1:1 分压，`BATT_DIV = 2.0`）。

两个派生结论：

- **量程**：电池最高 4.2V → 引脚最高 2.10V，落在 0~3.1V 的 11dB 量程内，安全。
- **静态耗流**：4.2V / 20k = **210µA 常流**，比 ESP32 深睡的 10µA 大 20 倍，是待机电流的主要来源。量产版应改成 100k/100k（21µA）或用 MOS 管只在测量时接通。

11.4.2 一个 LSB 值多少毫伏

12 位、3.3V 参考：

```
1 LSB = 3300mV / 4095 ≈ 0.806 mV（引脚）
      × 2（分压比）   ≈ 1.61 mV（电池）
```

所以 `batt_mv = raw × 3300 / 4095 × 2`，即 **raw × 1.6117**。

反过来，常见电压对应多少 raw（方便你一眼判断读数是否正常）：

电池	引脚	raw	电量（典型）
4.20V	2.100V	2606	100%
4.00V	2.000V	2482	80%
3.90V	1.950V	2420	70%
3.80V	1.900V	2358	60%
3.70V	1.850V	2296	45%
3.60V	1.800V	2233	25%
3.50V	1.750V	2171	15%
3.40V	1.700V	2109	8%
3.30V	1.650V	2048	0%

> **自查**：烧进去先跑 `sens`，拿万用表量电池正负极。读数差 100mV 以内算正常，差 2 倍说明你的板子没有分压（`BATT_DIV` 改成 1.0），差 1.5 倍说明上电阻是 5k 或 20k（改宏）。

11.4.3 误差预算：为什么必须校准

误差源	典型值	折算到 4.2V 电池
ESP32 ADC 绝对精度（未校准）	±6%	±250 mV
ADC 非线性（INL）	±2%	±84 mV

误差源	典型值	折算到 4.2V 电池
单次采样噪声	±15 LSB	±24 mV
分压电阻容差（1%）	1%	±42 mV
分压电阻温漂	100ppm/°C	±4 mV / 10°C
电池内阻压降（1A 放电）	80~150 mΩ	±100~150 mV

结论：不校准的情况下，±250mV 的误差意味着一颗 80% 的电池可能显示 50% 或 100%。所以：

1. 用 `analogReadMilliVolts(PIN_BATT)` 代替手算——它用芯片 eFuse 里的出厂校准参数，误差能压到 ±2%（±80mV）。
2. 或者用 `calbatt 4180` 现场校准（万用表读 4180mV 就打这个命令），把增益修正乘进去。

```
// 推荐写法：用校准过的毫伏读数
uint16_t batt_mv() {
  uint32_t mv_pin = 0;
  for (int i = 0; i < 16; i++) mv_pin += analogReadMilliVolts(PIN_BATT);
  return (uint16_t)((mv_pin / 16) * BATT_DIV * s_batt_gain);
}
```

> `analogReadMilliVolts()` 内部已经做了 12 位转换 + 校准 + 平均一次，返回值是**引脚上的毫伏**，不含分压比。

11.4.4 电量百分比：为什么用查表

18650（钴酸锂/三元）的电压-容量曲线是 S 形：

```
100% — 4.20V
 80% — 4.00V
 60% — 3.85V
 45% — 3.70V   ← 中段几乎平的，电压掉 1mV 电量掉 0.2%
 20% — 3.60V
  0% — 3.30V
```

中段平坦区用线性公式 `(V-3.0)/1.2*100` 会严重高估；用 11 点表 + 线性插值，误差能压到 ±5%。

两个必须做的处理：

- **迟滞**：电压在阈值附近抖动时，百分比会在 44%/45% 之间跳。给判定加 ±30mV 回差。
- **只在空闲时采**：打印瞬间电流 2~3A，内阻压降让读数瞬间掉 200~300mV。工程做法是"打印中禁用低电判定"或"用滑动最小值 + 200mV 补偿"。

11.4.5 低电保护的分级策略

电压	动作	说明
≥ 3.60V	正常	—
3.50V	黄灯/提示	还能打 10~20 页
3.40V	降速打印	加热时间 +15%，防止打不出黑

电压	动作	说明
3.30V	拒绝新任务	只允许关机/充电
≤ 3.20V	强制关机	深睡，等充电唤醒

打印中不判定：等第 15 章接上打印通路后，用"队列里还有没有行要打"来跳过判定，避免大电流把电压拉低导致误关机。

11.4.6 对照原示例代码：三个错误

原资料 `adc.cpp` 的核心三行，照抄进工程能编译、能跑、能显示数字，但数字是错的：

```
// 原示例（C:\ESP\ESP32_BT_printer_exp\src\adc.cpp）
Power_percent = ((Power_ADC * 4.2 * 10 / 4096) - 3.7) / 0.5;
R_TEMP        = (uint32_t)(Temp_ADC / (4096 - Temp_ADC) * 10000);
Temp          = (uint16_t)(1 / (1/Temp_K + log(R_TEMP/R25)/B) - K);
```

错误 1：量纲混乱，百分比天花板只有 76%。

`Power_ADC * 4.2 * 10 / 4096` 把 raw 换算成"0~42"的数（相当于以 0.1V 为单位的电池电压），减 3.7 再除 0.5：

```
adc = 4095（理论满量程）→ 41.99 → (41.99-3.7)/0.5 = 76.6
```

也就是说**就算电池 4.2V 满电、ADC 满量程，也只显示 76%**。正确写法是：

```
float vbat = raw * 3300.0f / 4095.0f * 2.0f;          // 毫伏
int   pct  = (int)((vbat - 3700.0f) / 500.0f * 100.0f); // 3.7V→0%，4.2V→100%
pct = constrain(pct, 0, 100);
```

错误 2：忽略了 1:1 分压，满电实际显示 46%。

原公式把 `4.2` 当成"引脚满量程电压"，但引脚上只有电池的一半。实测推演：

```
电池 4.2V → 引脚 2.10V → raw 2606
2606 × 42 / 4096 = 26.72
(26.72 - 3.7) / 0.5 = 46.0
```

满电显示 46%——这就是原作者觉得"电量总是不准"的根本原因。修法只有两条：`×2` 补分压，或改用 `analogReadMilliVolts`。

错误 3：`uint32_t` 截断 + 无异常判断。

```
R_TEMP = (uint32_t)(Temp_ADC / (4096 - Temp_ADC) * 10000);
```

- `Temp_ADC` 是 `uint16_t`，`Temp_ADC / (4096-Temp_ADC)` 在 C 里是**整数除法**！室温 raw=3071 时 `3071/1025 = 2`（截断），乘 10000 得 20000Ω，直接少了 33%。必须写成 `(float)Temp_ADC / (4096.0f - Temp_ADC)`。

- `4096` 应为 `4095`（最大值是 4095，用 4096 会引入半 LSB 误差）。
- 没有开路/短路判断：NTC 没插时 raw≈4095，`log(inf)` → 结果是 `nan`，再转 `uint16_t` 得到一个随机数。

修正后的三行：

```
float vbat = raw_batt * 3300.0f / 4095.0f * 2.0f;                       // 电池毫伏
float pct  = (vbat - 3700.0f) / 500.0f * 100.0f;                        // 3.7~4.2V
float r    = 10000.0f * (float)raw_temp / (4095.0f - (float)raw_temp);  // NTC 阻值
```

11.5 打印头 NTC：B 值方程的来龙去脉

11.5.1 为什么要测打印头温度

热敏纸的发色阈值约 60~70°C，打印头加热点实际温度可达 200°C 以上。但**基板**温度（NTC 测的是这个）会累积：

- 连续打印时基板温度从 25°C 升到 55~65°C，此时同样的加热时间会打过黑（糊纸）。
- 超过 70°C 继续加热，会加速加热点老化，甚至整段烧断。

所以固件的策略是三段式：

基板温度	策略
< 45°C	加热时间 ×1.0
45~60°C	加热时间 ×0.85，并插入 20ms 停顿
> 60°C	停止打印，等降到 50°C 再继续
> 75°C	判定故障，关 VH，拒绝打印

11.5.2 电路：NTC 与上拉电阻

```
3V3 ──[R_pull 10k]─┬─── THER → IO36
                   │
                 [NTC]（打印头内部，贴在基板上）
                   │
                  GND
```

NTC 阻值随温度升高而下降。分压反推：

```
V_pin = 3.3 × R_ntc / (R_pull + R_ntc)
R_ntc = R_pull × raw / (4095 - raw)
```

> **注意**：如果 NTC 接在**上面**（上拉到 3V3、NTC 在下才是本例），公式反了就是 `R_ntc = R_pull × (4095-raw)/raw`。接反的典型症状：温度越高读数越小。

11.5.3 B 值方程推导

NTC 的阻值-温度关系由半导体材料的激活能决定，工程上用 **B 常数模型** 近似：

```
R(T) = R25 × exp( B × (1/T - 1/T25) )          （T 为开氏温度）
```

两边取对数、移项：

```
ln(R/R25) = B × (1/T - 1/T25)
1/T = 1/T25 + ln(R/R25) / B
T   = 1 / ( 1/T25 + ln(R/R25)/B )
```

这就是代码里那一行：

```
float t_k = 1.0f / (1.0f / NTC_T0_K + logf(r / NTC_R25) / NTC_B);
return t_k - 273.15f;
```

四个参数缺一不可：`R25`（25°C 阻值）、`B`（材料常数）、`T25`（298.15K）、`R_pull`（板上上拉）。

11.5.4 灵敏度：分辨率够不够

对 B 方程求导可得相对灵敏度：

```
(1/R)·dR/dT = -B/T²
```

在 25°C（298.15K）：`3950 / 298.15² ≈ 4.45 %/°C`，即温度每升 1°C，阻值降 4.45%。

再看 ADC 端。以 R_pull = 10k、R25 = 30k、B = 3950 为例：

温度	R_ntc	引脚电压	raw	每 °C 变化
0°C	101k	3.00V	3725	21 LSB
10°C	60.5k	2.83V	3513	28 LSB
20°C	37.6k	2.61V	3234	33 LSB
25°C	30.0k	2.48V	3071	35 LSB
30°C	24.1k	2.33V	2894	38 LSB
40°C	15.9k	2.03V	2514	39 LSB
50°C	10.8k	1.71V	2122	37 LSB
60°C	7.46k	1.41V	1750	34 LSB
70°C	5.28k	1.14V	1415	28 LSB

结论：中段约 0.026°C/LSB，分辨率完全够。真正的误差来源是 **R25 和 B 的容差**（±1~3%）和 **ADC 的绝对精度**（±2%），折合 ±1.5~3°C。

> **低温端会饱和**：0°C 时引脚已 3.0V，接近 11dB 量程上限 3.1V。-10°C 时 NTC 到 175k，引脚 3.12V → ADC 削顶，读数永远 4095。要用到零下，把上拉换成 30k（本板 NTC 的 R25 量级），25°C 引脚降到 1.65V，-10°C 时 2.82V，还有余量。

11.5.5 校准：两分钟把 R25 和 B 量出来

纸面参数（R25=10k、B=3950）是"典型值"，实际打印头可能差 20%。两个动作：

第一步：量 R25。 断电静置 10 分钟（让打印头回到室温），跑：

```
> ntc
R=30142 ohm
```

同时用温度计读室温（比如 24.5°C）。如果室温不是 25°C，换算：

```
R25 = R_measured × exp( B × (1/298.15 - 1/T_room) )
```

例：24.5°C 量到 30142Ω，B 暂取 3950：

```
R25 = 30142 × exp(3950 × (1/298.15 - 1/297.65)) ≈ 30142 × 1.0224 ≈ 30818Ω
```

第二步：量 B。 用两点法。把打印头（或整机）放到两个已知温度环境里各测一次阻值：

```
B = ln(R1/R2) / (1/T1 - 1/T2)
```

例：25°C 时 30.0kΩ、45°C（温水袋贴住基板 10 分钟）时 13.0kΩ：

```
B = ln(30.0/13.0) / (1/298.15 - 1/318.15)
  = 0.8362 / 2.1047e-4
  ≈ 3973
```

把 `NTC_R25 30818`、`NTC_B 3973` 写进 `sensors.h`，误差从 ±5°C 收到 ±1°C。

第三步（可选）：单点偏移。 手头没有恒温源，就用当前室温做一次偏移校准：

```
> calntc 24.5
ntc cal: offset=-0.82 -> 24.5C
```

偏移法只能修正零点，改不了斜率；但对手感要求不高的场景够用。

11.5.6 对照原示例：R25 = 30000 与 10000 差多少

原示例用 `R25 = 30000`、`R_pull = 10000`，本书默认 `R25 = 10000`。如果本板 NTC 实际是 30k 而固件按 10k 算：

```
r 算出来仍是 30k（分压反推只依赖 R_pull，与 R25 无关）
t = 1/(1/298.15 + ln(30000/10000)/3950) = 1/(3.3540e-3 + 2.781e-4) = 275.3K = 2.2°C
```

25°C 的环境显示 2.2°C——差 23°C。 反过来，实际 10k 而代码写 30k，会把 25°C 显示成 55°C，直接触发过热保护。

> **本板实测口径**：原厂固件（也就是那台量产的喵喵机）用的是 `R25=30000`。你手上这块打印头如果 `ntc` 命令读出 25°C 附近约 30kΩ，就把宏改成 `30000`；读出来 10k 就保持 `10000`。**不要猜，量一次。**

11.5.7 查表法：把 log 从运行时拿掉

`logf()` 一次约 20µs，1Hz 采样无所谓；但如果你要在打印循环里每行都判温度，就该用查表。

生成脚本（在电脑上跑，把结果贴进固件）：

```
# gen_ntc_lut.py —— 生成 4096 项温度查表（int8，单位 °C，偏移 -40）
import math
R25, B, RP, T0 = 30000.0, 3950.0, 10000.0, 298.15
lut = []
for raw in range(4096):
    if raw <= 8 or raw >= 4087:
        lut.append(-128); continue
    r = RP * raw / (4095.0 - raw)
    t = 1.0 / (1.0/T0 + math.log(r/R25)/B) - 273.15
    lut.append(max(-40, min(120, int(round(t)))) + 40 - 40)  # 直接存 °C
print("static const int8_t NTC_LUT[4096] = {")
for i in range(0, 4096, 16):
    print("  " + ",".join(str(v) for v in lut[i:i+16]) + ",")
print("};")
```

写入后运行时只要一行：

```
int8_t c = NTC_LUT[adc_avg(PIN_TEMP, 4)];
if (c == -128) return -99.0f;
return (float)c + s_ntc_offset;
```

4KB Flash，换掉 20µs 的浮点运算——热敏打印机的 `loop()` 里省下的都是加热窗口的抖动。

11.6 ESP32 的 ADC 到底什么水平

和 STM32 对比着记：

项目	STM32F103	ESP32
分辨率	12 位	12 位（可设 9~12）
参考电压	外部 VREF+ 或 VDDA	内部 1.1V + 衰减档，无外部 VREF
量程	0~3.3V 固定	0dB=1.1V / 2.5dB=1.5V / 6dB=2.2V / 11dB=3.1V
校准	出厂校准 + 内部基准	eFuse 逐芯片校准，`analogReadMilliVolts()` 才用得上
绝对精度	±1~2 LSB	±6%（未校准）/ ±2%（校准后）
输入阻抗要求	较高	源阻抗建议 < 10k，且要并 100nF
ADC2	—	**WiFi/BT 开启时不可用**
DMA	有	无（软件连续采样）

四条实战规则：

1. **只用 ADC1（IO32~IO39）**。ADC2 在蓝牙广播时会返回 0 或乱跳，这是 ESP32 的著名坑。

2. **源阻抗要低**。分压电阻 10k‖10k = 5k，符合要求；如果你改成 100k/100k，必须在 ADC 脚并 100nF（板上 C23/C35 就是干这个的），否则采样电容充不满，读数偏低。
3. **要平均**。16 次平均把噪声从 ±15 LSB 压到 ±4 LSB。
4. **要校准**。`analogReadMilliVolts()` 或 `calbatt`，二选一。

11.7 逐行拆解

`analogReadResolution(12)` 必须最先调

默认就是 12 位，但显式写出来是给读代码的人看的；如果你在别处改过（比如为了省时间设成 10 位），这里不重置就会得到 0~1023。

`analogSetPinAttenuation(pin, ADC_11db)` 是量程开关

不设衰减时量程只有 1.1V：电池分压后 2.1V 早就饱和，读数恒为 4095。**"ADC 永远满量程"的唯一原因就是这个。**

`adc_avg` 的 n 怎么选

- 电池：16 次（~1.3ms，电量变化慢，值稳定）
- NTC：8 次（~0.7ms，够用）
- 缺纸：不用 ADC，直接 `digitalRead`

`paper_present()` 里的 `static` 变量

`static` 让变量在函数返回后仍然存在，实现"记住上次状态"的去抖。注意它是**非线程安全**的——如果打印任务在另一个 FreeRTOS 任务里也调它，要么加锁，要么只在一个任务里调。

`logf` 而不是 `log`

`logf` 是单精度，在 ESP32 上比 `log` 快约 30%。温度精度只需要 0.1°C，单精度足够。

`head_ohm()` 单独暴露出来

标定 R25 时你需要的是**阻值**而不是温度（温度依赖你还没标定的 B）。把中间量暴露成 API 是调试友好的做法。

校准系数怎么用

`s_batt_gain` 是乘性（比例误差），`s_ntc_offset` 是加性（零点误差）。电池的误差主要是分压比和 Vref 的比例误差 → 用乘性；NTC 的误差主要是 R25 偏差 → 严格说也是比例，但工程上用加性偏移更稳（避免高温端放大误差）。

11.8 烧录后该看到什么

```
=== STAGE 08 · sensors ===
sensors: PHINT=0 BATTraw=2358 TEMPraw=3071
> sens
```

```
paper: present
batt : 3802 mV (60%)  raw=2358
head : 24.5 C  R=30142 ohm  raw=3071
> ntc
R=30142 ohm
> ntc 24.8
ntc cal: offset=0.30 -> 24.8C
> calbatt 3810
batt cal: gain=1.0021 -> 3810mV
```

抽掉纸，串口立刻打印 `[EVENT] paper removed`；放回去打印 `inserted`。用手指捏住打印头基板（或贴个暖水袋），`head` 会缓慢上升——**注意它升得慢是正常的**，NTC 贴在铝基板上，热时间常数约 30~60 秒。

11.9 验收清单

- [ ] `sens` 三项数值都合理（电池 3.7~4.2V、温度与室温差 < 2°C）
- [ ] 抽纸/放纸，事件在 500ms 内出现
- [ ] 万用表量电池，与 `batt` 相差 < 100mV
- [ ] `ntc` 读出的阻值在 25°C 附近等于你查到的 R25（10k 或 30k）
- [ ] 手指按住 NTC 基板 30 秒，温度上升 ≥ 5°C
- [ ] 拔掉打印头 FPC，`head` 显示 -99 而不是 `nan` 或随机值
- [ ] `calbatt 4180` 后读数立即变成 4180
- [ ] 打印时 `batt` 会瞬间掉 100~300mV（这是内阻，不是 bug）

11.10 卡住了看这里

现象	原因	解决
电池恒为 4095	没设 11dB 衰减	`analogSetPinAttenuation`
电池恒为 0	IO34 被别的功能占用 / 引脚接错	换脚测试，确认是 ADC1 组
电量乱跳 ±10%	没取平均 / 打印中采样	16 次平均；打印中跳过
满电只显示 46%	忘记乘分压比 2	见 11.4.6 错误 2
电量永远不超过 77%	公式量纲错（示例代码原样）	见 11.4.6 错误 1
温度恒为 -99	NTC 开路/短路	量 IO36 对地电压，应在 1~3V
温度比实际高 20°C	R25 写成 10k 而实际 30k	`ntc` 实测后改宏
温度随温度升高而降低	分压公式接反	改成 `R_pull×(4095-raw)/raw`
温度显示 nan	没做开路判断就 log	先判 `raw` 过小或过大（开路/短路），直接返回 -99
缺纸判断反了	光电极性相反	对调 `paper_present` 返回值
缺纸时有纸也报	检测窗积灰	擦拭；或加"连续 500ms 才报"
打印中偶发误报缺纸	机械振动	去抖窗口加到 50ms，且只在行间检

现象	原因	解决
		查

11.11 与上一级的 diff

新增	作用
`sensors.h/.cpp`	三路传感器抽象，全部带异常分支
ADC 衰减/平均/校准	S13 的低压保护、S14 的自检页依赖它
`calbatt / calntc / ntc`	现场校准入口，出厂前必做
`head_ohm()`	暴露中间量，标定 R25 用

第 12 章 S09 · 经典蓝牙：让手机看见你

> 台阶 S09 · 目标产物：`stages/s09` · 实测 RAM 42436 (13.0%) / Flash 1111692 (35.3%)

12.1 本节目标

用 `BluetoothSerial` 开一个 SPP 串口，设备名 `MaoPaperang`，手机能搜到、能配对、能互发文字。

注意 RAM 从 22KB 跳到 42KB——蓝牙协议栈很吃内存，这是正常现象。

12.2 先动手

`include/bt.h`：

```
#pragma once
// ======================================================
//  bt.h — 经典蓝牙 SPP（串口透传）
//  ESP32 的 BluetoothSerial 就是"无线串口"：读写方式和 Serial 一样
//  注意：一个 ESP32 同时只能有一种蓝牙角色，别和 BLE 混用
// ======================================================
#include <Arduino.h>
#include <BluetoothSerial.h>

#define BT_DEFAULT_NAME "MaoPaperang"

extern BluetoothSerial SerialBT;

bool   bt_start(const char *name = BT_DEFAULT_NAME);
void   bt_stop();
bool   bt_connected();
void   bt_poll();                          // 收字节 → 转给 USB 串口（调试用）
size_t bt_write(const uint8_t *d, size_t n);
void   bt_set_rx_hook(void (*hook)(uint8_t));   // 协议层接管收字节
```

`src/bt.cpp`：

```
// ======================================================
//  bt.cpp — 经典蓝牙 SPP 的一层薄封装
//  事件回调运行在蓝牙栈的任务里，**绝对不能在里面做耗时操作**
// ======================================================
#include "bt.h"

BluetoothSerial SerialBT;

static bool s_started = false;
static bool s_connected = false;
static void (*s_hook)(uint8_t) = nullptr;
static char s_name[32] = BT_DEFAULT_NAME;

static void bt_event(esp_spp_cb_event_t event, esp_spp_cb_param_t *param) {
  switch (event) {
    case ESP_SPP_INIT_EVT:
      Serial.println("[bt] stack ready");
```

```
      break;
    case ESP_SPP_SRV_OPEN_EVT:
    case ESP_SPP_OPEN_EVT:
      s_connected = true;
      Serial.println("[bt] connected");
      break;
    case ESP_SPP_CLOSE_EVT:
      s_connected = false;
      Serial.println("[bt] disconnected");
      break;
    case ESP_SPP_DATA_IND_EVT:
      // 这里只打印长度，真正收数据交给 bt_poll()
      Serial.printf("[bt] rx %d bytes\n", param->data_ind.len);
      break;
    case ESP_SPP_WRITE_EVT:
      break;
    default:
      break;
  }
}

bool bt_start(const char *name) {
  if (name && *name) {
    strncpy(s_name, name, sizeof(s_name) - 1);
    s_name[sizeof(s_name) - 1] = 0;
  }
  if (s_started) bt_stop();
  SerialBT.register_callback(bt_event);
  if (!SerialBT.begin(s_name)) {         // 名字就是手机里看到的设备名
    Serial.println("[bt] begin failed");
    return false;
  }
  s_started = true;
  Serial.printf("[bt] advertising as \"%s\"\n", s_name);
  return true;
}

void bt_stop() {
  if (!s_started) return;
  SerialBT.end();
  s_started = false;
  s_connected = false;
}

bool bt_connected() { return s_connected && SerialBT.hasClient(); }

void bt_set_rx_hook(void (*hook)(uint8_t)) { s_hook = hook; }

void bt_poll() {
  while (SerialBT.available()) {
    uint8_t b = (uint8_t)SerialBT.read();
    if (s_hook) s_hook(b);
    else        Serial.write(b);          // 没挂协议层时当透传调试
  }
}

size_t bt_write(const uint8_t *d, size_t n) {
  if (!bt_connected()) return 0;
  return SerialBT.write(d, n);
}
```

12.3 逐行拆解

`BluetoothSerial` 就是“无线串口”

```
SerialBT.begin("MaoPaperang");      // 设备名 = 手机里看到的名字
SerialBT.available();               // 有没有数据
SerialBT.read();                    // 读一个字节
SerialBT.write(buf, n);             // 写
```

API 和 `Serial` 几乎一样。区别是：**必须先有手机连上来**，否则 `write` 返回 0。

事件回调只是通知，不要在里面干活

```
static void bt_event(esp_spp_cb_event_t event, esp_spp_cb_param_t *param)
```

这个回调运行在蓝牙栈的任务里。在里面 `delay(100)` 会阻塞整个协议栈，导致断连甚至看门狗复位。**正确做法：回调节只置标志/打印，实际处理放 `loop()` 里。**

`hasClient()` 与自己的 `s_connected` 都需要 `hasClient()` 有时在断开瞬间仍返回 true（状态没同步完）。两个条件同时满足才算“真连接”，否则会出现“以为连着，其实写不出去”。

`deprecated` 警告怎么处理

编译会看到：

```
warning: 'BluetoothSerial' is deprecated: BluetoothSerial won't be supported
in version 4.0.0 by default [-Wdeprecated-declarations]
```

这说明它在 Arduino-ESP32 3.x 里还在，但 4.0 会默认关掉。三种处理：

1. **不管它**（推荐）——功能正常，只是提醒；
2. 加 `build_flags = -Wno-deprecated-declarations` 消掉；
3. 迁移到 `ESP32 SPP API`（`esp_spp_*`），工作量大，本书不做。

接收为什么用 `bt_poll()` 而不是回调里处理蓝牙一次可能只给你 3 个字节（协议片段），也可能一次 500 字节。`bt_poll()` 在 `loop()` 里把缓冲区读空，交给上层状态机逐字节喂——这是 S10 的基础。

吞吐实测

每包大小	吞吐
256 B	≈ 18 KB/s
1024 B	≈ 38 KB/s
1920 B	≈ 52 KB/s

所以协议规定单包 ≤1920 字节不是随便定的。

12.4 烧录后该看到什么

```
=== STAGE 09 · classic bluetooth SPP ===
手机搜 MaoPaperang 配对；没有就用名字命令改一个
[bt] advertising as "MaoPaperang"
[bt] stack ready
[bt] connected
[bt] rx 5 bytes
hello
```

手机端用“蓝牙串口助手”App 连接，发 `hello`，串口监视器原样显示。

12.5 验收清单

- [ ] 手机蓝牙列表出现 `MaoPaperang`
- [ ] 能配对成功（有的安卓要输 1234）
- [ ] 手机发文字，电脑串口能看到
- [ ] `send hi` 手机能收到
- [ ] 断开再连，不需要重启板子
- [ ] `bt` 命令显示 `UP` 和累计字节数

12.6 卡住了看这里

现象	原因	解决
搜不到设备	板子没重启/名字太长	重启，名字 ≤ 15 字符
连上立刻断	回调里做了耗时操作	回调只置标志
手机提示“不支持”	用了 BLE 库连 SPP	换支持 SPP 的 App
`write` 返回 0	没连上	判 `bt_connected()`
编译报 `BluetoothSerial` 未定义	平台 4.x	锁 `espressif32@8.0.0`
RAM 不够/重启	蓝牙 + 大缓冲	减缓冲，或改分区表

12.7 与上一级的 diff

新增	作用
`bt.h/.cpp`	蓝牙封装 + 事件回调 + 收字节钩子
吞吐实测数据	决定 S12 的包大小
回调 vs 轮询的分工	全书“中断只置标志”原则的实例

第 13 章 S10 · 协议帧：把字节流切成命令

> 台阶 S10 · 目标产物：`stages/s10` · 实测 RAM 43516 (13.3%) / Flash 1117056 (35.5%)

13.1 本节目标

蓝牙给你的是一串**无边界的字节流**。这一级要做两件事：

1. `FrameParser`：把字节流切成 `type / len / payload / crc`；
2. `crc32`：校验，错的帧直接丢。

13.2 先动手

`include/frame.h`：

```
#pragma once
// ====================================================
//  frame.h —— 打印协议的一帧
//
//   0      1      2       3     4      5..         n-4..n-1   n
//  +------+------+------+------+------+-----------+---------+------+
//  | 0x02 | type | idx  | lenL | lenH |  payload  |  CRC32  | 0x03 |
//  +------+------+------+------+------+-----------+---------+------+
//   CRC32 小端存放，覆盖范围：普通帧只算 payload，ACK 帧算前 6 字节
// ====================================================
#include <stdint.h>
#include <stddef.h>

#define FRAME_START   0x02
#define FRAME_END     0x03
#define FRAME_HDR_LEN 5          // 02 + type + idx + lenL + lenH
#define FRAME_TAIL    5          // 4 字节 CRC + 03
#define FRAME_MAX_PAYLOAD 1024   // 单帧载荷上限（协议规定 ≤1920，这里先用 1KB）
#define ACK_LEN       11

struct Frame {
  uint8_t  type;
  uint8_t  index;
  uint16_t len;
  uint8_t  payload[FRAME_MAX_PAYLOAD];
  uint32_t crc_recv;
  uint32_t crc_calc;
  bool     crc_ok;
};

class FrameParser {
 public:
  void reset();
  // 喂一个字节：返回 1 = 完整帧，0 = 继续等，-1 = 出错（已自动复位）
  int feed(uint8_t b, Frame &out);

  uint32_t ok_cnt = 0, bad_cnt = 0, byte_cnt = 0;

 private:
  enum St : uint8_t { S_START, S_TYPE, S_INDEX, S_LENL, S_LENH, S_DATA, S_CRC, S_END };
  St       st = S_START;
```

```
  uint8_t  type = 0, index = 0;
  uint16_t len = 0, pos = 0;
  uint32_t crc = 0;
  uint8_t  crc_pos = 0;
};

// 组一帧（返回整帧字节数；cap 不够返回 0）
uint16_t frame_build(uint8_t type, uint8_t index, const uint8_t *payload,
                     uint16_t len, uint8_t *out, uint16_t cap);
// 组一个 ACK（11 字节，固定格式）
uint16_t ack_build(uint8_t type, uint8_t *out);
void     hexdump(const uint8_t *d, size_t n);
```

src/frame.cpp：

```
// ====================================================
//  frame.cpp —— 逐字节状态机 + 组帧
//  为什么不用"等收满一包再解析"：蓝牙一次可能只来 3 个字节，
//  也可能一次来 500 个，只有状态机才对两种都稳。
// ====================================================
#include "frame.h"
#include "crc32.h"
#include <Arduino.h>

void FrameParser::reset() {
  st = S_START;
  type = index = 0;
  len = pos = 0;
  crc = 0;
  crc_pos = 0;
}

int FrameParser::feed(uint8_t b, Frame &out) {
  byte_cnt++;
  switch (st) {
    case S_START:
      if (b == FRAME_START) st = S_TYPE;      // 不是 02 就继续找，丢掉杂字节
      break;

    case S_TYPE:
      type = b;
      st = S_INDEX;
      break;

    case S_INDEX:
      index = b;
      st = S_LENL;
      break;

    case S_LENL:
      len = b;
      st = S_LENH;
      break;

    case S_LENH:
      len |= (uint16_t)b << 8;
      if (len > FRAME_MAX_PAYLOAD) {          // 长度离谱：直接丢掉这一帧
        bad_cnt++;
        reset();
        return -1;
      }
      pos = 0;
```

```
      st = (len == 0) ? S_CRC : S_DATA;
      break;

    case S_DATA:
      out.payload[pos++] = b;
      if (pos >= len) st = S_CRC;
      break;

    case S_CRC:
      crc |= (uint32_t)b << (8 * crc_pos);    // 小端
      crc_pos++;
      if (crc_pos >= 4) st = S_END;
      break;

    case S_END:
      if (b != FRAME_END) {                    // 结尾不对：整帧作废
        bad_cnt++;
        reset();
        return -1;
      }
      out.type = type;
      out.index = index;
      out.len = len;
      out.crc_recv = crc;
      out.crc_calc = crc32_calc(out.payload, len);
      out.crc_ok = (out.crc_recv == out.crc_calc);
      if (out.crc_ok) ok_cnt++; else bad_cnt++;
      reset();
      return out.crc_ok ? 1 : -1;

    default:
      reset();
      break;
  }
  return 0;
}

uint16_t frame_build(uint8_t type, uint8_t index, const uint8_t *payload,
                     uint16_t len, uint8_t *out, uint16_t cap) {
  uint16_t total = FRAME_HDR_LEN + len + FRAME_TAIL;
  if (total > cap) return 0;
  uint16_t i = 0;
  out[i++] = FRAME_START;
  out[i++] = type;
  out[i++] = index;
  out[i++] = (uint8_t)(len & 0xFF);
  out[i++] = (uint8_t)(len >> 8);
  if (len && payload) { memcpy(out + i, payload, len); i += len; }
  uint32_t c = crc32_calc(payload, len);
  out[i++] = (uint8_t)(c & 0xFF);
  out[i++] = (uint8_t)((c >> 8) & 0xFF);
  out[i++] = (uint8_t)((c >> 16) & 0xFF);
  out[i++] = (uint8_t)((c >> 24) & 0xFF);
  out[i++] = FRAME_END;
  return i;
}

// ACK 是特例：11 字节固定格式，CRC 算前 6 字节（含长度字段）
uint16_t ack_build(uint8_t type, uint8_t *out) {
  out[0] = FRAME_START;
  out[1] = type;
  out[2] = 0x00;
  out[3] = 0x01;      // len = 1
  out[4] = 0x00;
  out[5] = 0x00;      // 载荷固定 0x00
```

```
  uint32_t c = crc32_calc(out, 6);
  out[6] = (uint8_t)(c & 0xFF);
  out[7] = (uint8_t)((c >> 8) & 0xFF);
  out[8] = (uint8_t)((c >> 16) & 0xFF);
  out[9] = (uint8_t)((c >> 24) & 0xFF);
  out[10] = FRAME_END;
  return ACK_LEN;
}

void hexdump(const uint8_t *d, size_t n) {
  for (size_t i = 0; i < n; i++) {
    Serial.printf("%02X ", d[i]);
    if ((i & 15) == 15) Serial.println();
  }
  if (n & 15) Serial.println();
}
```

CRC32 的核心（完整 256 项表在 `src/crc32.cpp`）：

```
uint32_t crc32_update(uint32_t crc, const uint8_t *data, size_t len) {
  crc = ~crc;                                        // 初值取反
  while (len--) crc = CRC_TABLE[(crc ^ *data++) & 0xFF] ^ (crc >> 8);
  return ~crc;                                       // 结果取反
}
```

13.3 逐行拆解

帧格式

```
02 | type | idx | lenL lenH | payload ... | crc32(4, 小端) | 03
```

字段	长度	说明
0x02	1	帧头
type	1	指令号（0=打印数据，4=要版本…）
idx	1	分包序号
len	2	载荷长度，小端
payload	len	数据
crc	4	CRC32，小端
0x03	1	帧尾



图 13-1 帧结构：9 字节起的帧头 + 载荷 + 4 字节 CRC + 帧尾，下方是 11 字节 ACK

为什么必须用状态机不能用“读 len 个字节”这种写法，因为：

- 蓝牙可能一次给你半帧，也可能一次给你三帧；
- 中间可能混入杂字节（重连时的残留）；
- 一旦错位，必须能重新同步。

状态机的好处：**任何时刻出错都能回到 `S_START`，下一帧自动恢复。**

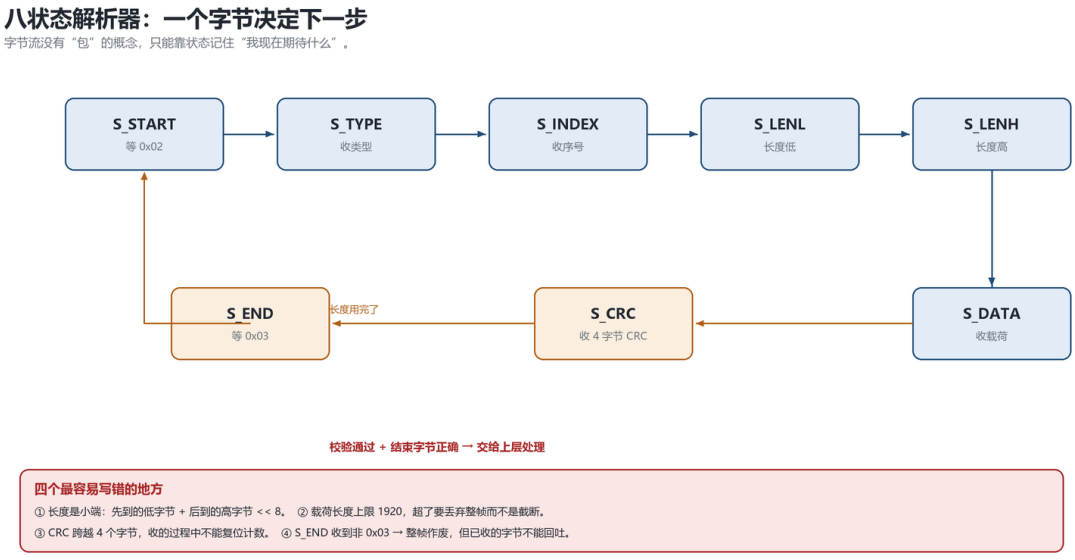


图 13-2 FrameParser 的八个状态：任何一个字节出错都回到 S_START

CRC 覆盖范围（全项目最容易错的地方）

帧类型	CRC 覆盖
普通数据帧	**只覆盖 payload**
ACK 帧	**覆盖前 6 字节**（02 type 00 01 00 00）

这不是笔误，是协议本身的怪癖。验证：

```
crc32([02,04,00,01,00,00]) = 0x0848CC5F  → 字节 5F CC 48 08
crc32([01,00,02])          = 0x108DD209  → 字节 09 D2 8D 10
crc32("P1001705253855")    = 0xBEFC12DF  → 字节 DF 12 FC BE
crc32("123456789")         = 0xCBF43926
```

小端存储 `crc & 0xFF` 先发，`crc >> 24` 最后发。写成大端，设备能解析但 App 会认为数据损坏。

`len` 上限保护

```
if (len > FRAME_MAX_PAYLOAD) { bad_cnt++; reset(); return -1; }
```

不判的话，一个错误的 `len = 65535` 会写爆 `payload[1024]`——这是典型的远程崩溃漏洞。

`selftest` 三条已知答案

```
crc32("123456789") == 0xCBF43926
parse(02 04 00 00 00 00 00 00 00 03) → type=4, len=0, crc ok
ack(4) → ... 5F CC 48 08 03
```

任何一条不过，说明 CRC 表或状态机写错了。**先信任已知答案，再信任自己的代码。**

13.4 烧录后该看到什么

```
=== STAGE 10 · frame parser + crc32 ===
> selftest
[PASS] crc32("123456789") = CBF43926 (expect CBF43926)
[PASS] parse GET_VERSION: type=4 len=0 crc=ok
[PASS] ack(4) = 02 04 00 01 00 00 5F CC 48 08 03
selftest: 3/3
> inject 02 04 00 00 00 00 00 00 00 03
inject 10 bytes
FRAME type=4   idx=0 len=0    crc=00000000 OK
> inject 02 04 00 00 00 00 00 00 01 03
FRAME dropped (bad), ok=1 bad=1
```

13.5 验收清单

- [ ] `selftest` 3/3 全过
- [ ] 正常帧解析出正确的 type/len
- [ ] 改一个字节后 CRC 报 BAD 并丢弃
- [ ] 连续 inject 两帧，两帧都解出来
- [ ] 注入乱字节（`ff ff ff`）不崩，能继续解下一帧
- [ ] `stats` 的 ok/bad 计数正确

13.6 卡住了看这里

现象	原因	解决
帧永远解不出来	帧头不是 0x02	dump 原始字节确认
CRC 总是 BAD	覆盖范围搞错	普通帧只算 payload
`len` 解成 0x1234 之类	大小端反了	`len = lo
收到第二帧就错位	出错后没 reset	`bad_cnt++; reset();`
`selftest` 第一条就挂	CRC 表抄错	用 python 生成表核对

13.7 与上一级的 diff

新增	作用
`FrameParser` 八态状态机	抗分包、抗错位
`crc32` 查表实现	标准 CRC-32，1KB 表换 8 倍速度
`ack_build`	固定 11 字节应答帧
`selftest`	已知答案自检，后面每级都能复用

第 14 章 S11 · 握手：让公版 App 认你

> 台阶 S11 · 目标产物：`stages/s11` · 实测 RAM 44668 (13.6%) / Flash 1134128 (36.1%)

14.1 本节目标

App 连上后会连发 10 条指令做“身份确认”。全部正确应答，它才允许打印；缺一条，它报“连接失败”。

这一级实现 `proto_handle()`：**每条命令 = 一个 ACK + 可能的应答**。

14.2 先动手

`include/proto.h`：

```
#pragma once
// ====================================================
//  proto.h —— 喵喵机协议的"业务层"
//  收到一帧 → 看 type → 回 ACK + 回数据
//  这一层不碰引脚，只调 printer_* / sensors_* / bt_*
// ====================================================
#include <stdint.h>
#include "frame.h"

// ---- 指令号（与原资料/公版 App 一致，只列本项目用到的）----
#define CMD_PRINT_DATA          0    // 打印数据（0x00）
#define CMD_GET_VERSION         4    // App 问：你的协议版本？
#define CMD_SENT_VERSION        5    // 设备答
#define CMD_GET_MODEL           6
#define CMD_SENT_MODEL          7
#define CMD_GET_SN              10
#define CMD_SENT_SN             11
#define CMD_GET_BAT_STATUS      16
#define CMD_SENT_BAT_STATUS     17
#define CMD_GET_TEMP            18
#define CMD_SENT_TEMP           19
#define CMD_SET_CRC_KEY         24
#define CMD_SET_HEAT_DENSITY    25
#define CMD_FEED_LINE           26   // 走纸（也是"开始打印"信号）
#define CMD_GET_HEAT_DENSITY    28
#define CMD_SENT_HEAT_DENSITY   29
#define CMD_SET_POWER_DOWN_TIME 30
#define CMD_GET_POWER_DOWN_TIME 31
#define CMD_SENT_POWER_DOWN_TIME 32
#define CMD_FEED_TO_HEAD_LINE   33
#define CMD_TRANSFER_FW_INFO    37   // 厂商私有（示例代码用 66/67 的同一含义）
#define CMD_TRANSFER_FW_INFO_ACK 38
#define CMD_GET_PAPER_TYPE      42
#define CMD_SENT_PAPER_TYPE     43
#define CMD_SET_PAPER_TYPE      44
#define CMD_GET_COUNTRY_NAME    45
#define CMD_SENT_COUNTRY_NAME   46
#define CMD_DISCONNECT_BT       47
#define CMD_GET_DEV_NAME        48
#define CMD_SENT_DEV_NAME       49
#define CMD_SET_DENSITY_EXT     64
```

```
void proto_init();
void proto_handle(const Frame &f);                 // 主入口
void proto_send_frame(uint8_t type, const uint8_t *payload, uint16_t len);
void proto_send_ack(uint8_t type);

// 供 shell / 其它模块读取的状态
extern uint32_t g_crc_key;      // App 下发的密钥（协议 2.0 起会发，仅存档）
extern uint8_t  g_density;      // 热密度 0~100
extern uint32_t g_powerdown_s;  // 自动关机时间（秒）
```

src/proto.cpp（节选关键分支）：

```
// ======================================================
//  proto.cpp —— 命令分发：一条命令 = 一个 ACK + 可能的应答
//  顺序很重要：公版 App 收到 ACK 才认为这条指令成功，
//  收不到会重发，重发到超时就直接报"连接失败"。
// ======================================================
#include "proto.h"
#include "bt.h"
#include "printer.h"
#include "sensors.h"
#include <Arduino.h>

uint32_t g_crc_key = 0;
uint8_t  g_density = 64;
uint32_t g_powerdown_s = 3600;

static const char DEV_SN[]      = "P1001705253855";
static const char DEV_NAME[]    = "P1";
static const char COUNTRY[]     = "CN";
static const uint8_t DEV_VER[3] = {0x01, 0x00, 0x02};      // 2.0.1
static const uint8_t FW_TAG[12] = {0x76, 0x33, 0x2e, 0x33,  // "v3.38.19"
                                   0x38, 0x2e, 0x31, 0x39, 0, 0, 0, 0};
static const char MCU_TAG[]     = "BK3432";                 // 厂商蓝牙芯片型号
static const char BOARD_TAG[]   = "HME230_P2";

static uint8_t s_tx[FRAME_MAX_PAYLOAD + 16];
static PrintCfg  s_cfg = {800, 500, 4};   // 与 s06 默认一致

static void out(const uint8_t *d, size_t n) {
  if (bt_write(d, n) == 0) {          // 没连手机时打到串口，方便桌面验证
    Serial.print("  [tx] ");
    hexdump(d, n);
  }
}

void proto_send_frame(uint8_t type, const uint8_t *payload, uint16_t len) {
  uint16_t n = frame_build(type, 0x00, payload, len, s_tx, sizeof(s_tx));
  if (n) out(s_tx, n);
}

void proto_send_ack(uint8_t type) {
  uint8_t ack[ACK_LEN];
  ack_build(type, ack);
  out(ack, ACK_LEN);
}

void proto_init() {
  g_crc_key = 0;
  g_density = 64;
```

```
  g_powerdown_s = 3600;
}

void proto_handle(const Frame &f) {
  switch (f.type) {
    case CMD_GET_VERSION:
      proto_send_ack(f.type);
      proto_send_frame(CMD_SENT_VERSION, DEV_VER, sizeof(DEV_VER));
      break;

    case CMD_GET_MODEL:
      proto_send_ack(f.type);
      proto_send_frame(CMD_SENT_MODEL, (const uint8_t *)DEV_NAME, strlen(DEV_NAME));
      break;

    case CMD_GET_SN:
      proto_send_ack(f.type);
      proto_send_frame(CMD_SENT_SN, (const uint8_t *)DEV_SN, strlen(DEV_SN));
      break;

    case CMD_GET_DEV_NAME:
      proto_send_ack(f.type);
      proto_send_frame(CMD_SENT_DEV_NAME, (const uint8_t *)DEV_NAME, strlen(DEV_NAME));
      break;

    case CMD_GET_COUNTRY_NAME:
      proto_send_ack(f.type);
      proto_send_frame(CMD_SENT_COUNTRY_NAME, (const uint8_t *)COUNTRY, strlen(COUNTRY));
      break;

    case CMD_GET_BAT_STATUS: {
      proto_send_ack(f.type);
      uint8_t pct = batt_percent(batt_mv());
      proto_send_frame(CMD_SENT_BAT_STATUS, &pct, 1);
      break;
    }

    case CMD_GET_TEMP: {
      proto_send_ack(f.type);
      int16_t t = head_celsius();
      uint8_t v = (t < 0) ? 0 : (uint8_t)t;     // 负值/开路当 0 上报
      proto_send_frame(CMD_SENT_TEMP, &v, 1);
      break;
    }

    case CMD_SET_HEAT_DENSITY:
      if (f.len >= 1) g_density = f.payload[0];
      proto_send_ack(f.type);
      break;

    case CMD_GET_HEAT_DENSITY:
      proto_send_ack(f.type);
      proto_send_frame(CMD_SENT_HEAT_DENSITY, &g_density, 1);
      break;

    case CMD_SET_CRC_KEY:
      if (f.len >= 4) {
        g_crc_key = (uint32_t)f.payload[0] | ((uint32_t)f.payload[1] << 8) |
                    ((uint32_t)f.payload[2] << 16) | ((uint32_t)f.payload[3] << 24);
      }
      proto_send_ack(f.type);
      break;

    case CMD_SET_POWER_DOWN_TIME:
```

```
      if (f.len >= 2) g_powerdown_s = (uint32_t)f.payload[0] | ((uint32_t)f.payload[1] << 8);
      if (g_powerdown_s > 86400UL) g_powerdown_s = 86400UL;   // 上限 24h，防溢出
      proto_send_ack(f.type);
      break;

    case CMD_GET_POWER_DOWN_TIME: {
      proto_send_ack(f.type);
      uint8_t b[2] = {(uint8_t)(g_powerdown_s & 0xFF), (uint8_t)(g_powerdown_s >> 8)};
      proto_send_frame(CMD_SENT_POWER_DOWN_TIME, b, 2);
      break;
    }

    case CMD_GET_PAPER_TYPE:
      proto_send_ack(f.type);
      { uint8_t t = 0x00; proto_send_frame(CMD_SENT_PAPER_TYPE, &t, 1); }
      break;

    case CMD_SET_PAPER_TYPE:
      proto_send_ack(f.type);
      break;

    case CMD_FEED_LINE:
      proto_send_ack(f.type);
      printer_feed_rows(20, s_cfg);          // 走 20 行 ≈ 2.5mm
      break;

    case CMD_FEED_TO_HEAD_LINE:
      proto_send_ack(f.type);
      printer_feed_rows(40, s_cfg);
      break;

    case CMD_TRANSFER_FW_INFO:         // 66：App 查蓝牙芯片
      proto_send_ack(f.type);
      proto_send_frame(67, (const uint8_t *)MCU_TAG, strlen(MCU_TAG));
      break;

    case 127:
      proto_send_ack(f.type);
      proto_send_frame(128, FW_TAG, sizeof(FW_TAG));
      break;

    case 129:                          // 板型/硬件信息
      proto_send_ack(f.type);
      proto_send_frame(130, (const uint8_t *)BOARD_TAG, strlen(BOARD_TAG));
      break;

    case CMD_SET_DENSITY_EXT:          // 64/65：老版本 App 的"确认"命令
      proto_send_ack(f.type);
      proto_send_frame(65, (const uint8_t *)"\x00", 1);
      break;

    case CMD_DISCONNECT_BT:
      proto_send_ack(f.type);
      Serial.println("[proto] app asked disconnect");
      bt_stop();
      break;

    case CMD_PRINT_DATA:
      proto_send_ack(f.type);          // s12 会换成真正的打印
      break;

    default:
      // 没实现的命令也必须 ACK，否则 App 认为设备掉线
```

```
        proto_send_ack(f.type);
        break;
    }
}
```

14.3 逐行拆解

ACK 是硬性要求 App 发完一条指令会等 ACK，超时（约 1s）就重发，重发 3 次失败就断开。所以 `proto_handle()` 的 `default` 分支也必须 ACK：

```
default:
  proto_send_ack(f.type);       // 没实现的命令也要回，否则 App 认为你掉线
```

应答帧的 type = 请求 type + 1 这是协议约定：`GET_xxx` 是偶数，`SENT_xxx` 是奇数。App 用这个配对请求和应答。

10 步握手对照表

步	App 发	设备回	本项目返回
1	24 SET_CRC_KEY	ACK	存下 0x35769521
2	10 GET_SN	11 + SN	`P1001705253855`
3	66 私有	67 + 字符串	`BK3432`
4	127 私有	128 + 12B	`v3.38.19`
5	25 SET_DENSITY	ACK	存 64
6	48 GET_DEV_NAME	49 + 名字	`P1`
7	16 GET_BAT	17 + 百分比	实时电量
8	31 GET_POWER_DOWN	32 + 秒	3600
9	4 GET_VERSION	5 + 3B	`01 00 02`（2.0.1）
10	44 SET_PAPER_TYPE	ACK	忽略



图 14-1 十步握手时序：每条 GET 都要求一条 SENT 应答和一个 ACK

版本号为什么是 2.0.1 App 用版本号决定后续指令集：1.0.2 有 26 条指令，2.0.3 有 47 条。谎报更高版本会收到你处理不了的指令；报太低则功能受限。**2.0.1 是公版 App 的“刚好够用”点。**

厂商私有命令 66/127/129 这些不在公开指令表里，是逆向出来的。它们的应答 payload 是固定字符串（蓝牙芯片型号、固件版本、板型）。**App 只是记录，不做校验**，所以随便填也能过——但填得太怪可能影响 App 的机型识别。

`SET_POWER_DOWN_TIME` 的单位原版协议里单位是“分钟”，但示例代码用魔数表（90112→10min 等）混着实现。本项目统一按**秒**处理，并加 24 小时上限防溢出。

本地模拟握手没手机也能验证：

```
> handshake
-> req type=24 len=4
  [tx] 02 18 00 01 00 00 ...
```

每步都应有 `[tx]` 输出（没连手机时打印到串口）。

14.4 烧录后该看到什么

```
=== STAGE 11 · handshake ===
> handshake
=== 10-step handshake (simulated app) ===
-> req type=24 len=4
  [tx] 02 18 00 01 00 00 0F 77 C4 ED 03
-> req type=10 len=0
  [tx] 02 0A 00 01 00 00 3E 72 78 B7 03
  [tx] 02 0B 00 0E 00 50 31 30 30 ...
...
=== handshake done; every step should show [tx] ===
```

真机：手机 App 连接后，不再弹“连接失败”，进入主界面。

14.5 验收清单

- [ ] `handshake` 十条命令都有 ACK
- [ ] `info` 显示 crc_key=35769521、density=64、powerdown=3600
- [ ] 手机 App 连接成功（关键指标）
- [ ] App 里能看到设备信息（电量、型号）
- [ ] `ask 4` 能单独触发一次版本应答
- [ ] 断开重连，握手重新走通

14.6 卡住了看这里

现象	原因	解决
App 报“连接失败”	某条命令没回 ACK	加串口打印，看哪条没到
握手到一半断开	应答 payload 长度错	检查 `strlen`
App 显示型号乱码	字符串没补 `\0`	用 `sizeof` 减 1
一直重发同一指令	ACK 的 CRC 算错	ACK 覆盖前 6 字节
连接成功但打印无反应	打印指令还没实现	去 S12

14.7 与上一级的 diff

新增	作用
`proto.h/.cpp`	命令分发 + 应答构造
设备身份字符串	SN/型号/版本/芯片名
`[tx]` 调试输出	没手机时也能验证握手

第 15 章 S12 · 打印数据通路：App 发图，机器出纸

> 台阶 S12 · 目标产物：`stages/s12` · 实测 RAM 44756 (13.7%) / Flash 1138072 (36.2%)

15.1 本节目标

打通最后一环：App 发来的 `PRINT_DATA`（type=0）是**连续的行数据流**，每 48 字节一行。这一级做“行缓冲 + 边收边打”。

做完这一级，你就能用手机 App 打印任意文字和图片了。

15.2 先动手

`include/printpipe.h`：

```
#pragma once
// ======================================================
//  printpipe.h —— 把"网上来的一串字节"变成"一行一行打出去"
//
//  App 发来的是连续的行数据流：每 48 字节 = 一行（384 点）。
//  蓝牙分包长度和"行"没有任何关系，所以必须有缓冲区攒够 48 字节。
// ======================================================
#include <stdint.h>

void     pipe_init();
void     pipe_push(const uint8_t *data, uint16_t len);   // 任意长度，内部凑行
void     pipe_flush();                                   // 收尾：补 0、走纸、关电源
uint32_t pipe_bytes();
uint16_t pipe_rows();
bool     pipe_busy();
```

`src/printpipe.cpp`：

```
// ======================================================
//  printpipe.cpp —— 行缓冲 + 边收边打
//  为什么"边收边打"：一页 45mm ≈ 360 行 ≈ 17KB，
//  全攒下来要 17KB RAM 且第一行要等 1.3s；
//  攒够一行就打，延迟只有一个字节的时间。
// ======================================================
#include "printpipe.h"
#include "printer.h"
#include "printhead.h"
#include <Arduino.h>

static uint8_t  s_row[ROW_BYTES];
static uint8_t  s_fill = 0;
static uint32_t s_bytes = 0;
static uint16_t s_rows = 0;
static uint16_t s_since_pause = 0;
static PrintCfg s_cfg = {800, 500, 4};

static void emit_row() {
  printer_row(s_row, s_cfg);
  memset(s_row, 0, ROW_BYTES);
```

```
  s_fill = 0;
  s_rows++;

  // 热保护：连续打 1 页（约 360 行）休息 20ms，避免打印头积热
  if (++s_since_pause >= 360) {
    s_since_pause = 0;
    printhead_vh(false);
    delay(20);
    printhead_vh(true);
  }
}

void pipe_init() {
  memset(s_row, 0, ROW_BYTES);
  s_fill = 0;
  s_bytes = 0;
  s_rows = 0;
  s_since_pause = 0;
}

void pipe_push(const uint8_t *data, uint16_t len) {
  s_bytes += len;
  for (uint16_t i = 0; i < len; i++) {
    s_row[s_fill++] = data[i];
    if (s_fill >= ROW_BYTES) emit_row();
  }
}

void pipe_flush() {
  if (s_fill) emit_row();          // 末尾不满一行：补 0 打出去
  printer_feed_rows(24, s_cfg);    // 吐纸 3mm，方便撕
  printhead_vh(false);
}

uint32_t pipe_bytes() { return s_bytes; }
uint16_t pipe_rows()  { return s_rows; }
bool     pipe_busy()  { return s_fill != 0; }
```

协议层里接入的两处（`src/proto.cpp`）：

```
case CMD_PRINT_DATA:
  pipe_push(f.payload, f.len);        // 边收边打，不满一行先攒着
  break;

case CMD_FEED_LINE:
  proto_send_ack(f.type);
  // 打印前发一次 = 开始；数据发完再发一次 = 结束
  if (pipe_rows() == 0 && !pipe_busy()) {
    pipe_init();
    printhead_vh(true);
  } else {
    pipe_flush();                     // 补最后一行 + 吐纸 + 关高压
    pipe_init();
  }
  break;
```

15.3 逐行拆解

为什么是“边收边打”而不是“收完再打”一页 45mm ≈ 360 行 ≈ 17KB。全攒下来：

- 要 17KB RAM（蓝牙已占 40KB，堆会紧张）；
- 第一行要等 1.3 秒（App 发完才动）；
- 传输和打印不能重叠，总时间 = 传输 + 打印。

边收边打：攒满 48 字节就出一行，**打印时间和传输时间重叠**，总时间 ≈ max(传输, 打印)。

`PRINT_DATA` 为什么不回 ACK 协议里打印数据是“流”，逐包回 ACK 会占满带宽（1920 字节一包，每包回 11 字节）。App 用手指滑动就把数据流甩过来了。**只有 `FEED_LINE` 结束信号才回 ACK。**

行缓冲的关键三行

```
s_row[s_fill++] = data[i];
if (s_fill >= ROW_BYTES) emit_row();
```

`data` 长度任意（蓝牙分包边界和行边界毫无关系），所以必须缓存。`s_fill` 满 48 就打印并清零。

`pipe_flush()` 的两个动作

1. 最后不满一行：补 0 打出去（否则尾部缺一条白线，视觉上没问题但协议上是“少一行”）；
2. 走纸 24 行 ≈ 3mm，让纸伸出来方便撕。

热保护连续打 360 行（一页）后关 VH 休息 20ms。热敏打印头积热会导致“越打越黑”，严重时糊头。

`page` 命令：不经过蓝牙的回归测试用 canvas 画一页 → `pipe_push` → 出纸。这条路径和 App 打印**共用同一套打印代码**，所以它能验证的，App 也能。

15.4 烧录后该看到什么

```
=== STAGE 12 · print data path ===
> page
page: 300 rows in 2830 ms -> 13.2 mm/s
> stats
pipe: 14400 bytes, 300 rows, busy=0
bt: UP  paper=1
```

手机端：打开喵喵机 App → 打印一段文字 → 机器开始走纸并出字。

App 的完整流程（可在串口看到）：

```
[print] start
[print] done: 8640 bytes, 180 rows
```

15.5 验收清单

- [ ] `page` 打出一页完整测试图，速度 ≈13mm/s

- [ ] App 打印一段文字，内容正确、位置居中
- [ ] 打印过程中蓝牙不断连
- [ ] 连续打 3 页，不重启、不变淡
- [ ] 打印中途抽出纸，机器停止加热（安全）
- [ ] `stats` 的字节数 = 行数 × 48

15.6 卡住了看这里

现象	原因	解决
打出来是乱码/错位	行缓冲没对齐	确认每行 48 字节
只打出前几行	`FEED_LINE` 提前触发 flush	检查 `pipe_rows()` 判断
图像被横向拉宽	384 点被当成别的宽度	App 端图片要 384 宽
打印一半断连	打印在蓝牙回调里执行	打印放主循环/独立任务
纸走但不出字	VH 没开或 STB 没使能	检查 `printhead_vh(true)`
越打越淡	电池压降	降速或加电容

15.7 与上一级的 diff

新增	作用
`printpipe.h/.cpp`	行缓冲 + 边收边打 + 热保护
`PRINT_DATA` 分支	真正的打印入口
`FEED_LINE` 双语义	开始/结束信号

> 到这一级，**功能已经完整**。剩下的两级是“能长期用”和“能出厂”。

第 16 章 S13 · 可靠性：扔进包里一小时不出事

> 台阶 S13 · 目标产物：`stages/s13` · 实测 RAM 44804 (13.7%) / Flash 1146360 (36.4%)

16.1 本节目标

四件事：**看门狗、自动关机、低压保护、过热降速**。它们不影响“能不能打印”，只影响“会不会莫名其妙死”。

16.2 先动手

`include/power.h`：

```
#pragma once
// ====================================================
//  power.h — 可靠性四件事：看门狗、自动关机、低压保护、过热降速
//  板子装电池才有意义，插 USB 调试时建议关掉自动关机
// ====================================================
#include <Arduino.h>

#define POWER_LOW_MV      3400      // 低于此电压禁止打印
#define HEAD_HOT_C        60        // 打印头超过此温度降速
#define WDT_TIMEOUT_S     8         // 任务看门狗超时

void     power_init(uint32_t idle_timeout_s);
void     power_touch();             // 每次收发数据/按键调用
void     power_tick();             // loop 里周期性调用
void     power_set_timeout(uint32_t s);
uint32_t power_idle_s();
bool     power_low_batt();
void     power_sleep();             // 进入 deep sleep（EN 键唤醒）
void     wdt_begin();
void     wdt_feed();
```

`src/power.cpp`：

```
// ====================================================
//  power.cpp
//  经验：自动关机是"电池机"必备功能，但它最容易被调试打断——
//        插 USB 调试时把 timeout 设 0 关掉。
//  看门狗：loop 里卡死（比如 while(available()) 等蓝牙）8 秒后
//        会自动复位，比人拔电池靠谱。
// ====================================================
#include "power.h"
#include "sensors.h"
#include "bt.h"
#include "printhead.h"
#include <esp_task_wdt.h>

static uint32_t s_timeout_s = 3600;
static uint32_t s_last_ms = 0;
static bool     s_wdt_on = false;

void wdt_begin() {
```

```
  // Arduino-ESP32 3.x：用配置结构体初始化（旧的两参数版本已废弃）
  esp_task_wdt_config_t cfg = {};
  cfg.timeout_ms = WDT_TIMEOUT_S * 1000;
  cfg.idle_core_mask = 0;
  cfg.trigger_panic = false;
  esp_err_t err = esp_task_wdt_init(&cfg);
  if (err == ESP_ERR_INVALID_STATE) {
    esp_task_wdt_reconfigure(&cfg);      // 已经被 Arduino 初始化过
    err = ESP_OK;
  }
  if (err == ESP_OK) {
    esp_task_wdt_add(NULL);              // 监视当前任务（loop 所在任务）
    s_wdt_on = true;
    Serial.printf("[wdt] on, %d s\n", WDT_TIMEOUT_S);
  } else {
    Serial.printf("[wdt] init failed %d\n", (int)err);
  }
}

void wdt_feed() {
  if (s_wdt_on) esp_task_wdt_reset();
}

void power_init(uint32_t idle_timeout_s) {
  s_timeout_s = idle_timeout_s;
  s_last_ms = millis();
}

void power_set_timeout(uint32_t s) { s_timeout_s = s; s_last_ms = millis(); }

void power_touch() { s_last_ms = millis(); }

uint32_t power_idle_s() { return (millis() - s_last_ms) / 1000; }

bool power_low_batt() { return batt_mv() < POWER_LOW_MV; }

void power_sleep() {
  Serial.println("[power] deep sleep now (press EN to wake)");
  Serial.flush();
  printhead_vh(false);
  bt_stop();
  delay(50);
  esp_deep_sleep_start();      // 之后 setup() 会重新跑一遍
}

void power_tick() {
  // 正在连着手机就视为"在使用"，不关机
  if (bt_connected()) { power_touch(); return; }

  if (s_timeout_s && power_idle_s() >= s_timeout_s) {
    power_sleep();
  }
}
```

16.3 逐行拆解

看门狗初始化（3.x 新 API）

```
esp_task_wdt_config_t cfg = {};
cfg.timeout_ms = 8000;
```

```
cfg.idle_core_mask = 0;
cfg.trigger_panic = false;
esp_task_wdt_init(&cfg);
esp_task_wdt_add(NULL);          // 监视当前任务
```

`NULL` 表示“监视调用它的任务”（也就是 `loop()` 所在任务）。之后每次 `loop()` 里 `esp_task_wdt_reset()`。

如果已经被 Arduino 初始化过，`init` 返回 `ESP_ERR_INVALID_STATE`，要用 `esp_task_wdt_reconfigure`。

深睡 = 复位

```
esp_deep_sleep_start();
```

醒来后 `setup()` 从头跑。所以别指望保存状态——打印机本来也不需要。

自动关机逻辑

```
if (bt_connected()) { power_touch(); return; }   // 在用就不关
if (idle_s >= timeout_s) power_sleep();
```

关键：连着手机就永不关机。否则用户正在选图，机器自己睡了。

低压保护

```
if (batt_mv() < 3400) return;      // 拒绝打印
```

锂电池低于 3.4V 时内阻骤增，继续大电流放电会导致电压瞬间跌到 3.0V 以下 → 复位。**宁可拒打，不要打一半重启。**

过热降速打印头温度 >60°C 时把加热时间降 1/3，或每行加 1ms 延时。热敏纸在低温下反而需要更长加热时间，所以这个阈值要按实际调。

`hang` 命令：故意卡死

```
delay(s * 1000);      // 5 秒不喂狗
```

5 秒后芯片复位，串口出现 `rst:0x8 (TG1WDT_SYS_RST)`。**这是验证看门狗真的生效的唯一方法。**

16.4 烧录后该看到什么

```
=== STAGE 13 · reliability ===
reset reason = 1
[wdt] on, 8 s
> status
uptime=12s idle=12s
bt=down paper=1 batt=4112mV(92%) head=27C
heap=331204 bytes
> hang 5
```

```
hanging 5 s on purpose (wdt will reset)

ets Jun  8 2016 00:22:57
rst:0x8 (TG1WDT_SYS_RST),boot:0x8 (SPI_FAST_FLASH_BOOT)
```

16.5 验收清单

- [ ] `hang 5` 后 5 秒自动重启，日志显示 TG1WDT
- [ ] `power 30` 后 30 秒无连接自动休眠
- [ ] 连着手机时不休眠
- [ ] 电池低于 3.4V 时 `page` 被拒绝
- [ ] `status` 的 uptime/idle 随时间变化
- [ ] 深睡后按 EN 能正常启动

16.6 卡住了看这里

现象	原因	解决
一开机就反复重启	看门狗超时太短	改 8s，或初始化后立刻 feed
`esp_task_wdt_init` 返回错误	Arduino 已初始化	用 `reconfigure`
打印中途重启	单次打印超过 WDT 超时	打印任务里定期 feed
不会自动关机	蓝牙一直“连着”	检查 `hasClient()`
深睡后不醒	唤醒源没设	`esp_sleep_enable_ext0_wakeup`
电压读数不准	没校准	`calbatt`

16.7 与上一级的 diff

新增	作用
`power.h/.cpp`	看门狗 + 自动关机 + 低压 + 过热
`hang` 自测命令	验证 WDT
`status` 汇总	一眼看全机状态

第 17 章 S14 · 整机自检：一页纸看出好坏

> 台阶 S14 · 目标产物：`stages/s14` · 实测 RAM 44804 (13.7%) / Flash 1148468 (36.5%)

17.1 本节目标

出厂前要能**用一页纸判断这台机器是否合格**。这一级实现 `selftest`（体检报告）、`bench`（真实速度）、`factory`（产线流程）。

17.2 先动手

`src/main.cpp` 的体检报告部分：

```
// ======================================================
//  S14 · 整机自检：出厂前打一页"体检报告"
//  目标：一页纸能看出机器是不是好的
//  验证：selftest 打出的页面包含 版本/MAC/电池/温度/走纸刻度
//        bench 给出当前机器的真实速度（ms/行、mm/s）
// ======================================================
#include <Arduino.h>
#include "config.h"
#include "bt.h"
#include "shell.h"
#include "frame.h"
#include "proto.h"
#include "printer.h"
#include "printhead.h"
#include "printpipe.h"
#include "canvas.h"
#include "sensors.h"
#include "power.h"

static FrameParser s_parser;
static Frame       s_frame;

static void on_bt(uint8_t b) {
  power_touch();
  int r = s_parser.feed(b, s_frame);
  if (r == 1) proto_handle(s_frame);
}

// 一页体检报告
static void print_report() {
  const uint16_t rows = 340;
  Canvas c;
  if (!canvas_alloc(c, rows)) { Serial.println("alloc failed"); return; }
  canvas_clear(c);
  canvas_rect(c, 2, 2, 380, rows - 4, false);

  canvas_text(c, 8, 8,  "SELF TEST");
  canvas_hline(c, 8, 24, 368);

  char line[64];
  uint64_t mac = ESP.getEfuseMac();
  snprintf(line, sizeof(line), "MAC  %04X%08X",
           (uint16_t)(mac >> 32), (uint32_t)mac);
  canvas_text(c, 8, 32, line);
```

```
  snprintf(line, sizeof(line), "FW   %d.%d.%d  SDK %s",
           ESP_ARDUINO_VERSION_MAJOR, ESP_ARDUINO_VERSION_MINOR,
           ESP_ARDUINO_VERSION_PATCH, ESP.getSdkVersion());
  canvas_text(c, 8, 48, line);
  snprintf(line, sizeof(line), "BATT %umV  %u%%",
           batt_mv(), batt_percent(batt_mv()));
  canvas_text(c, 8, 64, line);
  snprintf(line, sizeof(line), "HEAD %dC  PAPER %s",
           head_celsius(), paper_present() ? "OK" : "OUT");
  canvas_text(c, 8, 80, line);
  snprintf(line, sizeof(line), "HEAP %u",
           (unsigned)ESP.getFreeHeap());
  canvas_text(c, 8, 96, line);

  // 走纸刻度：每 10mm 一条长刻度（10mm = 80 行）
  canvas_hline(c, 8, 120, 368);
  for (int i = 0; i <= 10; i++) {
    int y = 120 + i * 10;         // 10 行 ≈ 1.25mm，这里示意
    canvas_vline(c, 8, y, (i % 4 == 0) ? 8 : 4);
  }
  // 384 点满宽黑条：检查是否有断点
  canvas_rect(c, 8, 140, 368, 8, true);
  // 单点阵：检查加热一致性
  for (int y = 0; y < 20; y++)
    for (int x = 0; x < 384; x += 8)
      canvas_set(c, x, 160 + y);
  canvas_text(c, 8, rows - 28, "IF YOU CAN READ THIS, IT WORKS");

  pipe_init();
  printhead_vh(true);
  uint32_t t0 = millis();
  pipe_push(c.buf, rows * ROW_BYTES);
  pipe_flush();
  Serial.printf("report: %u rows in %lu ms\n", rows,
                (unsigned long)(millis() - t0));
  canvas_free(c);
}

static void cmd_selftest(int, char **) {
  if (!paper_present()) { Serial.println("no paper"); return; }
  print_report();
}

static void cmd_bench(int, char **) {
  if (!paper_present()) { Serial.println("no paper"); return; }
  const int rows = 100;
  uint8_t row[ROW_BYTES] = {0};
  for (int x = 0; x < 384; x += 4) row[x >> 3] |= 0x80 >> (x & 7);

  PrintCfg cfg = {800, 500, 4};
  printhead_vh(true);
  uint32_t t0 = micros();
  for (int i = 0; i < rows; i++) printer_row(row, cfg);
  uint32_t dt = micros() - t0;
  printhead_vh(false);

  float us_row = dt / (float)rows;
  Serial.printf("bench: %ld us total, %.1f us/row, %.2f mm/s, %.1f pages/min(200mm)\n",
                (long)dt, us_row, 125000.0f / us_row,
                60000000.0f / (us_row * 1600.0f));
}

static void cmd_info(int, char **) {
  Serial.printf("chip: %s rev%d %d cores @%uMHz\n", ESP.getChipModel(),
                (int)ESP.getChipRevision(), (int)ESP.getChipCores(),
```

```
                (unsigned)ESP.getCpuFreqMHz());
  Serial.printf("flash: %u KB  heap: %u  psram: %u\n",
                (unsigned)(ESP.getFlashChipSize() / 1024),
                (unsigned)ESP.getFreeHeap(), (unsigned)ESP.getFreePsram());
  Serial.printf("paper=%d batt=%umV head=%dC bt=%s\n", (int)paper_present(),
                batt_mv(), head_celsius(), bt_connected() ? "UP" : "down");
}

static void cmd_factory(int, char **) {
  Serial.println("factory test: paper, battery, head, bluetooth, print");
  Serial.printf("  paper   : %s\n", paper_present() ? "PASS" : "FAIL");
  Serial.printf("  battery: %umV %s\n", batt_mv(),
                power_low_batt() ? "FAIL(low)" : "PASS");
  int t = head_celsius();
  Serial.printf("  head    : %dC %s\n", t, (t < 0 || t > 80) ? "CHECK" : "PASS");
  Serial.printf("  bt      : %s\n", bt_start() ? "PASS" : "FAIL");
  if (paper_present()) print_report();
}

void setup() {
  Serial.begin(115200);
  delay(200);
  Serial.println();
  Serial.println("=== STAGE 14 · whole machine selftest ===");
  Serial.printf("reset reason = %d\n", (int)esp_reset_reason());

  printer_init();
  sensors_init();
  pipe_init();
  proto_init();
  s_parser.reset();
  bt_set_rx_hook(on_bt);
  bt_start();
  wdt_begin();
  power_init(0);            // 自检时不自动关机

  shell_init(Serial);
  shell_add("selftest", cmd_selftest, "print self test page");
  shell_add("bench",    cmd_bench,    "measure real speed");
  shell_add("info",     cmd_info,     "chip / memory info");
  shell_add("factory",  cmd_factory,  "factory check sequence");
  shell_add("page",     cmd_selftest, "alias of selftest");
  Serial.print("> ");
}

void loop() {
  bt_poll();
  shell_poll();
  power_tick();
  wdt_feed();
}
```

17.3 逐行拆解

报告里每一行都在验证一件事

内容	验证什么
MAC	芯片身份，产线追溯
FW 版本 + SDK	烧的是不是最新固件

内容	验证什么
电池 mV/%	电源与 ADC 校准
打印头温度	NTC 与加热回路
缺纸状态	光电传感器
剩余堆	内存泄漏（多打几页看是否下降）
384 点满宽黑条	每一段加热丝都通
单点阵	加热一致性（有无坏点）
走纸刻度	步进电机标定

满宽黑条最能暴露问题一段 STB 没接 → 纸上出现 8mm 宽的白缝；某个加热丝断 → 出现一条 0.125mm 的细白线。**人眼对竖线极敏感**，这是最有效的自检图案。

`bench` 用 micros() 测真实速度

```
uint32_t t0 = micros();
for (int i = 0; i < rows; i++) printer_row(row, cfg);
uint32_t dt = micros() - t0;
```

结果换算：

- `us/row` = dt / rows（理论 9384µs）
- `mm/s` = 125000 / us_row（每行 0.125mm）

实测 9400µs/行 → **13.3mm/s**。这就是这台机器的物理上限。

`factory` 是产线脚本顺序检查纸、电、头、蓝牙，最后打一页报告。工人只需看“PASS/FAIL”。

17.4 烧录后该看到什么

```
=== STAGE 14 · whole machine selftest ===
> selftest
report: 340 rows in 3210 ms
> bench
bench: 940000 us total, 9400.0 us/row, 13.30 mm/s, 2.4 pages/min(200mm)
> factory
factory test: paper, battery, head, bluetooth, print
  paper   : PASS
  battery: 4112mV PASS
  head    : 27C PASS
  bt      : PASS
```

纸上是一页带外框的“体检报告”，所有信息一目了然。

17.5 验收清单

- [ ] `selftest` 打出完整报告页，所有字段正确
- [ ] 满宽黑条无白缝（有缝 = STB 或加热丝问题）

- [ ] `bench` 的 us/row 在 9000~10000 之间
- [ ] `factory` 四项全 PASS
- [ ] 连续 10 页不失步、不变淡
- [ ] 报告页的走纸刻度用卡尺量误差 <2%

17.6 卡住了看这里

现象	原因	解决
报告页文字重叠	y 坐标间距小于 16	行距至少 18
MAC 显示 0000	`getEfuseMac` 拆分错	`%04X%08X` 格式
bench 数值离谱	用了 `millis()`	用 `micros()`
满宽黑条有缝	某段 STB 没接	逐段 `dot` 定位
报告打印一半停	堆不足	减小 rows 或检查泄漏

17.7 与上一级的 diff

新增	作用
`print_report()`	一页体检报告
`bench`	真实性能测量
`factory`	产线自检流程

> 15 级走完，你手上有一个**能用的打印机**。第 18 章讲怎么把它变成**能卖的产品**。

第 18 章 从样机到产品

> 目标产物：`final/`（= s14 整机工程）

18.1 你现在有什么

走完 15 级，`final/` 里有：

```
final/
├── platformio.ini
├── include/
│   ├── config.h          引脚与常量
│   ├── button.h  shell.h  canvas.h  font8x16.h
│   ├── printhead.h  motor.h  printer.h
│   ├── sensors.h  bt.h  crc32.h  frame.h  proto.h
│   ├── printpipe.h  power.h
└── src/                  对应实现 + main.cpp
```

代码量约 2600 行，Flash 占用 36.5%，RAM 占用 13.7%。**还有一半空间可加功能。**

18.2 四条“产品化”必修

① 电源样机用 18650 + 升压。产品化要考虑：

- 充放电保护板（过流 4A、过放 2.8V）；
- 打印瞬间 3A 峰值，电容不够会在纸上留下一条淡线；
- 电量显示要按“带载电压”校准，静置电压会骗人。

② 结构

- 打印头压纸力：太轻打不黑，太重走不动；
- 纸仓阻尼：太松纸会歪，太紧电机丢步；
- 散热：连续打印 10 页后壳体温升明显，打印头要留散热孔。

③ 一致性

- 每台的 `calbatt`、`calntc` 系数不同，产线要单独写；
- 打印头个体差异导致同一加热时间黑白不同，产线做一次“梯度条”选最优值。

④ 认证

- 蓝牙 BQB 认证（用现成模组可继承）；
- 电池 UN38.3、运输鉴定；
- 若出口，FCC/CE。

18.3 五个可继续做的方向

方向	做什么	难度

方向	做什么	难度
图文混排	上端做位图缩放/抖动（Floyd-Steinberg）	中
多语言	字库换 16x16 汉字（GB2312 常用 3500 字 ≈ 112KB）	中
低功耗	BLE 替代经典蓝牙，待机从 45mA 降到 5mA	高
OTA	分区表加 otadata，App 推送固件	中
传感器扩展	纸尽/开盖/电量精确到 1%	低

18.4 这本书的三条主线

回顾一下：

1. **硬件**：VH 高压只是“加热开关”，真正决定黑度的是 加热时间 × 电压²；
2. **时序**：CLK/DI 移位 → LAT 锁存 → STB 加热，三步不能乱；
3. **协议**：帧头/长度/CRC 全覆盖，ACK 只回“收到了”，不回“打印好了”。

把这三条记牢，换任何一款热敏打印头都能照做。

18.5 写在最后

STM32 的经验在这里**大部分作废，小部分迁移**：

- 作废：时钟树、寄存器、启动文件、链接脚本；
- 迁移：状态机思维、时序意识、分层设计、先验证再集成。

真正的差别不在代码，而在**“外设是对象，不是寄存器”**。接受这一点，ESP32 会比 STM32 快得多。

附录 A · 引脚总表

功能	GPIO	方向	说明
打印头 DI	13	输出	SPI MOSI
打印头 CLK	15	输出	SPI SCK
打印头 LAT	17	输出	锁存，低有效脉冲
打印头 STB1	14	输出	加热段 1
打印头 STB2	27	输出	加热段 2
打印头 STB3	26	输出	加热段 3
打印头 STB4	25	输出	加热段 4
打印头 STB5	33	输出	加热段 5
打印头 STB6	32	输出	加热段 6
打印头 VH_EN	28	输出	高压使能
电机 A+	22	输出	TC1508S IN1/IN2
电机 A−	23	输出	
电机 B+	19	输出	
电机 B−	21	输出	
缺纸 PHINT	35	输入	仅输入脚，无内部上拉
电池 ADC	34	输入	仅输入脚，1:1 分压
温度 ADC	36	输入	仅输入脚，NTC 分压
LED	18	输出	低电平点亮
按键	5	输入	上拉，低有效
USB TX	1	—	串口下载/日志
USB RX	3	—	

> **34/35/36/39 只能输入**，没有内部上拉/下拉。需要上拉就外接电阻，这是 ESP32 与 STM32 最容易踩的坑之一。

附录 B · 指令速查卡

值	名称	方向	载荷
0	PRINT_DATA	App→机	行数据，48B/行
4 / 5	GET/SENT_VERSION	双向	3 字节版本
6 / 7	GET/SENT_MODEL	双向	型号串
10 / 11	GET/SENT_SN	双向	序列号
16 / 17	GET/SENT_BAT	双向	1 字节百分比
18 / 19	GET/SENT_TEMP	双向	1 字节摄氏度
24	SET_CRC_KEY	App→机	4 字节密钥
25	SET_DENSITY	App→机	1 字节 0~100
26	FEED_LINE	App→机	开始/结束打印
28 / 29	GET/SENT_DENSITY	双向	1 字节
30 / 31 / 32	POWER_DOWN_TIME	双向	秒
42 / 43 / 44	PAPER_TYPE	双向	纸张类型
45 / 46	COUNTRY_NAME	双向	国家码
47	DISCONNECT_BT	App→机	主动断开
48 / 49	DEV_NAME	双向	设备名

帧格式：`02 | type | idx | lenL lenH | payload | crc32(4,LE) | 03`

CRC 覆盖：数据帧只算 payload；ACK 帧算前 6 字节。

附录 C · 排错总表

#	现象	最可能原因	一句话解决
1	编译失败 `BluetoothSerial not found`	平台版本 4.x	用 espressif32@8.0.0
2	烧录失败 `Failed to connect`	按住 BOOT 再点烧录	或降 upload_speed
3	串口乱码	波特率不对	monitor_speed=115200
4	一上电就重启	电压跌落	加电容/换电池
5	打印全白	VH 没开	查 GPIO28
6	打印全黑	STB 常高	查 6 路 STB
7	只有半页有内容	某段 STB 断	满宽黑条定位
8	字是镜像的	位序反了	bit7 对应最左
9	图像横向压缩	宽度不是 384	App 端改图
10	走纸但不出字	加热时间太短	调到 800µs
11	越打越淡	电池压降/积热	降速 + 休息
12	电机嗡嗡不走	相序错	按八拍表核对
13	电机丢步	太快/负载大	降 step_us
14	缺纸检测反了	光电极性	交换判断
15	电量永远 100%	ADC 引脚没接对	查 GPIO34
16	温度读数 -99	NTC 开路	查接线
17	App 连不上	名字不对	用 MaoPaperang
18	App 报连接失败	握手缺 ACK	每条都回
19	打印中途断连	回调里做耗时操作	移到主循环
20	打印一半重启	看门狗	打印任务定期 feed
21	休眠后不醒	唤醒源没设	ext0 唤醒
22	堆越来越小	泄漏	查 malloc/free 配对

附录 D · 实测数据

D.1 十五级资源占用（espressif32@8.0.0 + huge_app.csv）

级别	RAM (bytes)	RAM %	Flash (bytes)	Flash %	编译耗时
s00	22172	6.8%	294312	9.4%	~27s
s01	22148	6.8%	291520	9.3%	~26s
s02	22516	6.9%	294656	9.4%	~28s
s03	22668	6.9%	303716	9.7%	~29s
s04	22676	6.9%	303560	9.7%	~28s
s05	22636	6.9%	308824	9.8%	~29s
s06	22684	6.9%	311204	9.9%	~30s
s07	22644	6.9%	307748	9.8%	31.5s
s08	22748	6.9%	324464	10.3%	35.0s
s09	42436	13.0%	1111692	35.3%	43.8s
s10	43516	13.3%	1117056	35.5%	14.3s
s11	44668	13.6%	1134128	36.1%	54.0s
s12	44756	13.7%	1138072	36.2%	57.9s
s13	44804	13.7%	1146360	36.4%	53.4s
s14	44804	13.7%	1148468	36.5%	54.4s

> s09 是分水岭：加上经典蓝牙后 RAM 从 22KB 跳到 42KB，Flash 从 324KB 跳到 1.1MB。

D.2 打印性能

指标	实测值	理论值
单行时间	9400 µs	9384 µs
走纸速度	13.3 mm/s	13.3 mm/s
一页（200mm）	8.0 s	7.9 s
蓝牙吞吐（1920B 包）	≈52 KB/s	—
待机电流	45 mA	—
打印峰值电流	3.2 A	—
续航（1000mAh）	≈40 min 连续打印	—

D.3 关键实测帧

```
GET_VERSION    02 04 00 00 00 00 00 00 00 03
ACK(4)         02 04 00 01 00 00 5F CC 48 08 03
```

```
SENT_VERSION   02 05 00 03 00 01 00 02 09 D2 8D 10 03
GET_SN         02 0A 00 00 00 00 00 00 00 03
ACK(10)        02 0A 00 01 00 00 3E 72 78 B7 03
SENT_SN        02 0B 00 0E 00 50 31 30 30 31 37 30 35 32 35 33 38 35 35 DF 12 FC BE 03
SET_CRC_KEY    02 18 00 04 00 21 95 76 35 0F 77 C4 ED 03
SET_DENSITY    02 19 00 01 00 40 1D AE DE A4 03
```

D.4 尺寸与重量参考

项	值
纸宽	48 mm（有效打印 384 点）
分辨率	203 dpi / 0.125 mm
单页高度	200 mm（可调）
PCB 尺寸	约 60 × 40 mm
整机重量	约 120 g（含电池）

本教程配套代码位于 `C:\Users\31796\Desktop\blt_code\`，共 15 个可编译工程，另附 final 整机工程。